計算機程式期末報告

沈威宇

2025 年 12 月 27 日

目錄

第一節 專題整體 1

一、 1-1-1 專題整體介紹與說明文案與 1-1-2 功能設計 1

二、 1-1-3 美工介面設計 1

三、 1-2-1 程式架構 2

(一) 核心類別模組 2

(二) 主程式架構（BeatRunner.cpp） 3

四、 1-3-1 心得與建議 5

(一) 心得 6

(二) 建議 6

第二節 個人程式設計說明 7

一、 沈威宇撰寫的部分 7

二、 2-A-1-1 描述 class 之間的關係 9

(一) include/tuple.hpp 9

(二) 其他 mystd 10

(三) mystd 以外（僅列沈威宇撰寫部分） 11

三、 2-A-1-2 畫出 class 之間的關係圖 11

四、 2-A-3-1 public 變數使用與原因 11

五、 2-A-3-2 private 變數使用與原因 12

六、 2-A-3-3 protected 變數使用與原因 12

七、 2-A-3-4 friend 使用與原因 13

八、 2-A-3-5 class composition 13

(一) Resource Acquisition Is Initialization (RAII) 13

(二) 專題使用 13

九、 2-B-1-1 建構子、複製建構子、解構子 14

(一) include/vector.hpp 14

(二) include/tuple.hpp 14

(三) include/priority_queue.hpp 20

(四) include/circulate.hpp 20

(五) Game.hpp 21

(六) Renderer.hpp 21

十、 2-B-1-2 指標成員處理 21
(一) iterator 成員處理一併說明 21
(二) Exception Safety Guarantees 21
(三) include/vector.hpp 21
(四) include/circulate.hpp 38
(五) BeatRunner.cpp（僅列沈威宇撰寫部分） 41
十一、 2-B-1-3 建構子、複製建構子、解構子特殊語法 42
(一) Rule of Three / Five / Zero 42
(二) Design Philosophy of STL Container Constructors 42
(三) include/vector.hpp 的 compile-time branching with if constexpr 優化 43
十二、 2-B-3-1-1 指出組裝關係（僅列沈威宇撰寫部分） 43
十三、 2-B-3-1-2 指出繼承關係（僅列沈威宇撰寫部分） 44
十四、 2-B-3-2 指標操作之巧思 45
十五、 2-B-3-3 功能巧思 45
(一) 判定旗標 45
(二) 模組註冊系統 45
(三) 紋理快取儲存 46
(四) 誤差修正系統 47
十六、 2-C-1-1 Operator Overloading 實作與呼叫位置 48
(一) allocator operator== 48
(二) array operator[] 49
(三) array operator== 與 operator<=> 49
(四) ignore_t operator= 50
(五) priority_queue operator= 50
(六) _tuple_impl 與 tuple operator= 50
(七) tuple operator== 與 operator<=> 52
(八) vector operator= 52
(九) vector operator[] 54
(十) vector operator== 與 operator<=> 55
(十一) _circulate_iterator and _circulate_const_iterator operators 56
(十二) circulate operator= 58
(十三) circulate operator[] 58

(十四) circulate operator== 59
(十五) Effect operator< 59
十七、 2-C-1-2 Operator Overloading 語法與使用方式描述與範例 59
(一) allocator operator== 59
(二) array operator[] 60
(三) array operator== 與 operator<=> 60
(四) ignore_t operator= 60
(五) priority_queue operator= 61
(六) _tuple_impl operator= 61
(七) tuple operator= 62
(八) tuple operator== 與 operator<=> 63
(九) vector operator= 63
(十) vector operator[] 64
(十一) vector operator== 與 operator<=> 64
(十二) _circulate_iterator and _circulate_const_iterator operators 65
(十三) circulate operator= 65
(十四) circulate operator[] 65
(十五) circulate operator== 66
(十六) Effect operator< 66
十八、 2-C-1-3 Operator Overloading 演算法設計 66
(一) 總論 66
(二) Argument-Dependent Lookup (ADL) 66
(三) Structured Bindings 66
(四) 2-C-1-4 Operator Overloading 有無的差異 67
(五) 2-C-1-5 Operator Overloading 巧思 67
(六) vector operator= 優化 67
(七) _circulate_iterator and _circulate_const_iterator operators 67
(八) tuple operator== 與 operator<=> 67
十九、 2-C-2-1 static member 宣告、2-C-3-2 static member 程式碼位置 68
(一) include/allocator.hpp 68
(二) include/tuple.hpp 69
(三) mods/GameOfLife.hpp 70

(四) Renderer.hpp 70
(五) BeatRunner.cpp 70
(六) Mods.hpp 71
二十、 2-C-2-3 static member 用途 71
(一) Static Initialization Order Fiasco (SIOF) 71
(二) 型別層級（type-level）的工具函式 71
(三) 與物件成員無關的純工具函式 72
(四) 跨物件共享的狀態或設定 72
(五) 區域內部的延遲初始化與狀態保存 72
(六) 延遲建立且全程共享的容器 72
二十一、 2-C-2-4 static member 有無的差異 72
二十二、 2-C-3-1 const member 宣告、2-C-3-2 const member 程式碼位置（僅列沈威宇撰寫部分） 73
(一) include/allocator.hpp 73
(二) include/array.hpp 73
(三) include/vector.hpp 74
(四) include/circulate.hpp 75
(五) include/ignore.hpp 77
(六) include/priority_queue.hpp 77
(七) include/tuple.hpp 77
(八) mods/GameOfLife.hpp 78
(九) Game.hpp 78
二十三、 2-C-3-3 const member 用途 79
(一) 支援 const 版本的物件 79
(二) 保證不可變的值封裝 79
(三) 常數定義 79
二十四、 2-C-3-4 const member 有無的差異 79
二十五、 2-C-4-1 polymorphism 宣告 79
(一) include/algorithm-copy.hpp 80
(二) include/algorithm-find.hpp 80
(三) include/algorithm-heap.hpp 80
(四) include/algorithm-remove.hpp 82

(五) include/allocator.hpp 82
(六) include/array.hpp 88
(七) include/circulate.hpp 89
(八) include/ignore.hpp 90
(九) include/priority_queue.hpp 90
(十) include/qsort.hpp 92
(十一) include/range-access.hpp 92
(十二) include/swap.hpp 94
(十三) include/tuple.hpp 95
(十四) include/vector.hpp 106
二十六、 2-C-4-2 polymorphism 成員 106
(一) include/allocator.hpp 106
(二) include/circulate.hpp 106
(三) include/priority_queue.hpp 107
(四) include/tuple.hpp 107
(五) include/vector.hpp 107
(六) Game.hpp 107
(七) Renderer.hpp 108
(八) mods/GameOfLife.hpp 108
(九) Mods.hpp 108
二十七、 2-C-4-3 polymorphism 相關的 class 或 function 間的對應關係 108
(一) Substitution Failure Is Not An Error (SFINAE) 108
(二) Concepts 108
(三) Class Template Argument Deduction (CTAD) 110
(四) Empty Base Optimization (EBO) 111
(五) Template Compile-Time Recursion 111
(六) std::declval template 111
(七) Ref-Qualified Member Functions 111
(八) decltype 112
(九) Curiously Recurring Template Pattern (CRTP) 112
(十) get functions in include/tuple.hpp 112

二十八、 2-D-1-1 演算法設計 115
(一) Heap 與 priority_queue 資料結構與演算法 115
(二) Quick sort 演算法 116
二十九、 2-D-1-2 程式執行效率優化 118
(一) Compile-time branching with if constexpr in pointer and iterator operations . . 118
(二) constexpr function 119
(三) Compile-time polymorphism-related optimization 121
(四) Named Return Value Optimization (NRVO) 121
(五) no_unique_address attribute 121
三十、 2-D-1-3 程式碼管理版本控制 121
三十一、 2-D-2-1 美工細節 123
三十二、 2-D-2-2 音樂細節 128

第一節　專題整體

一、　1-1-1 專題整體介紹與說明文案與 1-1-2 功能設計

《Beat Runner》是一款結合下落式音樂遊戲與滑鼠角色控制的創新遊戲。開始介面的部分有三個按鍵—Settings 按鍵可以調整遊戲的音量大小、Songs 按鍵可以選擇遊玩的歌曲，其中包括適合新手的 Thefatrat 的 Unity，在遊戲中顯示為《Tmp》，以及難度較高的 Sakuzyo 的《Destr0yer》，而按下 Start 則會開始遊戲。

開始遊戲之後，首先會進入遊戲難度的調整，包含 Lanes 軌道數的調整、Fragments 可見列數的調整、以及 Mods 遊戲模組的調整。Lanes 可以調整遊戲的軌道數，但由於我們的譜面並非隨機生成而是連結外部、我們自己寫的.txt 檔案，時間關係並沒有寫 Lanes = 4 以外的譜面。Fragments 部分則可以調整可見列數，數值越大，可以越早看見 Notes 的出現，但音符更窄故可能較難判斷與判定線的距離。Mods 可以選擇不同模式的康威生命遊戲（Conway’s Game of Life）模組，遊戲中將會在 TapNote 上執行相應的康威生命遊戲，但目前結算時的準確度不會根據之調整分母。選擇完畢後會倒數 3 秒，以供玩家準備。

遊戲中的下落式的音符可分為兩種，第一種是由鍵盤操作的 Key note，又細分為紅色的 Tap note 與綠色的 Hold note。Tap note 玩家必須在正確的節奏點按下對應的按鍵，Hold note 玩家則須在期間內持續按住對應的按鍵以持續得分。以軌道數 Lanes = 4 為例，對應的按鍵即為 A、S、D、F，若軌道數目更多則依序對應 G、H 等右方的鍵盤按鍵。Tap note 根據玩家按下對應按鍵的時機，評分判定分為五個等級——Perfect、Great、Good、Bad 與 Miss，分別能獲得 100%、75%、50%、25%、0% 的分數，滿分為 1000 分。持續避免 Bad 或 Miss 可以累積 Combo 數。

第二種是是由滑鼠操縱角色碰觸的 Mouse note，分為紅色與綠色兩種，碰觸綠色音符可以持續獲得分數，最多 400 分。相反地，紅色的音符則會造成等量的扣分。滑鼠左右操作可以使角色在白色線上移動，讓玩家得以控制角色在觸碰綠色音符的同時避開紅色音符。遊戲過程中，鍵盤的操作與滑鼠的移動必須同時進行，考驗玩家的左右手協調與反應力。

在遊戲中，你可以按下 ESC 進入暫停畫面，在這裡你可以回到遊戲、開啟新遊戲與離開遊戲。遊戲結束後玩家可以得知得分、最高連擊數、準確度，以及各個判定標準的數量，玩家可以選擇重試或回到主選單。在遊戲的任何時候，都可以用 Ctrl + W 退出遊戲。

二、　1-1-3 美工介面設計

美工介面設計分工如下：

- 遊戲主選單、音量調整滑桿、選歌頁面、遊戲背景、Mouse notes，以及角色動畫由于昀辰設計。
- Tap notes、Hold notes、遊戲中的計分與資訊文字、各類特效、設定頁面、倒數動畫，以及康威生命遊戲模組設定頁面由沈威宇設計，將於 **2-D-2-1 美工細節** 詳述。
- 結算頁面由林孜謙設計。

三、 1-2-1 程式架構

(一) 核心類別模組

1. **Game（遊戲邏輯核心）**
   功能：管理音符判定、分數計算與連擊系統
   - 維護音符軌道（highway）與玩家按鍵狀態
   - 處理 Tap/Hold 音符的判定窗口（PERFECT/GREAT/GOOD/BAD/MISS）
   - 計算分數與連擊數，記錄各判定等級數量
   - 每個 fragment 時間更新一次遊戲狀態

   關鍵資料：
   - `highway`: 音符軌道（二維循環陣列）
   - `score, combo, maxCombo`: 分數與連擊
   - `perfectCount ~ missCount`: 各判定統計
2. **Renderer（渲染引擎）**
   功能：將遊戲狀態轉換為視覺呈現
   - 繪製音符軌道與音符（Tap/Hold/MouseNote）
   - 顯示判定特效（PERFECT/GREAT 等）
   - 渲染即時統計資訊（分數、連擊、FPS）
   - 使用紋理快取提升效能

   依賴關係：
   - 持有 `Game&` 引用，讀取遊戲狀態
   - 使用 `ChartParser` 取得 MouseNote 資料
3. **ChartParser（譜面解析器）**
   功能：讀取並解析譜面檔案
   - 解析 BPM、offset、音樂檔案路徑
   - 解析 KeyNote（鍵盤音符）與 MouseNote（滑鼠物件）
   - 支援不同密度（density）與和弦（chord）
   - 將解析結果排序並載入 `keyNotes` 陣列

   輸出格式範例：

```
&bpm=120
&offset=0
&fragments=4
&keynotes=
1, 2, 3, 4
&mousenotes=
G1, R2, G3
```

4. **MusicManager（音樂管理器）**
   功能：管理背景音樂與音效播放
   - 載入並播放音樂（使用 SDL_mixer）
   - 控制音量（音樂與音效分開控制）
   - 提供暫停、繼續、停止功能
   - 偵測音樂播放狀態
5. **Character（角色類別）**
   功能：管理玩家角色的顯示與碰撞
   - 根據滑鼠位置更新角色位置
   - 交替切換左右腳動畫
   - 提供碰撞矩形供 MouseNote 碰撞檢測
6. **game_priority_queue（優先佇列）**
   功能：繼承 `mystd::priority_queue`，暴露底層容器
   - 用於管理特效的時間排序（centerEffects）
   - 允許 Renderer 直接存取容器內容進行渲染

(二) 主程式架構（BeatRunner.cpp）

遊戲狀態機

```
enum class GameState {
  MAIN_MENU, SETTINGS, COUNTDOWN, GAME, PAUSE, RESULT
};
```

狀態流程：

- MAIN_MENU → SETTINGS
- SETTINGS → COUNTDOWN
- COUNTDOWN → GAME
- GAME → PAUSE or RESULT
- PAUSE → GAME or SETTINGS
- RESULT → SETTINGS or MAIN_MENU

主要流程

1. **初始化階段**
   - 初始化 SDL、SDL_ttf、SDL_image、SDL_mixer
   - 載入字型與主選單紋理
   - 建立視窗與渲染器

- 設定初始音量

2. **主選單（MAIN_MENU）**
   - 顯示 Logo 與按鈕（Start/Songs/Settings）
   - 處理音量滑桿拖曳
   - 選歌介面（ChooseSong）：顯示歌曲封面，選擇譜面檔案
3. **遊戲設定（SETTINGS）**
   - 調整軌道數（LANES）、可見片段數（FRAGMENTS）及遊戲模組（MOD）
   - 載入譜面檔案（ChartParser）
   - 計算每個 fragment 精確時間（exactFragmentMs）
   - 建立 `Game` 與 `Renderer` 物件
4. **倒數計時（COUNTDOWN）**
   - 顯示 3、2、1、GO! 動畫
   - 記錄遊戲開始時間（gameStartTime）
   - 開始播放音樂
5. **遊戲進行（GAME）**
   - 計算 `offsetMs = currentTime - lastFragmentTime`
   - 當 `offsetMs >= MS_PER_FRAGMENT` 時：
     - `game->loadFragment()` 載入新音符
     - `adjustGameStartTime()` 修正時間誤差
     - 更新 `lastFragmentTime`
6. **輸入處理**
   - 鍵盤按鍵 → `game->keyPressed()/keyReleased()`
   - 滑鼠移動 → 更新角色位置
   - ESC/P → 暫停畫面
7. **碰撞檢測**
   - 取得可見 MouseNote
   - 檢查角色碰撞矩形重疊
   - 綠色加分、紅色扣分（`game->addMouseScore()`）
8. **渲染**
   - 繪製背景
   - `gameRenderer->render()` 繪製軌道與音符

- 繪製角色

9. **結束判定（RESULT）**
    - 音樂停止 → 記錄 `musicEndTime`
    - 延遲 2 秒 → 收集結算資料
    - 顯示總分、Max Combo、準確率及各判定等級數量
10. **暫停（PAUSE）**
    - 顯示半透明暗化背景
    - 提供 Resume/New Game/Exit 選項

關鍵機制

- 誤差修正系統：

```
adjustGameStartTime(currentTime, gameStartTime, exactFragmentMs, holdPressedTime)
```

  - 累積 `actualDelta - exactFragmentMs` 的誤差
  - 當誤差 ≥ 1ms 時調整 `gameStartTime`
  - 確保音符與音樂同步
- 判定系統：
  - Tap 音符：根據按鍵時間判定
  - Hold 音符：根據按住時間長度判定
  - 太早按：允許提前 0.4 個 fragment 判定
- 資料流向：

```
ChartParser → keyNotes → Game.notes → Game.loadFragment() → Game.highway → Renderer.render() → 螢幕
```

模組依賴關係

```
BeatRunner.cpp
  Game
  Renderer (依賴 Game)
  ChartParser
  MusicManager
  Character
  Mods
```

四、 1-3-1 心得與建議

在本專題《Beat Runner》的開發過程中，我獲得了寶貴的程式設計、遊戲開發以及專案管理經驗，也對團隊協作、版本控制與跨平台開發有了更深入的理解。

(一) 心得

- **C++20 STL 實作：**本專題中，我以 `namespace mystd` 實作了 STL 函式庫的部分及其他仿 STL 風格的演算法，例如 `allocator`、`allocator_traits` 等記憶體管理，`vector`、`tuple`、`array` 等容器，`priority_queue`、`circulate` 等 container adapter，以及 `algorithm-heap`、`qsort` 等演算法。透過這些實作，我深入理解了現代 C++ 的 template 機制、concept 限制、iterator 設計，以及 memory management 的細節，並對泛型程式設計與效能優化有更全面的掌握。

- **遊戲程式結構設計：**在設計 Game、Renderer、ChartParser 等類別時，我學到如何將遊戲邏輯與渲染流程分離、如何處理共用資料結構與物件，以及使程式碼層次清晰、責任明確。Game 模組負責音符判定、分數計算及連擊管理，Renderer 模組專注於視覺呈現，ChartParser 處理譜面解析。這種分層設計有效降低模組間耦合度，便於後續功能擴充或維護，也讓程式更具可測試性。

- **多人協作與版本控制經驗：**本專題使用 Git 作為版本控制工具，並以 GitHub 託管遠端倉庫。我負責教導組員使用 Git 與 GitHub CLI，包括分支建立、切換、合併，提交、推送、pull request 及 issue 管理等。我同時負責主要處理 merge conflict，當多位組員在同一程式碼區塊修改時，我需要分析各方意圖，判斷正確合併策略，並與組員協調程式碼風格與命名規則，確保最終程式碼可編譯與運行。這使我深刻體會到版本控制不僅是程式碼管理工具，更是團隊協作的核心保障。

- **跨平台發布與自動化：**我負責 Linux 編譯腳本，並修改 Windows 編譯腳本以解決 DLL 相依性問題。專案後期，我撰寫 Linux 與 Windows 的 release script，依老師要求將程式打包成 zip 並建立 GitHub Release。我也使用基於 Wine 的 Bottles 模擬 Windows 系統，測試組員編譯的 Windows 程式。這些自動化流程大幅提升專案編譯效率與穩定性，使組員能快速測試與展示最新版本。

- **遊戲時間與判定精度：**在遊戲中實作 `adjustGameStartTime` 及判定系統時，我深刻體會到時間誤差累積與判定設計對音樂遊戲體驗的影響。透過持續修正遊戲時間誤差與音樂播放同步、不斷與組員討論並改進遊戲判定標準，我也學會了如何在毫秒級別管理遊戲事件。

- **UI 與玩家體驗設計：**在主選單、設定頁面、倒數動畫與結算畫面的設計過程中，我學會了如何兼顧美觀、易用性與功能完整性。例如設定頁面、特效反饋、統計數據即時顯示，都需要細心調整以提升玩家沉浸感。這讓我認識到遊戲開發不僅是程式實作，更涉及使用者體驗設計。

- **SDL2 使用經驗：**專案中全面使用 SDL2 系列函式庫（SDL、SDL_ttf、SDL_image、SDL_mixer）進行圖形、文字、影像及音樂管理。透過 SDL2，我學會了事件處理、渲染循環設計、紋理管理與硬體加速等技術，並理解如何在跨平台環境下穩定運行遊戲。

(二) 建議

- **記憶體管理改進：**目前少數變數仍使用手動 new/delete 進行記憶體管理，容易產生記憶體洩漏或 dangling pointer 問題。建議全面遵循 Resource Acquisition Is Initialization (RAII) 原則，並廣泛引入 `std::unique_ptr`、`std::shared_ptr` 等智慧指標，以自動管理資源生命週期，降低潛在錯誤，也能提升程式安全性、可讀性與維護性。

- **進一步模組化與擴充性：**雖然目前程式架構已清晰，但部分功能仍可進一步抽象化。例如為遊

戲模式、譜面格式或特效建立統一接口與虛擬基底類別，便於未來新增功能或第三方模組整合。這樣不僅降低修改成本，也提高模組可插拔性與重用性，有助於專案長期維護與擴展，並讓測試與調試更具針對性。

- **增加測試與自動化流程：**建議針對 Game 判定系統、音符片段呈現、時間誤差修正、Renderer 渲染結果與特效顯示建立單元測試與整合測試。自動化測試可提前發現邏輯錯誤、時間同步偏差及邊界條件，減少開發過程中的錯誤與重工。可使用 C++ 測試框架（如 Google Test、Catch2）搭配 CI/CD 流程，自動執行測試與生成報告，確保每次提交都維持穩定性。
- **優化版本控制策略：**目前已使用 Git 分支管理，但可引入更嚴格的分支命名規範、標籤策略與合併流程（如 Git Flow 或 trunk-based development），以進一步提升多人協作效率。明確追蹤功能開發與版本，有助於快速定位問題與回溯歷史，也能降低 merge conflict 發生頻率，尤其在多人同時開發遊戲核心邏輯或特效模組時更為重要。
- **使用者體驗改進：**可加入更多即時視覺與聲音反饋，例如 combo 效果動畫、錯誤按鍵提示、滑鼠角色碰撞提示等，使玩家更直觀理解操作結果，提升沉浸感與操作滿意度。同時優化 UI 互動邏輯與選單流程，讓新手玩家快速上手並減少操作迷失，增強整體遊戲體驗。
- **提供 Documentation 與 Comment：**建議在程式碼中添加完整的 comment，清楚描述各區塊功能與邏輯流程，並提供全面的遊戲程式設計與使用說明文件（documentation），明確記錄類別責任、方法用途、資料流向、物件所有權管理、遊戲操作方法等細節。對於複雜邏輯或模組間關係，可提供使用範例、class diagram、state diagram 或流程圖等，方便使用者與新開發者快速理解系統，並提升程式碼可維護性與專案可擴展性。

第二節　個人程式設計說明

一、　沈威宇撰寫的部分

符合 C++20 標準，並實作於 `namespace mystd` 之下的仿 STL 函式庫：

- `include/algorithm-copy.hpp`：`copy`、`copy_if`、`copy_backward`。
- `include/algorithm-find.hpp`：`find_if`、`find`、`find_if_not`。
- `include/algorithm-remove.hpp`：`remove_if`、`remove`。
- `include/algorithm-heap.hpp`：`push_heap`、`pop_heap`、`make_heap`、`sort_heap`、`is_heap`、`is_heap_until`。
- `include/range-access.hpp`：range access 相關輔助函式。
- `include/swap.hpp`：`swap`。
- `include/ignore.hpp`：`ignore_t`。
- `include/tuple-using.hpp`：提供 `std::tuple_element`、`std::tuple_element_t`、`std::tuple_size`、`std::tuple_size_v` 之 ADL 與 structured binding 輔助，供 `array` 與 `tuple` 使用。
- `include/array.hpp`：完整實作 `<array>`。

- `include/tuple.hpp`：完整實作 `<tuple>`。
- `include/priority_queue.hpp`：完整實作 `<queue>` 中與 `priority_queue` 相關之內容。
- `include/allocator.hpp`：完整實作 `<memory>` 中的 `allocator`、`allocator_traits` 與相關輔助內容。
- `include/vector.hpp`：完整實作 `<vector>`。

非 STL 標準之自製資料結構與演算法，亦於 `namespace mystd` 之下：

- `include/circulate.hpp`：提供 circular iterator，可用於 `vector` 或使用者自訂容器。
- `include/qsort.hpp`：Quick sort 演算法，介面設計仿照 `std::sort`。

Tap notes 與 Hold notes 之判定、計分與渲染、遊戲資訊計算、各類特效之判定與渲染、設定頁面、倒數動畫、模組註冊系統，以及康威生命遊戲模組與其設定頁面，其相關程式碼分布於以下檔案，以下說明僅涵蓋本人撰寫之部分：

- `Mods.hpp`：模組註冊系統。
- `mods/GameOfLife.hpp`：康威生命遊戲模組及其設定頁面。
- `Game.hpp`：Tap notes 與 Hold notes 之判定、計分，以及避免 edge cases 之修補。
- `Renderer.hpp`：Tap notes、Hold notes 與各類特效之渲染。
- `KeyNoteData.hpp`：Tap notes 與 Hold notes 之資料結構，與林孜謙撰寫之 `ChartParser.hpp` 共用。
- `BeatRunner.cpp`：
  - 遊戲資訊計算
  - 設定頁面
  - 倒數動畫
  - 主迴圈中 next fragment 載入、事件導向流程、程式關閉事件處理，以及 key press / release 事件呼叫對應函式
  - SDL 初始化與設定
  - 各類 edge cases 之修補

測試用檔案：

- `generate-notes.hpp`：依指定數量與比例隨機產生 Tap notes 與 Hold notes，並以 quick sort 排序。
- `test0.cpp`：Tap notes 與 Hold notes 之初期測試程式。

二、 2-A-1-1 描述 class 之間的關係

(一) include/tuple.hpp

- `template <typename T> struct _is_tuple` inherits from `std::false_type`, serving as a primary type trait to determine whether a given type is a tuple. (line 20)
- ```
  template <std::size_t I, class Head, class... Tail> struct _tuple_impl<I, Head, Tail...>
  ```

  publicly inherits from `_tuple_impl<I + 1, Tail...>`, forming a recursive inheritance structure that represents each tuple element at a distinct level. (line 54)
- ```
  template <std::size_t I, class Head, class... Tail> struct _tuple_impl<I, Head, Tail...>
  ```

  contains a data member `Head head;`, demonstrating class composition where each level stores exactly one tuple element. (line 56)
- `template <class... Types> class tuple` composes an internal implementation object `_tuple_impl<0, Types...> _impl`, separating the public interface from the recursive storage implementation. (line 258)
- `template <typename... Ts> struct _is_tuple<mystd::tuple<Ts...>>` inherits from `std::true_type`, providing a specialization that marks `mystd::tuple` as a valid tuple type at compile time. (line 578)
- `template <class T> struct _count_of_type<T>` inherits from `std::integral_constant<std::size_t, 0>`, defining the recursion base case for counting occurrences of a type within a parameter pack. (line 603)
- ```
  template <class T, class Head, class... Tail> struct _count_of_type<T, Head, Tail...>
  ```

  inherits from

  ```
  std::integral_constant<std::size_t, (std::is_same_v<T, Head> ? 1 : 0) + _count_of_type<T, Tail...>::
      value>
  ```

  , implementing a recursive compile-time algorithm via inheritance. (line 606)
- `template <class T, class... Tail> struct _index_of_type<T, T, Tail...>` inherits from `std::integral_constant<std::size_t, 0>`, representing the base case for determining the index of a type within the tuple. (line 618)
- ```
  template <class T, class Head, class... Tail> struct _index_of_type<T, Head, Tail...>
  ```

  inherits from

  ```
  std::integral_constant<std::size_t, 1 + _index_of_type<T, Tail...>::value>
  ```

  , forming a recursive inheritance chain to compute the type index at compile time. (line 622)
- ```
  template <class... Types, class Alloc> struct uses_allocator<std::tuple<Types...>, Alloc>
  ```

  inherits from `std::true_type`, indicating that `mystd::tuple` participates in allocator-aware construction semantics. (line 732)
  ```

- `template <class... Types> struct tuple_size<mystd::tuple<Types...>>` inherits from `std::integral_constant<std::size_t, sizeof...(Types)>`, integrating `mystd::tuple` with the standard tuple interface. (line 739)
-
```
template <std::size_t I, class Head, class... Tail> struct tuple_element<I, mystd::tuple<Head, Tail
    ...>>
```

  inherits from `tuple_element<I - 1, mystd::tuple<Tail...>>`, forming a recursive inheritance structure to map indices to element types. (line 748)

(二) 其他 mystd

因數量甚多，僅列舉以下數例。

- `mystd::circulate` 與 `mystd::priority_queue` 皆為 container adapter，其預設底層容器均為 `mystd::vector`。（`include/circulate.hpp` line 191；`include/priority_queue.hpp` line 18）
- `mystd::circulate` 透過 iterator proxy classes `_circulate_iterator` 與 `_circulate_const_iterator` 實作循環式走訪語意，使 iterator 能在不改變原容器的情況下重複遍歷元素。（`include/circulate.hpp` line 15、line 92）
- `include/allocator.hpp` 中透過大量輔助 `struct` 實作 `allocator_traits` 所需之型別別名與推導機制，下列程式碼節錄示範其中一個 `rebind` 輔助結構之實作方式。（line 42–55）

```
1  template <typename Alloc, typename T, typename = void>
2  struct __rebind_helper {};
3
4  template <typename Alloc, typename T>
5  struct __rebind_helper<Alloc, T,
6                         std::void_t<typename Alloc::template rebind<T>::other>> {
7    using type = typename Alloc::template rebind<T>::other;
8  };
9
10 template <template <class, class...> class Alloc, class U, class... Args,
11           class T>
12 struct __rebind_helper<Alloc<U, Args...>, T> {
13   using type = Alloc<T, Args...>;
14 };
```

- `ignore_t` 定義於 `include/ignore.hpp`，供 `include/tuple.hpp` 中 `mystd::tie` 使用，以支援忽略指定綁定項目之語意。（`include/ignore.hpp` line 5–16）

```
1  struct ignore_t {
2    template <class T>
3    constexpr const ignore_t &operator=(const T &) const noexcept {
4      return *this;
5    }
6
7    template <class T>
8    constexpr const ignore_t &operator=(T &&) const noexcept {
9      return *this;
10   }
11 };
12
13 inline constexpr ignore_t ignore{};
```

(三) mystd 以外（僅列沈威宇撰寫部分）

- game_priority_queue<T> 公開繼承自 mystd::priority_queue<T>，並將其受保護成員 c（底層 container）提升為 public 成員。(Game.hpp line 17–20)

```
template <class T> class game_priority_queue : public mystd::priority_queue<T> {
public:
  using mystd::priority_queue<T>::c;
};
```

- game_priority_queue<Effect> centerEffects 為 game_priority_queue 之 template specialization，其儲存型別固定為 Effect，用於遊戲中特效之優先序管理。此架構除滿足現行需求外，亦預留未來功能擴充之彈性。(Game.hpp line 65)
- Game 具有成員 mystd::vector<KeyNoteData> &notes，其為對全域變數之參考，此設計係後期為配合 ChartParser 之整合與資料共享需求而進行之調整。(Game.hpp line 44)
- Game 透過成員 laneEffects 聚合多個 Effect，並透過 centerEffects 聚合一個 game_priority_queue<Effect>，以支援集中式特效排程與依優先序處理之設計。(Game.hpp line 64–65)
- Game 具有成員物件

```
mystd::vector<mystd::circulate<int8_t, mystd::vector<int8_t>>> highway
```

  ，儲存 highway 上的 tap notes and hold notes。(Game.hpp line 50)
- Renderer 具有成員物件 Game &game，持有遊戲邏輯之參考，以便於渲染階段即時存取遊戲狀態。由於兩者均在後期整合改為手動記憶體管理，故未使用 shared_ptr，若未來專案擴展或應使用之並避免手動記憶體管理。(Renderer.hpp line 54)
- Renderer 使用

```
std::unordered_map<mystd::tuple<int8_t, bool, uint32_t>, SDL_Texture *, NotesCacheHash>
```

  作為 key notes 的紋理快取容器，並透過自訂雜湊結構 NotesCacheHash 提供雜湊函式；其餘 std::unordered_map 則用於快取其他類型之 SDL_Texture*。（Renderer.hpp line 55–59)

三、 2-A-1-2 畫出 class 之間的關係圖

Class Diagram 均由沈威宇使用 clang-uml 自原始 C++ 程式碼中自動萃取類別結構資訊，再依實際設計意圖進行人工補註與整理，最後透過 PlantUML 生成。因圖片較大，請見 https://github.com/NTUEE-CPL-SDL2/report（超連結）。

四、 2-A-3-1 public 變數使用與原因

- mystd::array 中之 T elems[N == 0 ? 1 : N]; 必須宣告為 public，以符合 C++ 標準對 aggregate initialization 之規範；一個 class 若要成為 aggregate，其所有非靜態資料成員皆須為 public。(include/array.hpp line 33)

- `_tuple_impl<I, Head, Tail...>` 中之成員 `Head head;` 宣告為 `public`，以利實作階段進行遞迴存取與修改。由於該型別屬於 implementation detail，並不直接對使用者暴露，因此不影響介面封裝性。
- `template <class... Types> class tuple` 中之 `_tuple_impl<0, Types...> _impl` 宣告為 `public`，以支援

```
template <std::size_t I, class... Types> constexpr decltype(auto) get
```

  等介面函式的實作。該等函式的原理請參見 **Template Compile-Time Recursion**。
- `game_priority_queue<T>` 將 `mystd::priority_queue<T>` 中之受保護成員 `c`（底層 container）提升為 `public`，以供遍歷使用。若為規模更大的專案，提供 public const reference 以避免資料被任意修改，將是更佳的設計方案。（`Game.hpp` line 17–20）
- `Game` 類別之資料成員目前皆為 `public`，此為後期整合階段所做之調整。若未來系統擴充，較佳的設計方式為僅透過 `Renderer` 持有 `Game` 之 reference，與介面函式提供主迴圈所需存取。（`Game.hpp` line 44–65）
- `Renderer` 之 `fps` 成員宣告為 `public`，以供主迴圈即時更新並顯示 FPS 資訊。（`Renderer.hpp` line 499）

五、 2-A-3-2 private 變數使用與原因

- `Renderer` 中多數資料成員設為 `private`，以避免被其他獨立渲染元件錯誤修改，確保渲染狀態與資源管理的一致性。（`Renderer.hpp` line 54–70）
- `mystd::vector` 之 allocator（無實體大小）、實際資料指標、大小與容量等內部狀態皆設為 `private`，以防止使用者破壞容器不變式。（`include/vector.hpp` line 46–49）

```
[[no_unique_address]] Allocator alloc;
T *elems;
std::size_t sz;
std::size_t cap;
```

六、 2-A-3-3 protected 變數使用與原因

`mystd::priority_queue` 之底層容器 `c` 與比較子 `comp` 依據 C++ 標準設為 `protected`。原因如下：

- `priority_queue` 為 container adapter 而非 container，本身應允許被繼承與擴充。
- 若設為 `private`，將破壞 STL 一貫的可繼承設計風格。
- 若設為 `public`，使用者可能直接修改底層資料，破壞 heap invariant，使 `priority_queue` 的語意完全失效。

相關實作位於 `include/priority_queue.hpp` line 31–33：

```
protected:
  Container c = Container();
  Compare comp = Compare();
```

七、 2-A-3-4 friend 使用與原因

本專題中的 `friend` 宣告的主要目的為：

- 非成員運算子多載。
- iterator proxy 類別間的互通。
- 避免破壞 class 封裝性或擴大 public 介面。
- 與 C++20 標準或 STL 慣例保持一致。

擇要列舉如下：

- `include/circulate.hpp` 中之 `_circulate_iterator` 與 `_circulate_const_iterator` 大量使用 `friend` 宣告以實作非成員形式之 iterator 運算子，包括：`operator+`、`operator==`、`operator!=` 與 `operator<=>`。此設計可確保運算子滿足對稱性（symmetry），並避免將僅供運算子使用的內部狀態暴露為 public 成員。（line 72–82、152–162、171–181）
- `include/circulate.hpp` 中之 `_circulate_iterator` 與 `_circulate_const_iterator` 彼此宣告為 `friend`，以支援 mutable 與 const iterator 間的比較與排序關係。此作法符合 STL iterator 的設計慣例，並避免在介面層提供不必要的 getter 函式，確保型別安全與最小可見性。
- `circulate` 容器本身宣告

```
friend bool operator==(const circulate &lhs, const circulate &rhs)
```

  以允許直接比較兩個 `circulate` 物件之內部狀態。由於此等比較運算屬於容器語意的一部分，將其實作為非成員 friend 函式可維持與標準容器一致的使用方式。(line 333)
- `include/tuple.hpp` 中，`tuple` 類別使用 `friend` 宣告實作 `operator==` 與 `operator<=>`，用以支援 tuple 間的等價與三向比較。（line 519、537、555、559）此設計使比較運算可透過 ADL 正確解析，同時避免將內部實作細節（如遞迴結構）暴露。

八、 2-A-3-5 class composition

(一) Resource Acquisition Is Initialization (RAII)

RAII is a programming idiom in C++ used to manage resource lifetimes automatically. The key idea is:

- Resources (e.g., memory, file handles, sockets) are acquired in the constructor of a class.
- Resources are released in the destructor, ensuring deterministic cleanup and exception safety.

(二) 專題使用

本專題中，多數 class 採用 **composition（組合）**方式建立內部關係，其核心目標包括：

- 將資料與功能封裝於物件內部，避免 global state 干擾。
- 透過明確的成員關係管理所有權與生命週期，達成 **Resource Acquisition Is Initialization (RAII)**。

- 與 C++20 標準及 STL 慣例保持一致。

具體實例請參見 **2-A-3-1 public 變數使用與原因**、**2-A-3-2 private 變數使用與原因**、**2-A-3-3 protected 變數使用與原因** 與 **2-B-3-1-1 指出組裝關係**。

九、 2-B-1-1 建構子、複製建構子、解構子

(一) include/vector.hpp

- `constexpr vector() noexcept(noexcept(Allocator()))` (line 95)：預設建構子，建立空 vector。
- `explicit constexpr vector(const Allocator &alloc_) noexcept` (line 97)：使用指定 allocator 初始化 vector。
- `explicit vector(std::size_t count, const Allocator &alloc_ = Allocator())` (line 100)：建立指定大小的 vector，元素未初始化。
- ```
  constexpr vector(std::size_t count, const T &value, const Allocator &alloc_ = Allocator())
  ```

  (line 121–122)：建立指定大小的 vector，所有元素初始化為 value。
- ```
  constexpr vector(InputIt first, InputIt last, const Allocator &alloc_ = Allocator())
  ```

  (line 147–148)：由迭代器範圍初始化 vector。
- `constexpr vector(const vector &other)` (line 181)：複製建構子，深複製其他 vector。
- `constexpr vector(vector &&other) noexcept` (line 205)：移動建構子，將資源從 other 移入本物件。
- `constexpr vector(const vector &other, const Allocator &alloc_)` (line 266)：複製建構子並指定 allocator。
- ```
  vector(std::initializer_list<T> ilist, const Allocator &alloc_ = Allocator())
  ```

  (line 288)：初始化列表建構子。
- `constexpr ~vector()` (line 309)：解構子，釋放所管理的資源。

(二) include/tuple.hpp

`template <std::size_t I> struct _tuple_impl<I>`：

- `constexpr _tuple_impl()` (line 29)：預設建構子，建立空的 _tuple_impl。
- `constexpr _tuple_impl(const _tuple_impl &) noexcept` (line 30)：複製建構子，複製另一個空 _tuple_impl。
- `constexpr _tuple_impl(_tuple_impl &&) noexcept` (line 31)：移動建構子，移動另一個空 _tuple_impl。
- ```
  template <class Alloc> constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &)
  ```

- (line 33–34)：allocator 預設建構子。

```
template <class Alloc> constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &, const _tuple_impl
    &)
```

- (line 36–38)：allocator 複製建構子。

```
template <class Alloc> constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &, _tuple_impl &&)
```

- (line 40–42)：allocator 移動建構子。

```
template <std::size_t I, class Head, class... Tail> struct _tuple_impl<I, Head, Tail...>
```

：

- (line 62–64)

```
  constexpr _tuple_impl()
    requires(std::is_default_constructible_v<Head> &&
             (std::is_default_constructible_v<Tail> && ...))
```

：預設建構子，初始化 head 與 base。

- (line 67–71)

```
  explicit((!std::is_convertible_v<const Head &, Head> ||
           (!std::is_convertible_v<const Tail &, Tail> ||
            ...))) constexpr _tuple_impl(const Head &arg, const Tail &...args)
    requires std::is_constructible_v<Head, const Head &> &&
                 (std::is_constructible_v<Tail, const Tail &> && ...)
```

：逐層複製建構子，逐層 copy 初始化 head 與 base。

- (line 74–80)

```
  template <class UHead, class... UTail>
  explicit((!std::is_convertible_v<UHead, Head> ||
           (!std::is_convertible_v<UTail, Tail> ||
            ...))) constexpr _tuple_impl(UHead &&arg, UTail &&...args)
    requires(sizeof...(Tail) == sizeof...(UTail)) &&
                std::is_constructible_v<Head, UHead> &&
                (std::is_constructible_v<Tail, UTail> && ...)
```

：逐層移動建構子，支援 rvalue 初始化，逐層移動初始化 head 與 base。

- (line 83–86)

```
  template <typename Alloc>
  constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc)
    requires std::is_default_constructible_v<Head> &&
                 (std::is_default_constructible_v<Tail> && ...)
```

：allocator 預設建構子。

- (line 96–101)

```
  template <typename Alloc, class UHead, class... UTail>
  constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc, UHead &&arg,
                        UTail &&...args)
    requires(sizeof...(Tail) == sizeof...(UTail)) &&
                std::is_constructible_v<Head, UHead> &&
                (std::is_constructible_v<Tail, UTail> && ...)
```

：allocator 逐層移動建構子。

- (line 110–112)

```
constexpr _tuple_impl(_tuple_impl &&other)
  requires(std::is_move_constructible_v<Head> &&
           std::is_move_constructible_v<base>)
```

：移動建構子。

- (line 116–120)

```
template <class Alloc>
constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc,
                      const _tuple_impl &other)
  requires(std::is_copy_constructible_v<Head> &&
           std::is_copy_constructible_v<base>)
```

：allocator 複製建構子。

- (line 124–128)

```
template <class Alloc>
constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc,
                      _tuple_impl &&other)
  requires(std::is_move_constructible_v<Head> &&
           std::is_move_constructible_v<base>)
```

：allocator 移動建構子。

- (line 133–141)

```
template <class Alloc, class... UTypes>
constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc,
                      const _tuple_impl<I, UTypes...> &other)
  requires(sizeof...(Tail) + 1 == sizeof...(UTypes)) &&
             std::is_constructible_v<Head,
                                     const typename std::remove_cvref_t<
                                         _first_type_t<UTypes...>> &> &&
             std::is_constructible_v<
                 base, const typename _tuple_impl<I, UTypes...>::base &>
```

：allocator 逐層複製建構子，支援不同型別。

- (line 147–154)

```
template <class Alloc, class... UTypes>
constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc,
                      _tuple_impl<I, UTypes...> &&other)
  requires(sizeof...(Tail) + 1 == sizeof...(UTypes)) &&
             std::is_constructible_v<
                 Head, std::remove_cvref_t<_first_type_t<UTypes...>>> &&
             std::is_constructible_v<
                 base, typename _tuple_impl<I, UTypes...>::base>
```

：allocator 逐層移動建構子，支援不同型別。

`template <class... Types> class tuple`：

- (line 260–261)

```
constexpr tuple()
  requires(std::is_default_constructible_v<Types> && ...)
```

：預設建構子。

- (line 264–267)

```
explicit((!std::is_convertible_v<const Types &, Types> ||
          ...)) constexpr tuple(const Types &...args)
  requires(sizeof...(Types) >= 1) &&
          (std::is_copy_constructible_v<Types> && ...)
```

：逐層複製建構子。

- (line 270–282)

```
template <class... UTypes>
explicit((!std::is_convertible_v<UTypes, Types> ||
          ...)) constexpr tuple(UTypes &&...args)
  requires(sizeof...(Types) >= 1) &&
          (sizeof...(Types) == sizeof...(UTypes)) &&
          (std::is_constructible_v<Types, UTypes> && ...) &&
          (sizeof...(Types) > 1 ||
           !mystd::_is_tuple_v<
               std::remove_cvref_t<_first_type_t<UTypes...>>>) &&
          (sizeof...(Types) > 3 ||
           !std::is_same_v<std::remove_cvref_t<_first_type_t<UTypes...>>,
                           mystd::allocator_arg_t> ||
           std::is_same_v<_first_type_t<Types...>, mystd::allocator_arg_t>)
```

：逐層移動建構子，支援 rvalue 初始化。

- (line 285–293)

```
template <class... UTypes>
explicit((!std::is_convertible_v<Types, const UTypes &> ||
          ...)) constexpr tuple(const tuple<UTypes...> &other)
  requires(sizeof...(Types) == sizeof...(UTypes)) &&
          (std::is_constructible_v<Types, const UTypes &> && ...) &&
          (!(sizeof...(Types) == 1 &&
             ((std::is_convertible_v<UTypes, Types> || ...) ||
              (std::is_constructible_v<Types, UTypes> || ...) ||
              (std::is_same_v<Types, UTypes> || ...))))
```

：複製建構子，支援不同型別。

- (line 296–304)

```
template <class... UTypes>
explicit((!std::is_convertible_v<Types, UTypes &&> ||
          ...)) constexpr tuple(tuple<UTypes...> &&other)
  requires(sizeof...(Types) == sizeof...(UTypes)) &&
          (std::is_constructible_v<Types, UTypes &&> && ...) &&
          (!(sizeof...(Types) == 1 &&
             ((std::is_convertible_v<UTypes, Types> || ...) ||
              (std::is_constructible_v<Types, UTypes> || ...) ||
              (std::is_same_v<Types, UTypes> || ...))))
```

：移動建構子，支援不同型別。

- (line 307–319)

```
template <class U1, class U2>
explicit(
    (!std::is_convertible_v<
         U1, typename _tuple_impl<0, Types...>::template element_type<0>> ||
     !std::is_convertible_v<
         U2, typename _tuple_impl<0, Types...>::template element_type<
                 1>>)) constexpr tuple(const std::pair<U1, U2> &p)
  requires(sizeof...(Types) == 2) &&
          std::is_constructible_v<
              typename _tuple_impl<0, Types...>::template element_type<0>,
              U1> &&
          std::is_constructible_v<
              typename _tuple_impl<0, Types...>::template element_type<1>, U2>
```

：`std::pair` 複製建構子。

- (line 322–335)

```
template <class U1, class U2>
explicit((
    !std::is_convertible_v<
        U1 &&, typename _tuple_impl<0, Types...>::template element_type<0>> ||
    !std::is_convertible_v<
        U2 &&, typename _tuple_impl<0, Types...>::template element_type<
                   1>>)) constexpr tuple(std::pair<U1, U2> &&p)
  requires(sizeof...(Types) == 2) &&
          std::is_constructible_v<
              typename _tuple_impl<0, Types...>::template element_type<0>,
              U1 &&> &&
          std::is_constructible_v<
              typename _tuple_impl<0, Types...>::template element_type<1>,
              U2 &&>
```

：`std::pair` 移動建構子。

- `tuple(const tuple &other)` (line 338)：複製建構子。
- `tuple(tuple &&other)` (line 340)：移動建構子。
- (line 342–344)

```
template <class Alloc>
constexpr tuple(mystd::allocator_arg_t, const Alloc &a)
  requires(std::is_default_constructible_v<Types> && ...)
```

：allocator 預設建構子。

- (line 355–363)

```
template <class Alloc, class... UTypes>
explicit((!std::is_convertible_v<UTypes, Types> ||
          ...)) constexpr tuple(mystd::allocator_arg_t, const Alloc &a,
                                UTypes &&...args)
  requires(sizeof...(Types) >= 1) &&
          (sizeof...(Types) == sizeof...(UTypes)) &&
          (std::is_constructible_v<Types, UTypes> && ...) &&
          (sizeof...(Types) > 1 ||
           !mystd::_is_tuple_v<std::remove_cvref_t<_first_type_t<UTypes...>>>)
```

：allocator 逐層移動建構子。

- (line 366–375)

```
1  template <class Alloc, class... UTypes>
2  explicit((!std::is_convertible_v<Types, const UTypes &> ||
3            ...)) constexpr tuple(mystd::allocator_arg_t, const Alloc &a,
4                                  const tuple<UTypes...> &other)
5    requires(sizeof...(Types) == sizeof...(UTypes)) &&
6            (std::is_constructible_v<Types, const UTypes &> && ...) &&
7            (!(sizeof...(Types) == 1 &&
8               ((std::is_convertible_v<UTypes, Types> || ...) ||
9                (std::is_constructible_v<Types, UTypes> || ...) ||
10               (std::is_same_v<Types, UTypes> || ...))))
```

：allocator 複製建構子。

- (line 378–387)

```
1  template <class Alloc, class... UTypes>
2  explicit((!std::is_convertible_v<Types, UTypes &&> ||
3            ...)) constexpr tuple(mystd::allocator_arg_t, const Alloc &a,
4                                  tuple<UTypes...> &&other)
5    requires(sizeof...(Types) == sizeof...(UTypes)) &&
6            (std::is_constructible_v<Types, UTypes &&> && ...) &&
7            (!(sizeof...(Types) == 1 &&
8               ((std::is_convertible_v<UTypes, Types> || ...) ||
9                (std::is_constructible_v<Types, UTypes> || ...) ||
10               (std::is_same_v<Types, UTypes> || ...))))
```

：allocator 移動建構子。

- (line 390–403)

```
1  template <class Alloc, class U1, class U2>
2  explicit(
3      (!std::is_convertible_v<
4          U1, typename _tuple_impl<0, Types...>::template element_type<0>> ||
5       !std::is_convertible_v<
6          U2, typename _tuple_impl<0, Types...>::template element_type<
7                  1>>)) constexpr tuple(mystd::allocator_arg_t, const Alloc &a,
8                                        const std::pair<U1, U2> &p)
9    requires(sizeof...(Types) == 2) &&
10           std::is_constructible_v<
11               typename _tuple_impl<0, Types...>::template element_type<0>,
12               U1> &&
13           std::is_constructible_v<
14               typename _tuple_impl<0, Types...>::template element_type<1>, U2>
```

：allocator `std::pair` 複製建構子。

- (line 406–421)

```
1  template <class Alloc, class U1, class U2>
2  explicit((
3      !std::is_convertible_v<
4          U1 &&, typename _tuple_impl<0, Types...>::template element_type<0>> ||
5      !std::is_convertible_v<
6          U2 &&, typename _tuple_impl<0, Types...>::template element_type<
7                  1>>)) constexpr tuple(mystd::allocator_arg_t,
8                                        const Alloc &a,
9                                        std::pair<U1, U2> &&p)
10   requires(sizeof...(Types) == 2) &&
11           std::is_constructible_v<
12               typename _tuple_impl<0, Types...>::template element_type<0>,
13               U1 &&> &&
14           std::is_constructible_v<
```

```
15                  typename _tuple_impl<0, Types...>::template element_type<1>,
16                  U2 &&>
```

：allocator `std::pair` 移動建構子。

- (line 425–426)

```
1  template <class Alloc>
2  tuple(mystd::allocator_arg_t, const Alloc &a, const tuple &other)
```

：allocator 複製建構子。

- (line 429–430)

```
1  template <class Alloc>
2  tuple(mystd::allocator_arg_t, const Alloc &a, tuple &&other)
```

：allocator 移動建構子。

`template <> class tuple<>`：

- `constexpr tuple() noexcept = default;` (line 545)：編譯器生成預設建構子。
- `constexpr tuple(const tuple &) noexcept = default;` (line 547)：編譯器生成複製建構子。
- `constexpr tuple(tuple &&) noexcept = default;` (line 548)：編譯器生成移動建構子。

(三) include/priority_queue.hpp

- `priority_queue() : priority_queue(Compare(), Container()) {}` (line 36)：預設建構子。
- `explicit priority_queue(const Compare &compare)` (line 38)：指定比較函式。
- `priority_queue(const Compare &compare, const Container &cont)` (line 41)：指定比較函式與底層容器。
- `priority_queue(const Compare &compare, Container &&cont)` (line 46)：移動底層容器建構子。
- `priority_queue(const priority_queue &other)` (line 51)：複製建構子。
- `priority_queue(priority_queue &&other) noexcept` (line 53)：移動建構子。
- `constexpr ~priority_queue() = default;` (line 159)：編譯器生成解構子。

(四) include/circulate.hpp

- `circulate() = default;` (line 227)：預設建構子。
- `explicit circulate(Container cont, size_type start_ = 0)` (line 229)：帶初始容器建構子。
- `circulate(const circulate &other)` (line 237)：複製建構子。

- `circulate(circulate &&other) noexcept` (line 239)：移動建構子。

(五) Game.hpp

```
Game(std::size_t lanes_, std::size_t fragments_, uint32_t mpf, mystd::vector<KeyNoteData> &keynotes)
```

(line 67–70)：初始化遊戲主要參數與 notes 參考。

(六) Renderer.hpp

```
Renderer(Game &game_, int screenW_, int screenH_, SDL_Renderer *renderer, TTF_Font *large_font_, TTF_Font *
    medium_font_, TTF_Font *small_font_)
```

(line 501–505)：初始化渲染器與字型指標等。

十、 2-B-1-2 指標成員處理

(一) iterator 成員處理一併說明

本報告於「指標成員處理」章節中一併說明「iterator 成員處理」，理由如下：

- 在實作層面上，iterator 本質上即為一種受控的指標抽象（pointer abstraction），其內部通常直接持有原始指標，或持有可間接定位至元素之指標型成員（如 `T*`、container reference 加 index 等）。iterator 的比較運算、位移運算與解參考操作（`operator*`、`operator->`、`operator+` 等），均直接依賴內部指標或等價結構，其正確實作本質上即為指標操作的安全封裝。
- iterator 的正確性與安全性，直接取決於其所持有之指標成員的生命週期、有效性與邊界管理方式，並承擔以下責任：
  - 描述元素存取位置（position semantics）。
  - 維持與底層容器狀態的一致性。
  - 在不暴露底層指標的前提下，提供 STL 風格的介面。

(二) Exception Safety Guarantees

STL containers follow strict **exception safety guarantees** during operations such as insertion, deletion, or reallocation:

- **No-throw guarantee:** The operation never throws an exception.
- **Strong guarantee:** If an exception occurs, the container's state remains unchanged.
- **Basic guarantee:** Some changes may occur, but the container remains in a valid state.

In my implementation, I use `try-catch` blocks and proper deallocation to handle exceptions, prefer no-throwing branches when possible, and provide explicit `noexcept` specifications.

(三) include/vector.hpp

指出：Line 51–932：

```
  constexpr void deallocate() {
    if (elems) {
      mystd::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
      elems = nullptr;
    }
    sz = cap = 0;
  }

  constexpr void destroy_deallocate() {
    if (elems) {
      if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = 0; i < sz; ++i)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
      mystd::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
      elems = nullptr;
    }
    cap = 0;
  }

  template <bool UseMove>
  constexpr void transfer_elements(T *new_elems, std::size_t count) {
    if constexpr (std::is_trivially_copyable_v<T>) {
      std::memcpy(new_elems, elems, count * sizeof(T));
    } else {
      std::size_t i = 0;
      try {
        for (; i < count; ++i) {
          if constexpr (UseMove) {
            mystd::allocator_traits<Allocator>::construct(alloc, new_elems + i,
                                                          std::move(elems[i]));
          } else {
            mystd::allocator_traits<Allocator>::construct(alloc, new_elems + i,
                                                          elems[i]);
          }
        }
      } catch (...) {
        for (std::size_t j = 0; j < i; ++j)
          mystd::allocator_traits<Allocator>::destroy(alloc, new_elems + j);
        throw;
      }
    }
  }

public:
  constexpr vector() noexcept(noexcept(Allocator()))
      : alloc(Allocator()), elems(nullptr), sz(0), cap(0) {}
  explicit constexpr vector(const Allocator &alloc_) noexcept
      : alloc(alloc_), elems(nullptr), sz(0), cap(0) {}

  explicit vector(std::size_t count, const Allocator &alloc_ = Allocator())
      : alloc(alloc_), sz(count), cap(count) {
    if (count == 0)
      elems = nullptr;
    else {
      elems = mystd::allocator_traits<Allocator>::allocate(alloc, count);
      if constexpr (!std::is_trivially_copyable_v<T>) {
        std::size_t i = 0;
        try {
          for (; i < count; ++i)
            mystd::allocator_traits<Allocator>::construct(alloc, elems + i);
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
          deallocate();
```

```
        throw;
      }
    }
  }
}

constexpr vector(std::size_t count, const T &value,
                 const Allocator &alloc_ = Allocator())
    : alloc(alloc_), sz(count), cap(count) {
  if (count == 0)
    elems = nullptr;
  else {
    elems = mystd::allocator_traits<Allocator>::allocate(alloc, count);
    if constexpr (std::is_trivially_copyable_v<T>)
      std::fill_n(elems, count, value);
    else {
      std::size_t i = 0;
      try {
        for (; i < count; ++i)
          mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                        value);
      } catch (...) {
        for (std::size_t j = 0; j < i; ++j)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
        deallocate();
        throw;
      }
    }
  }
}

template <std::input_iterator InputIt>
constexpr vector(InputIt first, InputIt last,
                 const Allocator &alloc_ = Allocator())
    : alloc(alloc_), elems(nullptr), sz(0), cap(0) {
  if constexpr (std::forward_iterator<InputIt>) {
    if (first == last)
      return;
    std::size_t count = static_cast<std::size_t>(std::distance(first, last));
    if (count > 0) {
      elems = mystd::allocator_traits<Allocator>::allocate(alloc, count);
      sz = cap = count;
      if constexpr (std::is_trivially_copyable_v<T>) {
        if (std::contiguous_iterator<InputIt>)
          std::memmove(elems, &*first, count * sizeof(T));
        else
          mystd::copy(first, last, elems);
      } else {
        std::size_t i = 0;
        try {
          for (; first != last; ++first, ++i)
            mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                          *first);
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
          deallocate();
          throw;
        }
      }
    }
  } else
    for (; first != last; ++first)
      push_back(*first);
}
```

```

  constexpr vector(const vector &other)
      : alloc(mystd::allocator_traits<
              Allocator>::select_on_container_copy_construction(other.alloc)),
        elems(nullptr), sz(other.sz), cap(other.cap) {
    if (cap > 0) {
      elems = mystd::allocator_traits<Allocator>::allocate(alloc, cap);
      if constexpr (std::is_trivially_copyable_v<T>)
        std::memmove(elems, other.elems, sz * sizeof(T));
      else {
        std::size_t i = 0;
        try {
          for (; i < sz; ++i)
            mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                          other.elems[i]);
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
          mystd::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
          throw;
        }
      }
    }
  }

  constexpr vector(vector &&other) noexcept(
      mystd::allocator_traits<
          Allocator>::propagate_on_container_move_assignment::value ||
      mystd::allocator_traits<Allocator>::is_always_equal::value)
      : alloc(std::move(other.alloc)), elems(nullptr), sz(0), cap(0) {
    if constexpr (mystd::allocator_traits<Allocator>::
                      propagate_on_container_move_assignment::value ||
                  mystd::allocator_traits<Allocator>::is_always_equal::value ||
                  (requires {
                    { alloc == other.alloc } -> std::convertible_to<bool>;
                  } && (alloc == other.alloc))) {
      elems = std::exchange(other.elems, nullptr);
      sz = std::exchange(other.sz, 0);
      cap = std::exchange(other.cap, 0);
    } else {
      if (other.sz > 0) {
        elems = mystd::allocator_traits<Allocator>::allocate(alloc, other.sz);
        sz = cap = other.sz;
        if constexpr (std::is_trivially_move_constructible_v<T>)
          std::memmove(elems, other.elems, sz * sizeof(T));
        else if (std::is_nothrow_move_constructible_v<T> ||
                 !std::is_copy_constructible_v<T>) {
          std::size_t i = 0;
          try {
            for (; i < sz; ++i)
              mystd::allocator_traits<Allocator>::construct(
                  alloc, elems + i, std::move(other.elems[i]));
          } catch (...) {
            for (std::size_t j = 0; j < i; ++j)
              mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
            mystd::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
            throw;
          }
        } else {
          std::size_t i = 0;
          try {
            for (; i < sz; ++i)
              mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                            other.elems[i]);
          } catch (...) {
```

```
          for (std::size_t j = 0; j < i; ++j)
            mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
          mystd::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
          throw;
        }
      }
      if constexpr (!std::is_trivially_destructible_v<T> &&
                    !std::is_trivially_move_constructible_v<T>) {
        if (std::is_nothrow_move_constructible_v<T> ||
            !std::is_copy_constructible_v<T>) {
          for (std::size_t j = 0; j < sz; ++j)
            mystd::allocator_traits<Allocator>::destroy(alloc,
                                                        other.elems + j);
        }
      }
      other.elems = nullptr;
      other.sz = other.cap = 0;
    }
  }
}

constexpr vector(const vector &other, const Allocator &alloc_)
    : alloc(alloc_), elems(nullptr), sz(other.sz), cap(other.cap) {
  if (cap > 0) {
    elems = mystd::allocator_traits<Allocator>::allocate(alloc, cap);
    if constexpr (std::is_trivially_copyable_v<T>)
      std::memmove(elems, other.elems, sz * sizeof(T));
    else {
      std::size_t i = 0;
      try {
        for (; i < sz; ++i)
          mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                        other.elems[i]);
      } catch (...) {
        for (std::size_t j = 0; j < i; ++j)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
        mystd::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
        throw;
      }
    }
  }
}

vector(std::initializer_list<T> ilist, const Allocator &alloc_ = Allocator())
    : alloc(alloc_), elems(nullptr), sz(ilist.size()), cap(ilist.size()) {
  if (cap > 0) {
    elems = mystd::allocator_traits<Allocator>::allocate(alloc, cap);
    if constexpr (std::is_trivially_copyable_v<T>)
      std::memmove(elems, ilist.begin(), sz * sizeof(T));
    else {
      std::size_t i = 0;
      try {
        for (const T *j = ilist.begin(); j != ilist.end(); ++j, ++i)
          mystd::allocator_traits<Allocator>::construct(alloc, elems + i, *j);
      } catch (...) {
        for (std::size_t k = 0; k < i; ++k)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + k);
        mystd::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
        throw;
      }
    }
  }
}

constexpr ~vector() {
```

```
    if (elems) {
      if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = 0; i < sz; ++i)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
      mystd::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
    }
  }

  constexpr vector &operator=(const vector &other) {
    if (this != &other) {
      if constexpr (mystd::allocator_traits<Allocator>::
                        propagate_on_container_copy_assignment::value &&
                    (requires {
                      { alloc != other.alloc } -> std::convertible_to<bool>;
                    } && (alloc != other.alloc))) {
        destroy_deallocate();
        alloc = other.alloc;
        if (cap < other.sz) {
          elems = mystd::allocator_traits<Allocator>::allocate(alloc, other.sz);
          cap = other.sz;
        }
      } else if (other.sz > cap) {
        if (elems)
          destroy_deallocate();
        elems = mystd::allocator_traits<Allocator>::allocate(alloc, other.sz);
        cap = other.sz;
      } else if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = other.sz; i < sz; ++i)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
      if constexpr (std::is_trivially_copyable_v<T>)
        std::memcpy(elems, other.elems, other.sz * sizeof(T));
      else {
        std::size_t i = 0;
        try {
          for (; i < other.sz; ++i) {
            if (i < sz)
              elems[i] = other.elems[i];
            else
              mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                            other.elems[i]);
          }
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
          throw;
        }
      }
      sz = other.sz;
    }
    return *this;
  }

  constexpr vector &operator=(vector &&other) noexcept(
      mystd::allocator_traits<
          Allocator>::propagate_on_container_move_assignment::value ||
      mystd::allocator_traits<Allocator>::is_always_equal::value) {
    if (this != &other) {
      if constexpr (mystd::allocator_traits<Allocator>::
                        propagate_on_container_move_assignment::value ||
                    mystd::allocator_traits<
                        Allocator>::is_always_equal::value ||
                    (requires {
                      { alloc == other.alloc } -> std::convertible_to<bool>;
                    } && (alloc == other.alloc))) {
        destroy_deallocate();
```

```
      alloc = std::move(other.alloc);
      elems = std::exchange(other.elems, nullptr);
      sz = std::exchange(other.sz, 0);
      cap = std::exchange(other.cap, 0);
    } else {
      if (other.sz > cap) {
        if (elems)
          destroy_deallocate();
        elems = mystd::allocator_traits<Allocator>::allocate(alloc, other.sz);
        cap = other.sz;
      } else if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = other.sz; i < sz; ++i)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
      if constexpr (std::is_trivially_copyable_v<T>)
        std::memcpy(elems, other.elems, other.sz * sizeof(T));
      else if (std::is_nothrow_move_constructible_v<T> ||
               !std::is_copy_constructible_v<T>) {
        std::size_t i = 0;
        try {
          for (; i < other.sz; ++i)
            mystd::allocator_traits<Allocator>::construct(
                alloc, elems + i, std::move(other.elems[i]));
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
          throw;
        }
      } else {
        std::size_t i = 0;
        try {
          for (; i < other.sz; ++i)
            mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                          other.elems[i]);
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
          throw;
        }
      }
      sz = other.sz;
    }
  }
  return *this;
}

constexpr vector &operator=(std::initializer_list<T> ilist) {
  assign(ilist.begin(), ilist.end());
  return *this;
}

constexpr void assign(std::size_t count, const T &value) {
  if (count > cap) {
    destroy_deallocate();
    sz = 0;
    elems = mystd::allocator_traits<Allocator>::allocate(alloc, count);
    cap = count;
  }
  if constexpr (std::is_trivially_copyable_v<T>) {
    std::fill_n(elems, count, value);
  } else {
    std::size_t min_size = std::min(sz, count);
    for (std::size_t i = 0; i < min_size; ++i)
      elems[i] = value;
    for (std::size_t i = min_size; i < count; ++i)
      mystd::allocator_traits<Allocator>::construct(alloc, elems + i, value);
```

```
    if (sz > count)
      for (std::size_t i = count; i < sz; ++i)
        mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
  }
  sz = count;
}

template <std::input_iterator InputIt>
constexpr void assign(InputIt first, InputIt last) {
  if constexpr (std::forward_iterator<InputIt>) {
    std::size_t count = static_cast<std::size_t>(std::distance(first, last));
    if (count > cap) {
      destroy_deallocate();
      sz = 0;
      elems = mystd::allocator_traits<Allocator>::allocate(alloc, count);
      cap = count;
    }
    if constexpr (std::is_trivially_copyable_v<T>) {
      if constexpr (std::contiguous_iterator<InputIt>) {
        std::memmove(elems, &*first, count * sizeof(T));
      } else {
        mystd::copy(first, last, elems);
      }
    } else {
      std::size_t i = 0;
      for (; i < count && first != last; ++i, ++first) {
        if (i < sz)
          elems[i] = *first;
        else
          mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                        *first);
      }
      if (sz > count)
        for (std::size_t j = count; j < sz; ++j)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
    }
    sz = count;
  } else {
    clear();
    for (; first != last; ++first)
      push_back(*first);
  }
}

constexpr void assign(std::initializer_list<T> ilist) {
  assign(ilist.begin(), ilist.end());
}

constexpr allocator_type get_allocator() const noexcept { return alloc; }

constexpr T &at(std::size_t index) {
  if (index >= sz)
    throw std::out_of_range("vector");
  return elems[index];
}

constexpr const T &at(std::size_t index) const {
  if (index >= sz)
    throw std::out_of_range("vector");
  return elems[index];
}

constexpr T &operator[](std::size_t index) { return elems[index]; }
constexpr const T &operator[](std::size_t index) const {
  return elems[index];
```

```
455   }
456   constexpr T &front() { return *elems; }
457   constexpr const T &front() const { return *elems; }
458   constexpr T &back() { return elems[sz - 1]; }
459   constexpr const T &back() const { return elems[sz - 1]; }
460
461   constexpr T *data() noexcept { return elems; }
462   constexpr const T *data() const noexcept { return elems; }
463
464   constexpr iterator begin() noexcept { return elems; }
465   constexpr const_iterator begin() const noexcept { return elems; }
466   constexpr const_iterator cbegin() const noexcept { return elems; }
467
468   constexpr iterator end() noexcept { return elems + sz; }
469   constexpr const_iterator end() const noexcept { return elems + sz; }
470   constexpr const_iterator cend() const noexcept { return elems + sz; }
471
472   constexpr reverse_iterator rbegin() noexcept {
473     return reverse_iterator(end());
474   }
475   constexpr const_reverse_iterator rbegin() const noexcept {
476     return const_reverse_iterator(cend());
477   }
478   constexpr const_reverse_iterator crbegin() const noexcept {
479     return const_reverse_iterator(cend());
480   }
481
482   constexpr reverse_iterator rend() noexcept {
483     return reverse_iterator(begin());
484   }
485   constexpr const_reverse_iterator rend() const noexcept {
486     return const_reverse_iterator(cbegin());
487   }
488   constexpr const_reverse_iterator crend() const noexcept {
489     return const_reverse_iterator(cbegin());
490   }
491
492   constexpr bool empty() const noexcept { return sz == 0; }
493   constexpr std::size_t size() const noexcept { return sz; }
494   constexpr std::size_t max_size() const noexcept {
495     return std::min(
496         mystd::allocator_traits<Allocator>::max_size(alloc),
497         static_cast<std::size_t>(std::numeric_limits<std::ptrdiff_t>::max()));
498   }
499
500   constexpr void reserve(std::size_t new_cap) {
501     if (new_cap <= cap)
502       return;
503     if (new_cap > max_size())
504       throw std::length_error("vector");
505     T *new_elems = mystd::allocator_traits<Allocator>::allocate(alloc, new_cap);
506     if (elems) {
507       if constexpr (std::is_trivially_copyable_v<T>) {
508         std::memcpy(new_elems, elems, sz * sizeof(T));
509       } else if (std::is_nothrow_move_constructible_v<T> ||
510                  !std::is_copy_constructible_v<T>) {
511         transfer_elements<true>(new_elems, sz);
512       } else {
513         transfer_elements<false>(new_elems, sz);
514       }
515       destroy_deallocate();
516     }
517     elems = new_elems;
518     cap = new_cap;
519   }
```

```

  constexpr std::size_t capacity() const noexcept { return cap; }

  constexpr void shrink_to_fit() {
    if (cap != sz) {
      if (sz == 0)
        destroy_deallocate();
      else {
        T *new_elems = mystd::allocator_traits<Allocator>::allocate(alloc, sz);
        if constexpr (std::is_trivially_copyable_v<T>) {
          std::memcpy(new_elems, elems, sz * sizeof(T));
        } else if (std::is_nothrow_move_constructible_v<T> ||
                   !std::is_copy_constructible_v<T>) {
          transfer_elements<true>(new_elems, sz);
        } else {
          transfer_elements<false>(new_elems, sz);
        }
        destroy_deallocate();
        elems = new_elems;
        cap = sz;
      }
    }
  }

  constexpr void clear() noexcept {
    if constexpr (!std::is_trivially_destructible_v<T>)
      for (std::size_t i = 0; i < sz; ++i)
        mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
    sz = 0;
  }

  constexpr iterator insert(const_iterator pos, const T &value) {
    std::size_t index = pos - elems;
    if (index > sz) {
      if (index >= cap)
        reserve(std::max(sz ? sz * _MYSTD_VECTOR_GROW : 1, index + 1));
      for (std::size_t i = sz; i < index; ++i) {
        mystd::allocator_traits<Allocator>::construct(alloc, elems + i);
      }
      sz = index;
    } else if (sz >= cap)
      reserve(sz ? sz * _MYSTD_VECTOR_GROW : 1);
    if (index < sz) {
      if constexpr (std::is_trivially_copyable_v<T>) {
        std::memmove(elems + index + 1, elems + index,
                     (sz - index) * sizeof(T));
        elems[index] = value;
      } else {
        mystd::allocator_traits<Allocator>::construct(alloc, elems + sz,
                                                      std::move(elems[sz - 1]));
        for (std::size_t i = sz - 1; i > index; --i)
          elems[i] = std::move(elems[i - 1]);
        elems[index] = value;
      }
    } else {
      mystd::allocator_traits<Allocator>::construct(alloc, elems + index,
                                                    value);
    }
    ++sz;
    return elems + index;
  }

  constexpr iterator insert(const_iterator pos, T &&value) {
    std::size_t index = pos - elems;
    if (index > sz) {
```

```
585       if (index >= cap)
586         reserve(std::max(sz ? sz * _MYSTD_VECTOR_GROW : 1, index + 1));
587       for (std::size_t i = sz; i < index; ++i) {
588         mystd::allocator_traits<Allocator>::construct(alloc, elems + i);
589       }
590       sz = index;
591     }
592     if (sz >= cap)
593       reserve(sz ? sz * _MYSTD_VECTOR_GROW : 1);
594     if (index < sz) {
595       if constexpr (std::is_trivially_copyable_v<T>) {
596         std::memmove(elems + index + 1, elems + index,
597                      (sz - index) * sizeof(T));
598         elems[index] = std::move(value);
599       } else {
600         mystd::allocator_traits<Allocator>::construct(alloc, elems + sz,
601                                                       std::move(elems[sz - 1]));
602         for (std::size_t i = sz - 1; i > index; --i)
603           elems[i] = std::move(elems[i - 1]);
604         elems[index] = std::move(value);
605       }
606     } else {
607       mystd::allocator_traits<Allocator>::construct(alloc, elems + index,
608                                                     std::move(value));
609     }
610     ++sz;
611     return elems + index;
612   }
613
614   constexpr iterator insert(const_iterator pos, std::size_t count,
615                             const T &value) {
616     if (count == 0)
617       return const_cast<iterator>(pos);
618     std::size_t index = pos - elems;
619     if (index > sz) {
620       if (index >= cap)
621         reserve(std::max(sz ? sz * _MYSTD_VECTOR_GROW : 1, index + count));
622       for (std::size_t i = sz; i < index; ++i) {
623         mystd::allocator_traits<Allocator>::construct(alloc, elems + i);
624       }
625       sz = index;
626     }
627     if (sz + count > cap)
628       reserve(std::max(sz ? sz * _MYSTD_VECTOR_GROW : 1, sz + count));
629     if constexpr (std::is_trivially_copyable_v<T>) {
630       std::memmove(elems + index + count, elems + index,
631                    (sz - index) * sizeof(T));
632       std::fill_n(elems + index, count, value);
633     } else {
634       for (std::size_t i = sz + count - 1; i >= index + count; --i) {
635         if (i < sz)
636           elems[i] = std::move(elems[i - count]);
637         else
638           mystd::allocator_traits<Allocator>::construct(
639               alloc, elems + i, std::move(elems[i - count]));
640       }
641       for (std::size_t i = index; i < index + count; ++i) {
642         if (i < sz)
643           elems[i] = value;
644         else
645           mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
646                                                         value);
647       }
648     }
649     sz += count;
```

```
    return elems + index;
  }

  template <std::input_iterator InputIt>
  constexpr iterator insert(const_iterator pos, InputIt first, InputIt last) {
    std::size_t index = pos - elems;
    if (index > sz) {
      if constexpr (std::forward_iterator<InputIt>) {
        std::size_t count =
            static_cast<std::size_t>(std::distance(first, last));
        if (index >= cap)
          reserve(std::max(sz ? sz * _MYSTD_VECTOR_GROW : 1, index + count));
      } else {
        if (index >= cap)
          reserve(std::max(sz ? sz * _MYSTD_VECTOR_GROW : 1, index + 1));
      }
      for (std::size_t i = sz; i < index; ++i) {
        mystd::allocator_traits<Allocator>::construct(alloc, elems + i);
      }
      sz = index;
    }
    if constexpr (std::forward_iterator<InputIt>) {
      std::size_t count = static_cast<std::size_t>(std::distance(first, last));
      if (count == 0)
        return const_cast<iterator>(pos);
      if (sz + count > cap)
        reserve(std::max(sz ? sz * _MYSTD_VECTOR_GROW : 1, sz + count));
      if constexpr (std::is_trivially_copyable_v<T>) {
        std::memmove(elems + index + count, elems + index,
                     (sz - index) * sizeof(T));
        if constexpr (std::contiguous_iterator<InputIt>)
          std::memmove(elems + index, &*first, count * sizeof(T));
        else
          mystd::copy(first, last, elems + index);
      } else {
        for (std::size_t i = sz + count - 1; i >= index + count; --i) {
          if (i < sz)
            elems[i] = std::move(elems[i - count]);
          else
            mystd::allocator_traits<Allocator>::construct(
                alloc, elems + i, std::move(elems[i - count]));
        }
        std::size_t i = index;
        for (; i < index + count && first != last; ++i, ++first) {
          if (i < sz)
            elems[i] = *first;
          else
            mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                          *first);
        }
      }
      sz += count;
      return elems + index;
    } else {
      vector<T, Allocator> tmp(first, last, alloc);
      return insert(pos, tmp.begin(), tmp.end());
    }
  }

  constexpr iterator insert(const_iterator pos,
                            std::initializer_list<T> ilist) {
    return insert(pos, ilist.begin(), ilist.end());
  }

  template <class Arg>
```

```
715   constexpr iterator emplace(const_iterator pos, Arg &&arg) {
716     std::size_t index = pos - elems;
717     if (index > sz) {
718       if (index >= cap)
719         reserve(std::max(sz ? sz * _MYSTD_VECTOR_GROW : 1, index + 1));
720       for (std::size_t i = sz; i < index; ++i) {
721         mystd::allocator_traits<Allocator>::construct(alloc, elems + i);
722       }
723       sz = index;
724     } else if (sz >= cap)
725       reserve(sz ? sz * _MYSTD_VECTOR_GROW : 1);
726     if (index == sz) {
727       if constexpr (std::is_trivially_move_constructible_v<T>)
728         elems[sz] = T(std::forward<Arg>(arg));
729       else
730         mystd::allocator_traits<Allocator>::construct(alloc, elems + sz,
731                                                       std::forward<Arg>(arg));
732     } else {
733       if constexpr (std::is_trivially_copyable_v<T>) {
734         std::memmove(elems + index + 1, elems + index,
735                      (sz - index) * sizeof(T));
736         elems[index] = T(std::forward<Arg>(arg));
737       } else {
738         mystd::allocator_traits<Allocator>::construct(alloc, elems + sz,
739                                                       std::move(elems[sz - 1]));
740         for (std::size_t i = sz - 1; i > index; --i)
741           elems[i] = std::move(elems[i - 1]);
742         elems[index] = T(std::forward<Arg>(arg));
743       }
744     }
745     ++sz;
746     return elems + index;
747   }
748
749   constexpr iterator erase(const_iterator pos) {
750     std::size_t index = pos - elems;
751     if (index >= sz)
752       return elems + sz;
753     if constexpr (std::is_trivially_copyable_v<T>) {
754       std::memmove(elems + index, elems + index + 1,
755                    (sz - index - 1) * sizeof(T));
756     } else {
757       for (std::size_t i = index; i < sz - 1; ++i)
758         elems[i] = std::move(elems[i + 1]);
759       mystd::allocator_traits<Allocator>::destroy(alloc, elems + sz - 1);
760     }
761     --sz;
762     return elems + index;
763   }
764
765   constexpr iterator erase(const_iterator first, const_iterator last) {
766     if (first == last)
767       return const_cast<iterator>(first);
768     std::size_t index = first - elems;
769     std::size_t end_index = last - elems;
770     if (index >= sz)
771       return elems + sz;
772     if (end_index > sz)
773       end_index = sz;
774     std::size_t count = end_index - index;
775     if constexpr (std::is_trivially_copyable_v<T>) {
776       std::memmove(elems + index, elems + index + count,
777                    (sz - index - count) * sizeof(T));
778     } else {
779       for (std::size_t i = index; i + count < sz; ++i)
```

```
      elems[i] = std::move(elems[i + count]);
    for (std::size_t i = sz - count; i < sz; ++i)
      mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
  }
  sz -= count;
  return elems + index;
}

constexpr void push_back(const T &value) {
  if (sz >= cap)
    reserve(sz ? sz * _MYSTD_VECTOR_GROW : 1);
  if constexpr (std::is_trivially_copyable_v<T>)
    elems[sz++] = value;
  else {
    mystd::allocator_traits<Allocator>::construct(alloc, elems + sz, value);
    ++sz;
  }
}

constexpr void push_back(T &&value) {
  if (sz >= cap)
    reserve(sz ? sz * _MYSTD_VECTOR_GROW : 1);
  if constexpr (std::is_trivially_move_constructible_v<T>)
    elems[sz++] = std::move(value);
  else {
    mystd::allocator_traits<Allocator>::construct(alloc, elems + sz,
                                                  std::move(value));
    ++sz;
  }
}

template <class Arg> constexpr T &emplace_back(Arg &&arg) {
  if (sz >= cap)
    reserve(sz ? sz * _MYSTD_VECTOR_GROW : 1);
  if constexpr (std::is_trivially_move_constructible_v<T>)
    elems[sz] = T(std::forward<Arg>(arg));
  else
    mystd::allocator_traits<Allocator>::construct(alloc, elems + sz,
                                                  std::forward<Arg>(arg));
  return elems[sz++];
}

constexpr void pop_back() {
  --sz;
  if constexpr (!std::is_trivially_destructible_v<T>)
    mystd::allocator_traits<Allocator>::destroy(alloc, elems + sz);
}

constexpr void resize(std::size_t new_size) {
  if (new_size < sz) {
    if constexpr (!std::is_trivially_destructible_v<T>)
      for (std::size_t i = new_size; i < sz; ++i)
        mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
    sz = new_size;
    return;
  }
  if (new_size > cap)
    reserve(new_size);
  if (new_size > sz) {
    if constexpr (std::is_trivially_default_constructible_v<T>) {
      std::uninitialized_default_construct(elems + sz, elems + new_size);
    } else if constexpr (std::is_default_constructible_v<T>) {
      for (std::size_t i = sz; i < new_size; ++i)
        mystd::allocator_traits<Allocator>::construct(alloc, elems + i);
    } else {
```

```
        throw std::logic_error("vector");
      }
      sz = new_size;
    }
  }

  constexpr void resize(std::size_t new_size, const T &value) {
    if (new_size < sz) {
      if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = new_size; i < sz; ++i)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
      sz = new_size;
      return;
    }
    if (new_size > cap)
      reserve(new_size);
    if (new_size > sz) {
      if constexpr (std::is_trivially_default_constructible_v<T>) {
        std::uninitialized_fill(elems + sz, elems + new_size, value);
      } else {
        for (std::size_t i = sz; i < new_size; ++i)
          mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                        value);
      }
      sz = new_size;
    }
  }

  constexpr void swap(vector &other) noexcept(
      mystd::allocator_traits<Allocator>::propagate_on_container_swap::value ||
      mystd::allocator_traits<Allocator>::is_always_equal::value) {
    if constexpr (mystd::allocator_traits<
                      Allocator>::propagate_on_container_swap::value)
      mystd::swap(alloc, other.alloc);
    mystd::swap(elems, other.elems);
    mystd::swap(sz, other.sz);
    mystd::swap(cap, other.cap);
  }
};

template <class InputIt,
          class Allocator =
              allocator<typename std::iterator_traits<InputIt>::value_type>>
vector(InputIt, InputIt, Allocator = Allocator())
    -> vector<typename std::iterator_traits<InputIt>::value_type, Allocator>;

template <class T, class Allocator>
constexpr void
swap(mystd::vector<T, Allocator> &lhs,
     mystd::vector<T, Allocator> &rhs) noexcept(noexcept(lhs.swap(rhs))) {
  lhs.swap(rhs);
}

template <class T, class Allocator, class U>
constexpr typename mystd::vector<T, Allocator>::size_type
erase(mystd::vector<T, Allocator> &c, const U &value) {
  auto it = mystd::remove(c.begin(), c.end(), value);
  auto r = c.end() - it;
  c.erase(it, c.end());
  return r;
}

template <class T, class Allocator, class Pred>
constexpr typename mystd::vector<T, Allocator>::size_type
erase_if(mystd::vector<T, Allocator> &c, Pred pred) {
```

```
  auto it = mystd::remove_if(c.begin(), c.end(), pred);
  auto r = c.end() - it;
  c.erase(it, c.end());
  return r;
}

} // namespace mystd

template <class T, class Allocator>
constexpr bool operator==(const mystd::vector<T, Allocator> &lhs,
                          const mystd::vector<T, Allocator> &rhs) {
  if (lhs.size() != rhs.size())
    return false;
  return std::equal(lhs.begin(), lhs.end(), rhs.begin());
}

template <class T, class Allocator>
constexpr std::strong_ordering
operator<=>(const mystd::vector<T, Allocator> &lhs,
            const mystd::vector<T, Allocator> &rhs) {
  return std::lexicographical_compare_three_way(
      lhs.begin(), lhs.end(), rhs.begin(), rhs.end(), std::compare_three_way());
}
```

描述：主要概念：

- 採用 RAII（請參見 **Resource Acquisition Is Initialization (RAII)**）管理資源。
- 對 trivially copyable 元素、contiguous iterator 情境使用 `memcpy`、`memmove` 或 `fill_n` 提升效率；對 forward iterator 情境預先計算元素數並配置所需空間。（請參見 **Compile-time branching with if constexpr in pointer and iterator operations**）
- 元素搬移時優先使用 move construct，若不可行則 fallback 至 copy construct。
- 對非 trivially copyable 元素，使用 `construct` 與 `destroy` 搭配 try-catch，確保 exception safety（請參見 **Exception Safety Guarantees**），若出現例外，會 destroy 已建立的元素並 deallocate。例如：

```
std::size_t i = 0;
try {
  for (; i < sz; ++i)
    mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                  other.elems[i]);
} catch (...) {
  for (std::size_t j = 0; j < i; ++j)
    mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
  mystd::allocator_traits<Allocator>::deallocate(alloc, elems, cap);
  throw;
}
```

- 根據 allocator 屬性決定是否可以直接交換內部指標，以避免不必要的元素搬移。

記憶體管理：

- 分配所需記憶體：

```
elems = mystd::allocator_traits<Allocator>::allocate(alloc, count);
```

- 釋放先前分配的記憶體，避免記憶體洩漏：

```
allocator_traits<Allocator>::deallocate(alloc, elems, cap);
```

初值填充：

- 對 trivially copyable 元素使用 `std::fill_n` 高效填充值：

```
if constexpr (std::is_trivially_copyable_v<T>) std::fill_n(elems, count, value);
```

- 對非 trivially copyable 元素逐一 construct，搭配 try-catch 確保 exception safety：

```
mystd::allocator_traits<Allocator>::construct(alloc, elems + i, value);
```

元素解構：

- 必要時呼叫元素的 destructor：

```
if constexpr (!std::is_trivially_destructible_v<T>) allocator_traits<Allocator>::destroy(alloc, elems +
    i);
```

元素複製：

- 高效複製元素：

```
if constexpr (std::is_trivially_copyable_v<T>) {
  if (std::contiguous_iterator<InputIt>)
    std::memmove(elems, &*first, count * sizeof(T));
  else mystd::copy(first, last, elems);
}
```

- 高效複製元素：

```
if constexpr (std::is_trivially_copyable_v<T>)
  std::memmove(elems, other.elems, sz * sizeof(T));
```

- 高效複製元素：

```
if constexpr (std::is_trivially_copyable_v<T>) std::memcpy(new_elems, elems, count * sizeof(T));
```

- 非 trivially copyable 元素逐一 copy construct，搭配 try-catch 確保 exception safety：

```
else mystd::allocator_traits<Allocator>::construct(alloc, new_elems + i, elems[i]);
```

元素移動：

- 若 allocator 可直接交換，則 exchange 指標，避免逐一移動：

```
if constexpr (mystd::allocator_traits<Allocator>::
                  propagate_on_container_move_assignment::value ||
              mystd::allocator_traits<Allocator>::is_always_equal::value ||
              (requires {
                { alloc == other.alloc } -> std::convertible_to<bool>;
              } && (alloc == other.alloc))) {
  elems = std::exchange(other.elems, nullptr);
  sz = std::exchange(other.sz, 0);
  cap = std::exchange(other.cap, 0);
}
```

- 高效移動元素：

```
if constexpr (std::is_trivially_move_constructible_v<T>) std::memmove(elems, other.elems, sz * sizeof(T
    ));
```

- 逐一 move construct，搭配 try-catch 確保 exception safety：

```
else if (std::is_nothrow_move_constructible_v<T> || !std::is_copy_constructible_v<T>) mystd::
    allocator_traits<Allocator>::construct(alloc, elems + i, std::move(other.elems[i]));
```

- 逐一 copy construct 作為 fallback，搭配 try-catch 確保 exception safety：

```
mystd::allocator_traits<Allocator>::construct(alloc, elems + i, other.elems[i]);
```

預留空間

- 對 forward iterator 情境預先計算元素數並配置所需空間，以避免逐個 `push_back` 發生多次配置：

```
if constexpr (std::forward_iterator<InputIt>) {
      if (first == last)
        return;
      std::size_t count = static_cast<std::size_t>(std::distance(first, last));
      // allocate or reserve
}
```

- 逐個 `push_back` 作為 fallback：

```
for (; first != last; ++first)
  push_back(*first);
```

(四) include/circulate.hpp

指出：

Line 15–189：

```
1  template <class circulate> class _circulate_iterator {
2  public:
3    using iterator_category = std::random_access_iterator_tag;
4    using difference_type = typename circulate::difference_type;
5    using value_type = typename circulate::value_type;
6    using pointer = typename circulate::pointer;
7    using reference = typename circulate::reference;
8
9  private:
10   circulate *circ;
11   difference_type idx;
12
13 public:
14   _circulate_iterator(circulate *c, difference_type i) : circ(c), idx(i) {}
15
16   reference operator*() { return (*circ)[idx]; }
17   pointer operator->() { return &(**this); }
18
19   _circulate_iterator &operator++() {
20     ++idx;
21     return *this;
22   }
23   _circulate_iterator operator++(int) {
24     _circulate_iterator tmp = *this;
25     ++*this;
26     return tmp;
27   }
28   _circulate_iterator &operator--() {
29     --idx;
30     return *this;
```

```
  }
  _circulate_iterator operator--(int) {
    _circulate_iterator tmp = *this;
    --*this;
    return tmp;
  }

  _circulate_iterator &operator+=(difference_type n) {
    idx += n;
    return *this;
  }
  _circulate_iterator &operator-=(difference_type n) {
    idx -= n;
    return *this;
  }
  _circulate_iterator operator+(difference_type n) const {
    return _circulate_iterator(circ, idx + n);
  }
  _circulate_iterator operator-(difference_type n) const {
    return _circulate_iterator(circ, idx - n);
  }
  difference_type operator-(const _circulate_iterator &other) const {
    return idx - other.idx;
  }

  reference operator[](difference_type n) { return *(*this + n); }

  friend _circulate_iterator operator+(difference_type n,
                                       const _circulate_iterator &it) {
    return it + n;
  }

  friend bool operator==(const _circulate_iterator &a,
                         const _circulate_iterator &b) noexcept {
    return a.circ == b.circ && a.idx == b.idx;
  }

  friend std::strong_ordering
  operator<=>(const _circulate_iterator &a,
              const _circulate_iterator &b) noexcept {
    if (a.circ != b.circ) {
      return a.circ <=> b.circ;
    }
    return a.idx <=> b.idx;
  }
};

template <class circulate> class _circulate_const_iterator {
public:
  using iterator_category = std::random_access_iterator_tag;
  using difference_type = typename circulate::difference_type;
  using value_type = typename circulate::value_type;
  using pointer = typename circulate::const_pointer;
  using reference = typename circulate::const_reference;

private:
  const circulate *circ;
  difference_type idx;

public:
  _circulate_const_iterator(const circulate *c, difference_type i)
      : circ(c), idx(i) {}
  _circulate_const_iterator(const _circulate_iterator<circulate> &it)
      : circ(it.circ), idx(it.idx) {}

```

```
  reference operator*() const { return (*circ)[idx]; }
  pointer operator->() const { return &(**this); }

  _circulate_const_iterator &operator++() {
    ++idx;
    return *this;
  }
  _circulate_const_iterator operator++(int) {
    _circulate_const_iterator tmp = *this;
    ++*this;
    return tmp;
  }
  _circulate_const_iterator &operator--() {
    --idx;
    return *this;
  }
  _circulate_const_iterator operator--(int) {
    _circulate_const_iterator tmp = *this;
    --*this;
    return tmp;
  }

  _circulate_const_iterator &operator+=(difference_type n) {
    idx += n;
    return *this;
  }
  _circulate_const_iterator &operator-=(difference_type n) {
    idx -= n;
    return *this;
  }
  _circulate_const_iterator operator+(difference_type n) const {
    return _circulate_const_iterator(circ, idx + n);
  }
  _circulate_const_iterator operator-(difference_type n) const {
    return _circulate_const_iterator(circ, idx - n);
  }
  difference_type operator-(const _circulate_const_iterator &other) const {
    return idx - other.idx;
  }

  reference operator[](difference_type n) const { return *(*this + n); }

  friend _circulate_const_iterator
  operator+(difference_type n, const _circulate_const_iterator &it) {
    return it + n;
  }

  friend bool operator==(const _circulate_const_iterator &a,
                         const _circulate_const_iterator &b) noexcept {
    return a.circ == b.circ && a.idx == b.idx;
  }

  friend std::strong_ordering
  operator<=>(const _circulate_const_iterator &a,
              const _circulate_const_iterator &b) noexcept {
    if (a.circ != b.circ) {
      return a.circ <=> b.circ;
    }
    return a.idx <=> b.idx;
  }

  friend bool operator==(const _circulate_iterator<circulate> &a,
                         const _circulate_const_iterator &b) noexcept {
    return a.circ == b.circ && a.idx == b.idx;
  }
```

```

  friend bool operator!=(const _circulate_iterator<circulate> &a,
                         const _circulate_const_iterator &b) noexcept {
    return a.circ != b.circ || a.idx != b.idx;
  }

  friend std::strong_ordering
  operator<=>(const _circulate_iterator<circulate> &a,
              const _circulate_const_iterator &b) noexcept {
    if (a.circ != b.circ) {
      return a.circ <=> b.circ;
    }
    return a.idx <=> b.idx;
  }
};
```

描述：

- 為 `circulate` 容器提供標準的隨機存取迭代器（iterator）介面，透過內部索引 `idx` 結合指向容器的指標 `circ`，實現簡單且常數時間的循環訪問。
- `_circulate_iterator` 與 `_circulate_const_iterator` 均提供完整的 STL 風格操作，其設計類似於 `std::deque` 與緊湊型 `std::vector<bool>` 的 iterator 實作。
- 透過 friend 比較運算子允許 `_circulate_iterator` 與 `_circulate_const_iterator` 之間的混合比較。
- 詳細描述與範例請參見 **_circulate_iterator and _circulate_const_iterator operators**。

(五) BeatRunner.cpp（僅列沈威宇撰寫部分）

指出：

Line 154–155：

```
Game *game;
Renderer *gameRenderer;
```

Line 763–769：

```
  if (game)
    delete game;
  game = new Game(LANES, FRAGMENTS, MS_PER_FRAGMENT, keyNotes);
  if (gameRenderer)
    delete gameRenderer;
  gameRenderer = new Renderer(*game, SCREEN_WIDTH, SCREEN_HEIGHT, renderer,
                              large_font, medium_font, small_font);
```

Line 1413–1414：

```
  delete gameRenderer;
  delete game;
```

描述：

- 這兩個指標 `game` 與 `gameRenderer` 為全域變數，供程式各處存取。
- 在 `showSettings` 函式中，若指標已經指向現有物件，先使用 `delete` 刪除，再建立新的物件。

- 程式最後手動刪除指標所指向的物件，確保釋放所有資源，避免記憶體洩漏。
- 此做法屬於手動記憶體管理（manual memory management），更大型專案中建議改用 `std::shared_ptr` 或 `std::unique_ptr` 以自動管理生命週期並避免記憶體洩漏。

十一、 2-B-1-3 建構子、複製建構子、解構子特殊語法

(一) Rule of Three / Five / Zero

- **Rule of Three:** If a class defines a destructor, copy constructor, or copy assignment operator, it likely needs all three.
- **Rule of Five:** Extends the Rule of Three by including move constructor and move assignment for C++11 and later.
- **Rule of Zero:** Prefer using RAII and standard containers to avoid manually managing special member functions.

These rules are applied consistently in my STL implementation to ensure correct and safe resource management, which are demonstrated in **2-B-1-1 建構子、複製建構子、解構子**.

(二) Design Philosophy of STL Container Constructors

- Single Responsibility: Establishing a Valid Initial State:
  - A constructor's sole responsibility is to establish a valid container invariant.
  - After construction, the container is guaranteed to be in a usable and well-defined state.
  - Containers with structural invariants guarantee correctness at the end of construction.
  - Constructors do not encode operational logic (e.g., sorting, filtering, or post-processing).
- Consistent Constructor Families Across Containers: STL containers expose a predictable set of constructor categories:
  - Default constructor (creates an empty container)
  - Size and value constructors (for sequence containers)
  - Iterator range constructors
  - Copy and move constructors
  - Initializer-list constructors (since C++11)

  This consistency reduces cognitive load when switching between container types.
- Iterator-Based Construction for Genericity: Constructors are designed around iterator ranges rather than concrete container types. This allows containers to accept data from any source that satisfies the iterator concept.
- Explicit Control of Overload Ambiguity: Constructor overloads are carefully designed to avoid ambiguity, `explicit`, `std::initializer_list`, **Substitution Failure Is Not An Error (SFINAE)**, and **Concepts** are used to enforce correct overload resolution.

- Allocator-Aware Resource Management: Most STL container constructors accept an allocator parameter. This decouples memory management policy from data structure semantics. Allocator propagation rules ensure predictable behavior during copy and move construction.
- Move Semantics as a First-Class Design Element: Move constructors enable efficient transfer of resources without deep copies. This allows containers to be used naturally in return-by-value and temporary-heavy code paths.
- Predictable Exception and Lifetime Behavior: Constructors adhere to strong or basic **Exception Safety Guarantees**, depending on context. Partial construction states are not observable by users. Resource acquisition follows RAII strictly, ensuring clean rollback on failure.

These rules are applied consistently in my STL implementation to ensure correct and safe resource management, which are demonstrated in **2-B-1-1 建構子、複製建構子、解構子**.

(三) include/vector.hpp 的 compile-time branching with if constexpr 優化

已於 **2-B-1-2 指標成員處理** 的 **include/vector.hpp** 列出，並於 **描述** 中描述。

十二、 2-B-3-1-1 指出組裝關係（僅列沈威宇撰寫部分）

- `_tuple_impl<0, Types...> _impl;`（`include/tuple.hpp` line 258）：`_tuple_impl<0, Types...>` 組裝於 `template <class... Types> class tuple` 中。
- `[[no_unique_address]] Allocator alloc;`（`include/vector.hpp` line 46）：`Allocator` 組裝於 `template <class T, class Allocator> class vector` 中。
- `Container c`（`include/priority_queue.hpp` line 32）：`Container` 組裝於 `template <class T, class Container, class Compare> class priority_queue` 中。
- `Compare comp`（`include/priority_queue.hpp` line 32）：`Compare` 組裝於 `template <class T, class Container, class Compare> class priority_queue` 中。
- `Container c;`（`include/circulate.hpp` line 205）：`Container` 組裝於 `template <class T, class Container> class circulate` 中。
-
```
static std::map<std::string, mystd::tuple<ModFunc, ModFunc, SettingsFunc>> modMap;
```

  （`Mods.hpp` line 17–18）：`std::string` 與

```
mystd::tuple<ModFunc, ModFunc, SettingsFunc>
```

  組裝於 `std::map` 中。
- `mystd::vector<mystd::circulate<int8_t, mystd::vector<int8_t>>> highway;`（`Game.hpp` line 50）：
  - `mystd::vector<int8_t>` 組裝於 `mystd::circulate` 中；

- mystd::circulate<int8_t, mystd::vector<int8_t>>組裝於mystd::vector中；
- mystd::vector<mystd::circulate<int8_t, mystd::vector<int8_t>>> 組裝於 Game 中。

- mystd::vector<bool> lanePressed; (Game.hpp line 53)：mystd::vector<bool>組裝於 Game 中。
- mystd::vector<uint32_t> holdPressedTime; (Game.hpp line 56)：mystd::vector<uint32_t>組裝於 Game 中。
- mystd::vector<Effect> laneEffects; (Game.hpp line 64)：mystd::vector<Effect>組裝於 Game 中。
- game_priority_queue<Effect> centerEffects; (Game.hpp line 65)：
  - mystd::vector<Effect>與 std::less<Effect>組裝於mystd::priority_queue中；
  - game_priority_queue<Effect>組裝於 Game 中。

```
std::unordered_map<mystd::tuple<int8_t, bool, uint32_t>, SDL_Texture *, NotesCacheHash>
    notesTextureCache;
```

(Renderer.hpp line 55–57)：

-
  - mystd::_tuple_impl<0, int8_t, bool, uint32_t>組裝於mystd::tuple中；
  - mystd::tuple<int8_t, bool, uint32_t> 與 NotesCacheHash 組裝於 std::unordered_map 中；

```
std::unordered_map<mystd::tuple<int8_t, bool, uint32_t>, SDL_Texture *, NotesCacheHash>
```

  組裝於 Renderer 中。

- std::unordered_map<std::string, SDL_Texture *> textTextureCache; (Renderer.hpp line 58)：
  - std::string 組裝於 std::unordered_map 中；
  - std::unordered_map<std::string, SDL_Texture *>組裝於 Renderer 中。
- std::unordered_map<std::string, SDL_Texture *> imageTextureCache; (Renderer.hpp line 59)：
  - std::string 組裝於 std::unordered_map 中；
  - std::unordered_map<std::string, SDL_Texture *>組裝於 Renderer 中。

十三、 2-B-3-1-2 指出繼承關係（僅列沈威宇撰寫部分）

-

```
template <std::size_t I, class Head, class... Tail> struct _tuple_impl<I, Head, Tail...> : public
    _tuple_impl<I + 1, Tail...>
```

(include/tuple.hpp line 53–54)：

```
template <std::size_t I, class Head, class... Tail> struct _tuple_impl<I, Head, Tail...>
```

公開繼承自 `_tuple_impl<I + 1, Tail...>`。

- ```
  static std::map<std::string, mystd::tuple<ModFunc, ModFunc, SettingsFunc>> modMap;
  ```

  (`Mods.hpp` line 17–18)：

  - `mystd::_tuple_impl<0, ModFunc, ModFunc, SettingsFunc>` 繼承自 `mystd::_tuple_impl<1, ModFunc, SettingsFunc>`；
  - `mystd::_tuple_impl<1, ModFunc, SettingsFunc>` 繼承自

    ```
    mystd::_tuple_impl<2, SettingsFunc>
    ```

    ；
  - `mystd::_tuple_impl<2, SettingsFunc>` 繼承自 `mystd::_tuple_impl<3>`。

```
template <class T> class game_priority_queue : public mystd::priority_queue<T>
```

(`Game.hpp` line 17)：`template <class T> class game_priority_queue` 公開繼承自 `mystd::priority_queue<T>`。

- `game_priority_queue<Effect> centerEffects;` (`Game.hpp` line 65)：`game_priority_queue<Effect>` 繼承自 `mystd::priority_queue<Effect>`。

十四、 2-B-3-2 指標操作之巧思

請參見 **2-B-1-2 指標成員處理** 中之各 **描述**。

十五、 2-B-3-3 功能巧思

程式執行效率相關請參見 **2-D-1-2 程式執行效率優化**；指標相關請參見 **2-B-1-2 指標成員處理** 中之各 **描述**；依照 STL C++20 標準實作的部分及其優化與巧思請參見本報告各章節與程式碼檔案，如 **2-B-1-2 指標成員處理** 中之各 **描述** 與 **2-C-1-2 Operator Overloading 語法與使用方式描述與範例** 中之各 **描述**；以下擇要描述我實作的其他部分之功能巧思。

(一) 判定旗標

在 `Game.hpp` 中有多種判定旗標（flags），如 `PERFECT`、`GREAT`、`GOOD`、`BAD`、`MISS`、`HOLD_RELEASED` 等。這些旗標之間會使用位元運算（bitwise operations），若改用 `enum` 或 `enum class`，只保證是整數型別，但不能保證位元操作結果，故在某些情況下可能需要額外強制轉型（cast）以進行位元運算，因此我選擇使用 `const uint32_t` 儲存。

(二) 模組註冊系統

在 `mods/GameOfLife.hpp` 中，模組註冊透過 `namespace { ... }` 匿名 namespace 完成，原因為：

- 將其中的所有變數、函式或物件限定在本翻譯單元（translation unit）內部。
```

- 可以在檔案載入時立即執行初始化。
- 避免 **Static Initialization Order Fiasco (SIOF)**，因為初始化延遲到首次使用時才執行。

(三) 紋理快取儲存

在 `Renderer.hpp` 中，我實作了一套紋理快取機制，用以避免在每一幀重複建立相同的 `SDL_Texture`，以大幅降低 GPU 資源建立成本與 CPU 呼叫開銷，提升整體渲染效能與穩定性。

依用途區分三類快取：(line 55–59)

```
55  std::unordered_map<mystd::tuple<int8_t, bool, uint32_t>, SDL_Texture *,
56                     NotesCacheHash>
57      notesTextureCache;
58  std::unordered_map<std::string, SDL_Texture *> textTextureCache;
59  std::unordered_map<std::string, SDL_Texture *> imageTextureCache;
```

對於音符（note fragment）的渲染，其外觀不僅取決於數值本身，亦與是否被按壓與長按持續時間相關。因此，我以 `mystd::tuple<int8_t, bool, uint32_t>` 作為 key，分別表示：

- 音符內容（tap / hold / empty 等）
- 是否為按壓狀態
- 長按持續時間

並自訂 `NotesCacheHash`，將 tuple 中各元素的 hash 組合，以支援 `std::unordered_map` 作為高效能快取容器：(line 43–50)

```
43 struct NotesCacheHash {
44   std::size_t operator()(const mystd::tuple<int8_t, bool, uint32_t> &t) const {
45     auto hash1 = std::hash<int8_t>{}(mystd::get<0>(t));
46     auto hash2 = std::hash<bool>{}(mystd::get<1>(t));
47     auto hash3 = std::hash<uint32_t>{}(mystd::get<2>(t));
48     return hash1 ^ (hash2 << 1) ^ (hash3 << 2);
49   }
50 };
```

延遲建立（Lazy Initialization）：在實際渲染流程（`render`）中，當需要顯示某一音符片段或文字時，會先以 key 查詢快取：若已存在，直接重用既有紋理；若不存在，才呼叫 `createFragmentTexture` 或 `drawText` 建立紋理並存入快取。此策略確保只為實際出現的狀態付出成本，避免無謂的資源配置。

固定圖示預載：對於特效等固定圖示，則於 `loadEffectImages` 中集中預載並放入 `imageTextureCache`。在繪製過程中，僅透過 key 查表即可取得紋理。

資源釋放與安全性 在 Renderer 關閉（`~Renderer();`）或螢幕大小改變（`void updateDimension(int screenW_, int screenH_)`）時，透過 `clearCache` 統一銷毀所有快取中的 `SDL_Texture`，並清空對應的 container：(line 72–87)

```
72  void clearCache() {
73    for (auto &pair : notesTextureCache) {
74      SDL_DestroyTexture(pair.second);
75    }
76    notesTextureCache.clear();
77
```

```
    for (auto &pair : textTextureCache) {
      SDL_DestroyTexture(pair.second);
    }
    textTextureCache.clear();

    for (auto &pair : imageTextureCache) {
      SDL_DestroyTexture(pair.second);
    }
    imageTextureCache.clear();
  }
```

整體而言，此設計在不增加渲染邏輯複雜度的前提下，有效提升效能。

(四) 誤差修正系統

由於理論上的片段（fragment）時間（由 BPM 推導而來的毫秒數）並非整數值，而遊戲中畫面更新與按鍵判定依賴相對於遊戲起始時刻之整數毫秒數，長時間累積後將不可避免地產生時間漂移，與實際音樂節奏漸行漸遠。

為解決此問題，我在 `adjustGameStartTime` 函式中實作了一套累積誤差補償機制，用以動態修正遊戲起始時刻 `gameStartTime`，使實際時間推進能長期貼合實際音樂節奏：(`BeatRunner.cpp` line 1027–1062)

```
void adjustGameStartTime(Uint32 currentTime, Uint32 &gameStartTime,
                         double exactFragmentMs,
                         mystd::vector<uint32_t> &holdPressedTime) {
  static bool initialized = false;
  static Uint32 lastFragmentTime;
  static double accumulatedError = 0.0;

  if (!initialized) {
    lastFragmentTime = currentTime;
    initialized = true;
    return;
  }

  Uint32 actualDelta = currentTime - lastFragmentTime;
  accumulatedError += exactFragmentMs - (double)actualDelta;

  if (std::abs(accumulatedError) >= 1.0) {
    int correction = static_cast<int>(std::floor(accumulatedError));
    Uint32 adjust = static_cast<Uint32>(std::abs(correction));

    if (correction > 0) {
      Uint32 safeAdjust = (gameStartTime > adjust) ? adjust : gameStartTime;
      gameStartTime -= safeAdjust;
      for (auto &t : holdPressedTime)
        t = (t > safeAdjust) ? (t - safeAdjust) : 0;
      accumulatedError -= (double)safeAdjust;
    } else {
      gameStartTime += adjust;
      for (auto &t : holdPressedTime)
        t += adjust;
      accumulatedError -= (double)correction;
    }
  }

  lastFragmentTime = currentTime;
}
```

此函式透過三個區域 static 變數保存跨呼叫狀態：

- `initialized`：確保初始化邏輯僅執行一次。
- `lastFragmentTime`：記錄上一次呼叫時的實際時間戳。
- `accumulatedError`：累積理論時間與實際時間之間的誤差（以浮點數表示）。

每次呼叫時計算實際經過時間 `actualDelta`，並將其與理論片段時間 `exactFragmentMs` 比較，差值累加至 `accumulatedError` 中。此作法避免單次誤差即進行修正，而是以平滑方式累積，降低抖動。

當累積誤差的絕對值在 1 毫秒以上時，才會觸發修正：

- 若誤差為正，表示實際進度落後於理論時間，需將 `gameStartTime` 往前調整。
- 若誤差為負，表示實際進度超前，則將 `gameStartTime` 向後推移。

修正量以整數毫秒為單位，並使用 `floor` 取得穩定的修正值，確保時間調整不會來回震盪。修正後，對應的誤差值亦會自 `accumulatedError` 中扣除，以維持數值一致性。

由於長按音符（hold note）亦依賴時間累積判定，其已記錄的 `holdPressedTime` 必須與遊戲時間保持同步。因此在修正 `gameStartTime` 的同時，會一併調整所有已存在的按壓時間，避免出現視覺與判定不同步的情況。

十六、 2-C-1-1 Operator Overloading 實作與呼叫位置

(一) allocator operator==

`include/allocator.hpp` line 36–40 實作：

```
template <class T1, class T2>
constexpr bool operator==(const allocator<T1> &lhs,
                          const allocator<T2> &rhs) noexcept {
  return true;
}
```

使用於 `include/vector.hpp` line 210–215：

```
    if constexpr (mystd::allocator_traits<Allocator>::
                      propagate_on_container_move_assignment::value ||
                  mystd::allocator_traits<Allocator>::is_always_equal::value ||
                  (requires {
                    { alloc == other.alloc } -> std::convertible_to<bool>;
                  } && (alloc == other.alloc))) {
```

、`include/vector.hpp` line 320–324：

```
      if constexpr (mystd::allocator_traits<Allocator>::
                        propagate_on_container_copy_assignment::value &&
                    (requires {
                      { alloc != other.alloc } -> std::convertible_to<bool>;
                    } && (alloc != other.alloc))) {
```

與 `include/vector.hpp` line 367–373：

```
      if constexpr (mystd::allocator_traits<Allocator>::
                        propagate_on_container_move_assignment::value ||
                    mystd::allocator_traits<
                        Allocator>::is_always_equal::value ||
                    (requires {
```

```
                    { alloc == other.alloc } -> std::convertible_to<bool>;
                  } && (alloc == other.alloc))) {
```

`include/allocator.hpp` line 409–432 實作與呼叫：

```
template <class T, class Alloc, class NonPair>
  requires is_pair_v<std::remove_cvref_t<T>> &&
           (!std::is_constructible_v<std::remove_cvref_t<T>,
                                     std::remove_cvref_t<NonPair>>)
constexpr auto uses_allocator_construction_args(const Alloc &alloc,
                                                NonPair &&np) noexcept {
  struct pair_constructor {
    const Alloc &alloc_;
    NonPair &u_;
    constexpr auto reconstruct(const std::remove_cv_t<T> &p) const {
      return std::make_obj_using_allocator<std::remove_cv_t<T>>(alloc_, p);
    }
    constexpr auto reconstruct(std::remove_cv_t<T> &&p) const {
      return std::make_obj_using_allocator<std::remove_cv_t<T>>(alloc_,
                                                                std::move(p));
    }
    constexpr operator std::remove_cv_t<T>() const {
      return reconstruct(std::remove_cv_t<T>(std::forward<NonPair>(u_).first,
                                             std::forward<NonPair>(u_).second));
    }
  };
  pair_constructor construction{alloc, np};
  return make_tuple(construction);
}
```

(二) array operator[]

`include/array.hpp` line 47–50 實作：

```
  constexpr T &operator[](std::size_t index) { return elems[index]; }
  constexpr const T &operator[](std::size_t index) const {
    return elems[index];
  }
```

`mods/GameOfLife.hpp` line 149 呼叫：`tempRules[0] ^= (1 << i);`。

`mods/GameOfLife.hpp` line 162 呼叫：`tempRules[1] ^= (1 << i);`。

(三) array operator== 與 operator<=>

`include/array.hpp` line 113–124 實作：

```
template <class T, std::size_t N>
constexpr bool operator==(const mystd::array<T, N> &lhs,
                          const mystd::array<T, N> &rhs) {
  return std::equal(lhs.begin(), lhs.end(), rhs.begin());
}

template <class T, std::size_t N>
constexpr std::compare_three_way_result_t<T>
operator<=>(const mystd::array<T, N> &lhs, const mystd::array<T, N> &rhs) {
  return std::lexicographical_compare_three_way(
      lhs.begin(), lhs.end(), rhs.begin(), rhs.end(), std::compare_three_way());
}
```

(四) ignore_t operator=

`include/ignore.hpp` line 6–13 實作：

```
1  template <class T>
2  constexpr const ignore_t &operator=(const T &) const noexcept {
3    return *this;
4  }
5
6  template <class T> constexpr const ignore_t &operator=(T &&) const noexcept {
7    return *this;
8  }
```

(五) priority_queue operator=

`include/priority_queue.hpp` line 161–175 實作：

```
1  priority_queue &operator=(const priority_queue &other) {
2    if (this != &other) {
3      c = other.c;
4      comp = other.comp;
5    }
6    return *this;
7  }
8
9  priority_queue &operator=(priority_queue &&other) noexcept {
10   if (this != &other) {
11     c = std::move(other.c);
12     comp = std::move(other.comp);
13   }
14   return *this;
15 }
```

(六) _tuple_impl 與 tuple operator=

`include/tuple.hpp` line 44–48 實作：

```
1  constexpr _tuple_impl &operator=(const _tuple_impl &) noexcept {
2    return *this;
3  }
4
5  constexpr _tuple_impl &operator=(_tuple_impl &&) noexcept { return *this; }
```

`include/tuple.hpp` line 160–202 實作與呼叫前述：

```
1  constexpr _tuple_impl &operator=(const _tuple_impl &other)
2    requires std::is_copy_assignable_v<Head> && std::is_copy_assignable_v<base>
3  {
4    head = other.head;
5    base::operator=(other);
6    return *this;
7  }
8
9  constexpr _tuple_impl &operator=(_tuple_impl &&other) noexcept(
10     std::is_nothrow_move_assignable_v<Head> &&
11     std::is_nothrow_move_assignable_v<base>)
12   requires(std::is_move_assignable_v<Head> && std::is_move_assignable_v<base>)
13 {
14   head = std::move(other.head);
15   base::operator=(std::move(other));
16   return *this;
17 }
```

```
18
19  template <class UHead, class... UTail>
20  constexpr _tuple_impl &operator=(const _tuple_impl<I, UHead, UTail...> &other)
21    requires(sizeof...(Tail) == sizeof...(UTail)) &&
22            std::is_assignable_v<Head &, const UHead &> &&
23            std::is_assignable_v<base &, const _tuple_impl<I + 1, UTail...> &>
24  {
25    head = other.head;
26    base::operator=(static_cast<const _tuple_impl<I + 1, UTail...> &>(other));
27    return *this;
28  }
29
30  template <class UHead, class... UTail>
31  constexpr _tuple_impl &
32  operator=(_tuple_impl<I, UHead, UTail...> &&other) noexcept(
33      std::is_nothrow_assignable_v<Head &, UHead &&> &&
34      std::is_nothrow_assignable_v<base &, _tuple_impl<I + 1, UTail...> &&>)
35    requires(sizeof...(Tail) == sizeof...(UTail)) &&
36            std::is_assignable_v<Head &, UHead &&> &&
37            std::is_assignable_v<base &, _tuple_impl<I + 1, UTail...> &&>
38  {
39    head = std::move(other.head);
40    base::operator=(
41        static_cast<_tuple_impl<I + 1, UTail...> &&>(std::move(other)));
42    return *this;
43  }
```

`include/tuple.hpp` line 433–464 實作與呼叫前述：

```
1   constexpr tuple &operator=(const tuple &other)
2     requires(std::is_copy_assignable_v<Types> && ...)
3   {
4     _impl = other._impl;
5     return *this;
6   }
7
8   constexpr tuple &operator=(tuple &&other) noexcept(
9       (std::is_nothrow_move_assignable_v<Types> && ...))
10    requires(std::is_move_assignable_v<Types> && ...)
11  {
12    _impl = std::move(other._impl);
13    return *this;
14  }
15
16  template <class... UTypes>
17  constexpr tuple &operator=(const tuple<UTypes...> &other)
18    requires(sizeof...(Types) == sizeof...(UTypes)) &&
19            (std::is_assignable_v<Types &, const UTypes &> && ...)
20  {
21    _impl = other._impl;
22    return *this;
23  }
24
25  template <class... UTypes>
26  constexpr tuple &operator=(tuple<UTypes...> &&other)
27    requires(sizeof...(Types) == sizeof...(UTypes)) &&
28            (std::is_assignable_v<Types &, UTypes &&> && ...)
29  {
30    _impl = std::move(other._impl);
31    return *this;
32  }
```

(七) tuple operator== 與 operator<=>

`include/tuple.hpp` line 511–540 實作：

```
private:
  template <std::size_t... I>
  static constexpr bool tuple_equal_impl(const tuple &a, const tuple &b,
                                         std::index_sequence<I...>) {
    return ((a.get<I>() == b.get<I>()) && ...);
  }

public:
  friend constexpr bool operator==(const tuple &a, const tuple &b) {
    return tuple_equal_impl(a, b, std::make_index_sequence<sizeof...(Types)>{});
  }

private:
  template <std::size_t... I>
  static constexpr auto tuple_three_way_impl(const tuple &a, const tuple &b,
                                             std::index_sequence<I...>)
      -> std::common_comparison_category_t<decltype(a.get<I>() <=>
                                                    b.get<I>())...> {
    using result_t = std::common_comparison_category_t<decltype(a.get<I>() <=>
                                                                b.get<I>())...>;
    result_t result = result_t::equal;
    ((result = (result != 0 ? result : (a.get<I>() <=> b.get<I>()))), ...);
    return result;
  }

public:
  friend constexpr auto operator<=>(const tuple &a, const tuple &b) {
    return tuple_three_way_impl(a, b,
                                std::make_index_sequence<sizeof...(Types)>{});
  }
```

(八) vector operator=

`include/vector.hpp` line 318–423 實作：

```
  constexpr vector &operator=(const vector &other) {
    if (this != &other) {
      if constexpr (mystd::allocator_traits<Allocator>::
                        propagate_on_container_copy_assignment::value &&
                    (requires {
                      { alloc != other.alloc } -> std::convertible_to<bool>;
                    } && (alloc != other.alloc))) {
        destroy_deallocate();
        alloc = other.alloc;
        if (cap < other.sz) {
          elems = mystd::allocator_traits<Allocator>::allocate(alloc, other.sz);
          cap = other.sz;
        }
      } else if (other.sz > cap) {
        if (elems)
          destroy_deallocate();
        elems = mystd::allocator_traits<Allocator>::allocate(alloc, other.sz);
        cap = other.sz;
      } else if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = other.sz; i < sz; ++i)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
      if constexpr (std::is_trivially_copyable_v<T>)
        std::memcpy(elems, other.elems, other.sz * sizeof(T));
      else {
        std::size_t i = 0;
```

```
      try {
        for (; i < other.sz; ++i) {
          if (i < sz)
            elems[i] = other.elems[i];
          else
            mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                          other.elems[i]);
        }
      } catch (...) {
        for (std::size_t j = 0; j < i; ++j)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
        throw;
      }
    }
    sz = other.sz;
  }
  return *this;
}

constexpr vector &operator=(vector &&other) noexcept(
    mystd::allocator_traits<
        Allocator>::propagate_on_container_move_assignment::value ||
    mystd::allocator_traits<Allocator>::is_always_equal::value) {
  if (this != &other) {
    if constexpr (mystd::allocator_traits<Allocator>::
                      propagate_on_container_move_assignment::value ||
                  mystd::allocator_traits<
                      Allocator>::is_always_equal::value ||
                  (requires {
                    { alloc == other.alloc } -> std::convertible_to<bool>;
                  } && (alloc == other.alloc))) {
      destroy_deallocate();
      alloc = std::move(other.alloc);
      elems = std::exchange(other.elems, nullptr);
      sz = std::exchange(other.sz, 0);
      cap = std::exchange(other.cap, 0);
    } else {
      if (other.sz > cap) {
        if (elems)
          destroy_deallocate();
        elems = mystd::allocator_traits<Allocator>::allocate(alloc, other.sz);
        cap = other.sz;
      } else if constexpr (!std::is_trivially_destructible_v<T>)
        for (std::size_t i = other.sz; i < sz; ++i)
          mystd::allocator_traits<Allocator>::destroy(alloc, elems + i);
      if constexpr (std::is_trivially_copyable_v<T>)
        std::memcpy(elems, other.elems, other.sz * sizeof(T));
      else if (std::is_nothrow_move_constructible_v<T> ||
               !std::is_copy_constructible_v<T>) {
        std::size_t i = 0;
        try {
          for (; i < other.sz; ++i)
            mystd::allocator_traits<Allocator>::construct(
                alloc, elems + i, std::move(other.elems[i]));
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
          throw;
        }
      } else {
        std::size_t i = 0;
        try {
          for (; i < other.sz; ++i)
            mystd::allocator_traits<Allocator>::construct(alloc, elems + i,
                                                          other.elems[i]);
```

```
        } catch (...) {
          for (std::size_t j = 0; j < i; ++j)
            mystd::allocator_traits<Allocator>::destroy(alloc, elems + j);
          throw;
        }
      }
      sz = other.sz;
    }
  }
  return *this;
}

constexpr vector &operator=(std::initializer_list<T> ilist) {
  assign(ilist.begin(), ilist.end());
  return *this;
}
```

`circulate` 函式對以上的呼叫請參見 **circulate =**。

(九) vector operator[]

`include/vector.hpp` line 502–505 實作：

```
constexpr T &operator[](std::size_t index) { return elems[index]; }
constexpr const T &operator[](std::size_t index) const {
  return elems[index];
}
```

`Game.hpp` line 107–131 呼叫：

```
  if (f <= 0.20) { // 0.0-0.2 = PERFECT
    score += 1000;
    perfectCount++;
    laneEffects[lane].content &= CLEAR;
    laneEffects[lane].content |= PERFECT;
    addCombo(nowMs);
  } else if (f <= 0.40) { // 0.2-0.4 = GREAT
    score += 700;
    greatCount++;
    laneEffects[lane].content &= CLEAR;
    laneEffects[lane].content |= GREAT;
    addCombo(nowMs);
  } else if (f <= 0.60) { // 0.4-0.6 = GOOD
    score += 300;
    goodCount++;
    laneEffects[lane].content &= CLEAR;
    laneEffects[lane].content |= GOOD;
    addCombo(nowMs);
  } else if (f <= 1.00) { // 0.6-1.0 = BAD
    score += 100;
    badCount++;
    laneEffects[lane].content &= CLEAR;
    laneEffects[lane].content |= BAD;
    resetCombo();
  }
```

`Game.hpp` line 134 呼叫：

```
  laneEffects[lane].endTime = nowMs + msPerFragment * fragments;
```

`Game.hpp` line 142 呼叫：

```
    uint32_t heldMs = nowMs - holdPressedTime[lane];
```

`Game.hpp` line 150–152 呼叫：

```
    laneEffects[lane].content &= CLEAR;
    laneEffects[lane].content |= HOLD_RELEASED;
    laneEffects[lane].endTime = nowMs + msPerFragment * fragments;
```

`Game.hpp` line 167–181 呼叫：

```
      if (bottom < 0) { // tap
        missCount++;
        laneEffects[lane].content &= CLEAR;
        laneEffects[lane].content |= MISS;
        laneEffects[lane].endTime = nowMs + msPerFragment * fragments;
        resetCombo();
        bottom = 0;
      } else if (bottom > 0) { // hold
        if (lanePressed[lane]) {
          addHoldScore(nowMs, lane);
        }
        bottom = 0;
      }
      holdPressedTime[lane] = nowMs;
    }
```

`Game.hpp` line 219–220 呼叫：

```
      lanePressed[lane] = true;
      holdPressedTime[lane] = nowMs;
```

`Game.hpp` line 308 呼叫：

```
    lanePressed[lane] = false;
```

`circulate` 函式對以上的呼叫請參見 **circulate []**。

(十) vector operator== 與 operator<=>

`include/vector.hpp` line 968–982 實作：

```
template <class T, class Allocator>
constexpr bool operator==(const mystd::vector<T, Allocator> &lhs,
                          const mystd::vector<T, Allocator> &rhs) {
  if (lhs.size() != rhs.size())
    return false;
  return std::equal(lhs.begin(), lhs.end(), rhs.begin());
}

template <class T, class Allocator>
constexpr std::strong_ordering
operator<=>(const mystd::vector<T, Allocator> &lhs,
            const mystd::vector<T, Allocator> &rhs) {
  return std::lexicographical_compare_three_way(
      lhs.begin(), lhs.end(), rhs.begin(), rhs.end(), std::compare_three_way());
}
```

(十一) _circulate_iterator and _circulate_const_iterator operators

`include/circulate.hpp` line 30–89 實作：

```
reference operator*() { return (*circ)[idx]; }
pointer operator->() { return &(**this); }

_circulate_iterator &operator++() {
  ++idx;
  return *this;
}
_circulate_iterator operator++(int) {
  _circulate_iterator tmp = *this;
  ++*this;
  return tmp;
}
_circulate_iterator &operator--() {
  --idx;
  return *this;
}
_circulate_iterator operator--(int) {
  _circulate_iterator tmp = *this;
  --*this;
  return tmp;
}

_circulate_iterator &operator+=(difference_type n) {
  idx += n;
  return *this;
}
_circulate_iterator &operator-=(difference_type n) {
  idx -= n;
  return *this;
}
_circulate_iterator operator+(difference_type n) const {
  return _circulate_iterator(circ, idx + n);
}
_circulate_iterator operator-(difference_type n) const {
  return _circulate_iterator(circ, idx - n);
}
difference_type operator-(const _circulate_iterator &other) const {
  return idx - other.idx;
}

reference operator[](difference_type n) { return *(*this + n); }

friend _circulate_iterator operator+(difference_type n,
                                     const _circulate_iterator &it) {
  return it + n;
}

friend bool operator==(const _circulate_iterator &a,
                       const _circulate_iterator &b) noexcept {
  return a.circ == b.circ && a.idx == b.idx;
}

friend std::strong_ordering
operator<=>(const _circulate_iterator &a,
            const _circulate_iterator &b) noexcept {
  if (a.circ != b.circ) {
    return a.circ <=> b.circ;
  }
  return a.idx <=> b.idx;
}
```

include/circulate.hpp line 110–188 實作：

```
  reference operator*() const { return (*circ)[idx]; }
  pointer operator->() const { return &(**this); }

  _circulate_const_iterator &operator++() {
    ++idx;
    return *this;
  }
  _circulate_const_iterator operator++(int) {
    _circulate_const_iterator tmp = *this;
    ++*this;
    return tmp;
  }
  _circulate_const_iterator &operator--() {
    --idx;
    return *this;
  }
  _circulate_const_iterator operator--(int) {
    _circulate_const_iterator tmp = *this;
    --*this;
    return tmp;
  }

  _circulate_const_iterator &operator+=(difference_type n) {
    idx += n;
    return *this;
  }
  _circulate_const_iterator &operator-=(difference_type n) {
    idx -= n;
    return *this;
  }
  _circulate_const_iterator operator+(difference_type n) const {
    return _circulate_const_iterator(circ, idx + n);
  }
  _circulate_const_iterator operator-(difference_type n) const {
    return _circulate_const_iterator(circ, idx - n);
  }
  difference_type operator-(const _circulate_const_iterator &other) const {
    return idx - other.idx;
  }

  reference operator[](difference_type n) const { return *(*this + n); }

  friend _circulate_const_iterator
  operator+(difference_type n, const _circulate_const_iterator &it) {
    return it + n;
  }

  friend bool operator==(const _circulate_const_iterator &a,
                         const _circulate_const_iterator &b) noexcept {
    return a.circ == b.circ && a.idx == b.idx;
  }

  friend std::strong_ordering
  operator<=>(const _circulate_const_iterator &a,
              const _circulate_const_iterator &b) noexcept {
    if (a.circ != b.circ) {
      return a.circ <=> b.circ;
    }
    return a.idx <=> b.idx;
  }

  friend bool operator==(const _circulate_iterator<circulate> &a,
                         const _circulate_const_iterator &b) noexcept {
    return a.circ == b.circ && a.idx == b.idx;
```

```
  }

  friend bool operator!=(const _circulate_iterator<circulate> &a,
                         const _circulate_const_iterator &b) noexcept {
    return a.circ != b.circ || a.idx != b.idx;
  }

  friend std::strong_ordering
  operator<=>(const _circulate_iterator<circulate> &a,
              const _circulate_const_iterator &b) noexcept {
    if (a.circ != b.circ) {
      return a.circ <=> b.circ;
    }
    return a.idx <=> b.idx;
  }
```

(十二) circulate operator=

`include/circulate.hpp` line 243–258 實作：

```
  circulate &operator=(const circulate &other) {
    if (this != &other) {
      c = other.c;
      start = other.start;
    }
    return *this;
  }

  circulate &operator=(circulate &&other) noexcept(
      std::is_nothrow_move_assignable_v<Container>) {
    if (this != &other) {
      c = std::move(other.c);
      start = std::exchange(other.start, 0);
    }
    return *this;
  }
```

(十三) circulate operator[]

`include/circulate.hpp` line 293–298 實作：

```
  constexpr reference operator[](difference_type index) {
    return c[circular_index(index)];
  }
  constexpr const_reference operator[](difference_type index) const {
    return c[circular_index(index)];
  }
```

`Game.hpp` line 193 呼叫：

```
      highway[lane][0] = (highway[lane][1] > 1) ? highway[lane][1] - 1 : 0;
```

`Game.hpp` line 201 呼叫：

```
        highway[nd.lane][0] = nd.holds;
```

`Game.hpp` line 227 呼叫：

```
      int8_t &secondLast = highway[lane][highway[lane].size() - 2];
```

(十四) circulate operator==

`include/circulate.hpp` line 333–335 實作：

```
  friend bool operator==(const circulate &lhs, const circulate &rhs) {
    return lhs.c == rhs.c && lhs.start == rhs.start;
  }
```

(十五) Effect operator<

`Game.hpp` line 40 實作：

```
bool operator<(Effect lhs, Effect rhs) { return lhs.endTime > rhs.endTime; }
```

`Game.hpp` line 65 呼叫（透過 `std::less<Effect>`）：

```
  game_priority_queue<Effect> centerEffects = game_priority_queue<Effect>();
```

`Game.hpp` line 85–86 呼叫（透過 `std::less<Effect>`）：

```
      centerEffects.push(
          {nowMs + msPerFragment * (uint32_t)fragments * 3, COMBO, combo});
```

`Game.hpp` line 96–98 呼叫（透過 `std::less<Effect>`）：

```
    while (!centerEffects.empty() && centerEffects.top().endTime <= nowMs) {
      centerEffects.pop();
    }
```

`Game.hpp` line 137–138 呼叫（透過 `std::less<Effect>`）：

```
      centerEffects.push(
          {nowMs + msPerFragment * (uint32_t)fragments * 3, SCORE, score});
```

`Game.hpp` line 156–157 呼叫（透過 `std::less<Effect>`）：

```
      centerEffects.push(
          {nowMs + msPerFragment * (uint32_t)fragments * 3, SCORE, score});
```

`Game.hpp` line 272–273 呼叫（透過 `std::less<Effect>`）：

```
          centerEffects.push(
              {nowMs + msPerFragment * (uint32_t)fragments * 3, SCORE, score});
```

`Game.hpp` line 303–304 呼叫（透過 `std::less<Effect>`）：

```
      centerEffects.push(
          {nowMs + msPerFragment * (uint32_t)fragments * 3, SCORE, score});
```

十七、 2-C-1-2 Operator Overloading 語法與使用方式描述與範例

(一) allocator operator==

- **檔案與行號：** `include/allocator.hpp` line 36–40
- **語法：**

```
template <class T1, class T2>
constexpr bool operator==(const allocator<T1> &lhs,
                          const allocator<T2> &rhs) noexcept;
```

- **使用方式描述：**比較兩個不同型別 allocator 是否等價。此實作永遠回傳 `true`，符合 STL allocator 的 `is_always_equal` 特性。
- **範例使用：**

```
if (alloc == other.alloc) { /* 移動或複製 assignment 的條件判斷 */ }
```

(二) array operator[]

- **檔案與行號：** `include/array.hpp` line 47–50
- **語法：**

```
constexpr T &operator[](std::size_t index);
constexpr const T &operator[](std::size_t index) const;
```

- **使用方式描述：**提供陣列元素存取，支持 const 與非 const 版本。
- **範例使用：**

```
tempRules[0] ^= (1 << i); // mods/GameOfLife.hpp line~149
tempRules[1] ^= (1 << i); // mods/GameOfLife.hpp line~162
```

(三) array operator== 與 operator<=>

- **檔案與行號：** `include/array.hpp` line 113–124
- **語法：**

```
template <class T, std::size_t N>
constexpr bool operator==(const mystd::array<T, N> &lhs,
                          const mystd::array<T, N> &rhs);

template <class T, std::size_t N>
constexpr std::compare_three_way_result_t<T>
operator<=>(const mystd::array<T, N> &lhs, const mystd::array<T, N> &rhs);
```

- **使用方式描述：**陣列內容比較，可用於 equality 與三向比較。
- **範例使用：**

```
if (arr1 == arr2) { ... }
auto cmp = arr1 <=> arr2;
```

(四) ignore_t operator=

- **檔案與行號：** `include/ignore.hpp` line 6–13
- **語法：**

```
template <class T>
constexpr const ignore_t &operator=(const T &) const noexcept;

template <class T>
constexpr const ignore_t &operator=(T &&) const noexcept;
```

- **使用方式描述：**
  - 此 `operator=` 無條件忽略賦值，回傳自身，不改變任何狀態。
  - 常用於佔位符，特別是在使用 `mystd::tie` 以 tuple 解構賦值時，用來忽略某些不需要的變數。
  - 與 `mystd::tie` 共同使用時，通常寫作：

    `mystd::tie(a, ignore, c) = some_tuple;`

    這樣可以只對需要的成員 `a` 和 `c` 賦值，忽略中間的元素。
- **範例使用：**

```
mystd::tie(x, ignore, z) = mystd::make_tuple(1, 2, 3);
// x = 1, z = 3, 第二個元素被 ignore 忽略
```

(五) priority_queue operator=

- **檔案與行號：** `include/priority_queue.hpp` line 161–175
- **語法：**

```
priority_queue &operator=(const priority_queue &other);
priority_queue &operator=(priority_queue &&other) noexcept;
```

- **使用方式描述：**支援複製與移動賦值，確保成員 `c` 與 `comp` 正確複製或移動。
- **範例使用：**

```
pq1 = pq2;          // copy assignment
pq1 = std::move(pq2); // move assignment
```

(六) _tuple_impl operator=

- **檔案與行號：** `include/tuple.hpp` line 44–48、160–202
- **語法：**

```
  constexpr _tuple_impl &operator=(const _tuple_impl &) noexcept;

  constexpr _tuple_impl &operator=(_tuple_impl &&) noexcept;

  constexpr _tuple_impl &operator=(const _tuple_impl &other)
    requires std::is_copy_assignable_v<Head> && std::is_copy_assignable_v<base>;

  constexpr _tuple_impl &operator=(_tuple_impl &&other) noexcept(
      std::is_nothrow_move_assignable_v<Head> &&
      std::is_nothrow_move_assignable_v<base>)
    requires(std::is_move_assignable_v<Head> && std::is_move_assignable_v<base>);

```

```
template <class UHead, class... UTail>
constexpr _tuple_impl &operator=(const _tuple_impl<I, UHead, UTail...> &other)
  requires(sizeof...(Tail) == sizeof...(UTail)) &&
          std::is_assignable_v<Head &, const UHead &> &&
          std::is_assignable_v<base &, const _tuple_impl<I + 1, UTail...> &>;

template <class UHead, class... UTail>
constexpr _tuple_impl &
operator=(_tuple_impl<I, UHead, UTail...> &&other) noexcept(
    std::is_nothrow_assignable_v<Head &, UHead &&> &&
    std::is_nothrow_assignable_v<base &, _tuple_impl<I + 1, UTail...> &&>)
  requires(sizeof...(Tail) == sizeof...(UTail)) &&
          std::is_assignable_v<Head &, UHead &&> &&
          std::is_assignable_v<base &, _tuple_impl<I + 1, UTail...> &&>;
```

- **使用方式描述：**tuple 內部使用。

(七) tuple operator=

- **檔案與行號：** `include/tuple.hpp` line 433–464
- **語法：**

```
constexpr tuple &operator=(const tuple &other)
  requires(std::is_copy_assignable_v<Types> && ...);

constexpr tuple &operator=(tuple &&other) noexcept(
    (std::is_nothrow_move_assignable_v<Types> && ...))
  requires(std::is_move_assignable_v<Types> && ...);

template <class... UTypes>
constexpr tuple &operator=(const tuple<UTypes...> &other)
  requires(sizeof...(Types) == sizeof...(UTypes)) &&
          (std::is_assignable_v<Types &, const UTypes &> && ...);

template <class... UTypes>
constexpr tuple &operator=(tuple<UTypes...> &&other)
  requires(sizeof...(Types) == sizeof...(UTypes)) &&
          (std::is_assignable_v<Types &, UTypes &&> && ...);
```

- **使用方式描述：**
  - 用於將一個 tuple 的值賦給另一個 tuple。
  - 支援 **複製賦值**、**移動賦值**，以及 **不同型別 tuple 的逐元素賦值**（前提是元素可賦值且長度相同）。
  - 當 tuple 內部的元素是自訂型別時，會依據元素的 copy/move operator= 自動進行對應操作。
  - 可搭配 `mystd::tie` 使用，將 tuple 內部元素依序解構賦值，並可透過 `ignore` 忽略不需要的成員。
- **範例使用：**

```
// 複製賦值
mystd::tuple<int, double, char> t1{1, 2.0, 'a'};
mystd::tuple<int, double, char> t2;
t2 = t1; // t2 = (1, 2.0, 'a')

// 移動賦值
```

```
mystd::tuple<std::string, int> t3{"hello", 42};
mystd::tuple<std::string, int> t4;
t4 = std::move(t3); // t4 = ("hello", 42), t3 進入未定義狀態

// 不同型別 tuple 賦值（可轉換型別）
mystd::tuple<long, float, char> t5;
mystd::tuple<int, double, char> t6{10, 3.14, 'z'};
t5 = t6; // t5 = (10L, 3.14f, 'z')

// 搭配 tie 與 ignore 解構賦值
int a; char c;
mystd::tie(a, mystd::ignore, c) = t6; // 只賦值 t6 的第一與第三元素
```

(八) tuple operator== 與 operator<=>

- **檔案與行號：** `include/tuple.hpp` line 511–540
- **語法：**

```
  friend constexpr bool operator==(const tuple &a, const tuple &b);

  friend constexpr auto operator<=>(const tuple &a, const tuple &b);
```

- **使用方式描述：**
  - `operator==` 比較兩個 tuple 是否大小相同且每個元素依序相等。
  - `operator<=>` 提供三向比較（spaceship operator），依序比對每個元素（字典序比較）。
- **範例使用：**

```
mystd::tuple<int, double, char> t1{1, 2.0, 'a'};
mystd::tuple<int, double, char> t2{1, 2.0, 'a'};
mystd::tuple<int, double, char> t3{1, 3.0, 'a'};

// operator==
bool equal1 = (t1 == t2); // true
bool equal2 = (t1 == t3); // false

// operator<=>
auto cmp1 = (t1 <=> t2); // std::strong_ordering::equal
auto cmp2 = (t1 <=> t3); // std::strong_ordering::less

mystd::tuple<int, double, char> tuples[] = {t3, t1, t2};
std::sort(std::begin(tuples), std::end(tuples),
          [](const auto &a, const auto &b) { return a < b; });
// 排序後順序： t1, t2, t3
```

(九) vector operator=

- **檔案與行號：** `include/vector.hpp` line 318–423
- **語法：**

```
  constexpr vector &operator=(const vector &other);

  constexpr vector &operator=(vector &&other) noexcept(
      mystd::allocator_traits<
          Allocator>::propagate_on_container_move_assignment::value ||
      mystd::allocator_traits<Allocator>::is_always_equal::value);

```

```
  constexpr vector &operator=(std::initializer_list<T> ilist);
```

- **使用方式描述：**vector copy/move assignment 支援 allocator propagation、trivial 與 non-trivial 類型，並支援 initializer list 賦值。
- **範例使用：**

```
vec1 = vec2;              // copy
vec1 = std::move(vec2);    // move
vec1 = {1,2,3,4};         // initializer list
```

(十) vector operator[]

- **檔案與行號：** `include/vector.hpp` line 502–505
- **語法：**

```
constexpr T &operator[](std::size_t index);
constexpr const T &operator[](std::size_t index) const;
```

- **使用方式描述：**支援 vector 元素直接存取。
- **範例使用：**

```
laneEffects[lane].content &= CLEAR;
laneEffects[lane].content |= PERFECT;
```

(十一) vector operator== 與 operator<=>

- **檔案與行號：** `include/vector.hpp` line 968–982
- **語法：**

```
template <class T, class Allocator>
constexpr bool operator==(const mystd::vector<T, Allocator> &lhs,
                          const mystd::vector<T, Allocator> &rhs);

template <class T, class Allocator>
constexpr std::strong_ordering
operator<=>(const mystd::vector<T, Allocator> &lhs,
            const mystd::vector<T, Allocator> &rhs);
```

- **使用方式描述：**
  - `operator==` 比較兩個 vector 是否大小相同且每個元素依序相等。
  - `operator<=>` 提供三向比較（spaceship operator），依序比對每個元素（字典序比較）。
- **範例使用：**

```
mystd::vector<int> v1{1, 2, 3};
mystd::vector<int> v2{1, 2, 3};
mystd::vector<int> v3{1, 2, 4};

// operator==
bool equal1 = (v1 == v2); // true
bool equal2 = (v1 == v3); // false

// operator<=>
```

```
auto cmp1 = (v1 <=> v2); // std::strong_ordering::equal
auto cmp2 = (v1 <=> v3); // std::strong_ordering::less

mystd::vector<int> vecs[] = {v3, v1, v2};
std::sort(std::begin(vecs), std::end(vecs),
          [](const auto &a, const auto &b){ return a < b; });
// 排序後順序： v1, v2, v3
```

(十二) _circulate_iterator and _circulate_const_iterator operators

- **檔案與行號：** `include/circulate.hpp` line 30–89, 110–188
- **語法：**支援：

```
operator*, operator->, operator++, operator--,
operator+=, operator-=, operator+, operator-,
operator[], operator==, operator!=, operator<=>
```

- **使用方式描述：**circulate iterator 提供隨機存取、算術運算、比較運算，支持 const 與非 const iterator。
- **範例使用：**

```
auto it = circ.begin();
++it;
*it = value;
auto cmp = it1 <=> it2;
```

(十三) circulate operator=

- **檔案與行號：** `include/circulate.hpp` line 243–258
- **語法：**

```
circulate &operator=(const circulate &other);
circulate &operator=(circulate &&other) noexcept(...);
```

- **使用方式描述：**支援 copy/move assignment，更新內部容器與 start 索引。
- **範例使用：**

```
c1 = c2;              // copy
c1 = std::move(c2);    // move
```

(十四) circulate operator[]

- **檔案與行號：** `include/circulate.hpp` line 293–298
- **語法：**

```
constexpr reference operator[](difference_type index);
constexpr const_reference operator[](difference_type index) const;
```

- **使用方式描述：**支援循環索引存取元素。
- **範例使用：**

```
highway[lane][0] = (highway[lane][1] > 1) ? highway[lane][1]-1 : 0;
```

(十五) circulate operator==

- **檔案與行號：** `include/circulate.hpp` line 333–335
- **語法：**

```
friend bool operator==(const circulate &lhs, const circulate &rhs);
```

- **使用方式描述：**判斷兩個 circulate 對象是否相等（容器內容與 start 均相同）。

(十六) Effect operator<

- **檔案與行號：** `Game.hpp` line 40
- **語法：**

```
bool operator<(Effect lhs, Effect rhs);
```

- **使用方式描述：**用於 priority queue 的比較，依 endTime 決定排序。
- **範例使用：**

```
game_priority_queue<Effect> centerEffects;
centerEffects.push({endTime, type, value});
```

十八、 2-C-1-3 Operator Overloading 演算法設計

(一) 總論

各 operator 之實作細節、語法、使用方式與範例使用，已於前述章節中分別說明；以下僅就整體演算法設計之理念進行總結性說明：

- 符合 C++20 標準與語意，例如比較運算子與三向比較運算子（`operator<=>`）使用字典序比較演算法。
- 與 STL 及現代 C++ idioms 保持一致，使自製容器、迭代器或型別能自然地套用既有演算法。
- 滿足 Structured Bindings 與 ADL（Argument-Dependent Lookup）等語言機制對介面的需求。(請參見 **Structured Bindings** 與 **Argument-Dependent Lookup (ADL)**
- 配合物件生命週期與資源管理需求，遵循 **Rule of Three / Five / Zero**，確保型別在指派語意上的一致性與安全性。

(二) Argument-Dependent Lookup (ADL)

ADL, also known as **Koenig lookup**, allows the compiler to search for functions not only in the current scope but also in the namespaces of the function argument types. This enables intuitive and unqualified function calls.

(三) Structured Bindings

Structured bindings, introduced in C++17, allow unpacking of tuples, pairs, arrays, or user-defined types into individual variables. In my STL implementation, `include/tuple-using.hpp` provides `std::tuple_size` and `std::tuple_element` support for `mystd::array` and `mystd::tuple`.

(四) 2-C-1-4 Operator Overloading 有無的差異

透過適當且具節制的 operator overloading，可達成下列目標：

- 符合 C++ 標準、語意及現代 C++ idioms，使自製容器或型別能自然地套用既有演算法（如 `mystd::qsort`、`std::sort`、`std::ranges`、`std::less`）。
- 提升程式碼的可讀性與表意性，避免以顯式函式呼叫取代本質上屬於「運算」或「比較」的語意行為。
- 減少呼叫端的樣板程式碼（boilerplate），使使用者端程式碼更貼近問題本身的抽象層次。

相對地，若缺乏必要的運算子支援，呼叫端往往需以輔助函式或顯式介面取代，導致程式碼冗長、語意分散，且不利於與標準函式庫及泛型演算法整合。

(五) 2-C-1-5 Operator Overloading 巧思

(六) vector operator= 優化

請參見 **2-B-1-2 指標成員處理** 中關於之的 **描述**。

(七) _circulate_iterator and _circulate_const_iterator operators

在設計 `_circulate_iterator` 與 `_circulate_const_iterator` 時，操作符重載的巧思主要體現在如何同時支援「循環訪問」、「隨機存取迭代器」與「STL 風格迭代器運算子語義」：

- 自動回繞通過 `circulate` 的 `operator[]` 呼叫模運算達成，故無須保持內部索引 `idx` 的範圍，任何整數皆可。
- `operator*` 與 `operator->` 提供標準解引用語義，直接透過內部索引 `idx` 訪問循環容器 `circ` 的元素。
- 前/後置 `operator++`、`operator--`、`operator+`、`operator-`、`operator+=`、`operator-=` 均僅基於索引值 `idx`，$O(1)$ 時間完成。
- `operator[]` 使用 `*(*this + n)` 形式提供隨機訪問能力。
- **friend 運算符：** 支援 `n + it` 的算術運算，並重載比較運算符，也支援 `_circulate_iterator` 與 `_circulate_const_iterator` 的混合比較。

總結：透過索引管理而非指針直接操作，實現對循環容器元素的高效訪問與運算符重載，並保持 STL 隨機訪問迭代器接口。

(八) tuple operator== 與 operator<=>

我最初嘗試為 `_tuple_impl`（tuple 的內部遞迴繼承實作）實作 `operator<=>`，但遇到若干困難：由於 tuple 的各元素可能為不同型別，其比較運算可能返還不同型別（`std::partial ordering`、`std::weak_ordering` 或 `std::strong_ordering`），使用 `auto` 進行型別推導不符合 C++ 標準；而嘗試以 `conditional_t` 遞迴推導型別，又會因未定義的模板參數（`_tuple_impl<I, [empty]>`）而造成初始化問題，即使這些分支最終不會被選用。雖然這些在 g++ 上均可以編譯通過，但不符合標準，且在 clang 上無法編譯。

後來，我在 `_tuple_impl` 中新增了 `element_type` 的內部型別別名，透過索引遞迴定義該別名。接著在 `tuple` 類別中直接使用這個別名與成員函式 `get`，以迭代方式（而非遞迴方式）推導型別進行初始化，成功解決了上述問題，並符合標準。

十九、 2-C-2-1 static member 宣告、2-C-3-2 static member 程式碼位置

(一) include/allocator.hpp

Line 198–299：

```
198   static constexpr pointer allocate(Alloc &a, size_type n) {
199     return a.allocate(n);
200   }
201
202 private:
203   template <typename A>
204   static constexpr auto test(int)
205       -> decltype(std::declval<A &>().allocate(
206                       std::declval<size_type>(),
207                       std::declval<const_void_pointer>()),
208                   std::true_type{});
209
210   template <typename> static constexpr std::false_type test(...);
211
212   static constexpr bool has_allocate_with_hint =
213       decltype(test<Alloc>(0))::value;
214
215 public:
216   static constexpr pointer allocate(Alloc &a, size_type n,
217                                     const_void_pointer hint) {
218     if constexpr (has_allocate_with_hint) {
219       return a.allocate(n, hint);
220     } else {
221       (void)hint;
222       return a.allocate(n);
223     }
224   }
225
226   static constexpr void deallocate(Alloc &a, pointer p, size_type n) {
227     a.deallocate(p, n);
228   }
229
230   static constexpr size_type max_size(const Alloc &a) noexcept {
231     if constexpr (requires { a.max_size(); }) {
232       return a.max_size();
233     } else {
234       return std::numeric_limits<size_type>::max() / sizeof(value_type);
235     }
236   }
237
238 private:
239   template <typename A, typename T, typename... Args>
240   static constexpr auto has_construct(int)
241       -> decltype(std::declval<A>().construct(std::declval<T *>(),
242                                               std::declval<Args>()...),
243                   std::true_type{});
244   template <typename, typename, typename...>
245   static constexpr std::false_type has_construct(...);
246
247   template <typename A, typename T>
248   static constexpr auto has_destroy(int)
249       -> decltype(std::declval<A>().destroy(std::declval<T *>()),
250                   std::true_type{});
```

```
  template <typename, typename>
  static constexpr std::false_type has_destroy(...);

public:
  template <typename T, typename... Args>
  static constexpr void construct(Alloc &a, T *p, Args &&...args) {
    if constexpr (decltype(has_construct<Alloc, T, Args...>(0))::value) {
      a.construct(p, std::forward<Args>(args)...);
    } else {
      std::construct_at(p, std::forward<Args>(args)...);
    }
  }

  template <typename T> static constexpr void destroy(Alloc &a, T *p) {
    if constexpr (decltype(has_destroy<Alloc, T>(0))::value) {
      a.destroy(p);
    } else {
      std::destroy_at(p);
    }
  }

  static constexpr Alloc select_on_container_copy_construction(const Alloc &a) {
    if constexpr (requires { a.select_on_container_copy_construction(); }) {
      return a.select_on_container_copy_construction();
    } else {
      return a;
    }
  }
};

template <class Alloc>
static typename allocator_traits<Alloc>::pointer
allocate(Alloc &a, typename allocator_traits<Alloc>::size_type n) {
  return allocator_traits<Alloc>::allocate(a, n);
}

template <class Alloc>
static typename allocator_traits<Alloc>::pointer
allocate(Alloc &a, typename allocator_traits<Alloc>::size_type n,
         typename allocator_traits<Alloc>::const_void_pointer hint) {
  return allocator_traits<Alloc>::allocate(a, n, hint);
}

template <class Alloc>
static constexpr void
deallocate(Alloc &a, typename allocator_traits<Alloc>::pointer p,
           typename allocator_traits<Alloc>::size_type n) {
  allocator_traits<Alloc>::deallocate(a, p, n);
}
```

(二) include/tuple.hpp

Line 512–516：

```
  template <std::size_t... I>
  static constexpr bool tuple_equal_impl(const tuple &a, const tuple &b,
                                         std::index_sequence<I...>) {
    return ((a.get<I>() == b.get<I>()) && ...);
  }
```

Line 524–534：

```
  template <std::size_t... I>
```

```
  static constexpr auto tuple_three_way_impl(const tuple &a, const tuple &b,
                                             std::index_sequence<I...>)
      -> std::common_comparison_category_t<decltype(a.get<I>() <=>
                                                    b.get<I>())...> {
    using result_t = std::common_comparison_category_t<decltype(a.get<I>() <=>
                                                                b.get<I>())...>;
    result_t result = result_t::equal;
    ((result = (result != 0 ? result : (a.get<I>() <=> b.get<I>()))), ...);
    return result;
  }
```

(三) mods/GameOfLife.hpp

Line 14：

```
static mystd::array<uint16_t, 2> currentRules = {0b111111111, 0};
```

Line 18：

```
  const auto &rules = currentRules;
```

Line 105：

```
  mystd::array<uint16_t, 2> tempRules = currentRules;
```

(四) Renderer.hpp

Line 306–318：

```
  static void drawTextBackground(SDL_Renderer *rnd, int centerX, int centerY,
                                 const std::string &text, TTF_Font *font,
                                 int padding, SDL_Color bg) {
    int w = 0, h = 0;
    TTF_SizeUTF8(font, text.c_str(), &w, &h);

    SDL_Rect rect{centerX - w / 2 - padding, centerY - h / 2 - padding,
                  w + padding * 2, h + padding * 2};

    SDL_SetRenderDrawBlendMode(rnd, SDL_BLENDMODE_BLEND);
    SDL_SetRenderDrawColor(rnd, bg.r, bg.g, bg.b, bg.a);
    SDL_RenderFillRect(rnd, &rect);
  }
```

(五) BeatRunner.cpp

Line 1030–1041：

```
  static bool initialized = false;
  static Uint32 lastFragmentTime;
  static double accumulatedError = 0.0;

  if (!initialized) {
    lastFragmentTime = currentTime;
    initialized = true;
    return;
  }

  Uint32 actualDelta = currentTime - lastFragmentTime;
  accumulatedError += exactFragmentMs - (double)actualDelta;
```

(六) Mods.hpp

Line 15–20

```
inline std::map<std::string, mystd::tuple<ModFunc, ModFunc, SettingsFunc>> &
getModMap() {
  static std::map<std::string, mystd::tuple<ModFunc, ModFunc, SettingsFunc>>
      modMap;
  return modMap;
}
```

二十、 2-C-2-3 static member 用途

(一) Static Initialization Order Fiasco (SIOF)

In C++, the initialization order of non-local static objects is governed by the following rules:

- **Within a single translation unit:** Non-local static objects are initialized in the order of their definitions within that translation unit.
- **Across different translation units:** The initialization order of non-local static objects defined in different translation units is unspecified.

As a result, if a static object in one translation unit depends on another static object defined in a different translation unit during its initialization, there is no guarantee that the depended-on object has already been initialized. Accessing such an object before its initialization leads to undefined behavior.

To mitigate or avoid SIOF, the following design strategies are generally recommended:

- Prefer `constexpr` or `inline` variables where possible, as their initialization is guaranteed at compile time or follows well-defined rules.
- Use function-local static objects, whose initialization is guaranteed to occur upon first use and is thread-safe since C++11.
- Avoid cross-translation-unit dependencies between non-local static objects, especially during their initialization phase.

(二) 型別層級（type-level）的工具函式

在 `include/allocator.hpp` 中，`allocator_traits` 將所有操作（`allocate`、`deallocate`、`construct`、`destroy`、`max_size` 等）設計為 static member function。此設計符合 C++20 標準，其核心意義包括：

- 行為取決於 `Alloc` 型別本身，而非 traits 物件實例。
- 避免建立不必要的 traits 實體，降低語意與實作負擔。
- 可在編譯期透過 SFINAE 或 `if constexpr` 檢測 allocator 是否提供特定介面。請參見 **Substitution Failure Is Not An Error (SFINAE)** 與 **constexpr function**。

(三) 與物件成員無關的純工具函式

例如 `include/tuple.hpp` 中的 `tuple_equal_impl` 與 `tuple_three_way_impl`，以及 `Renderer.hpp` 中的 `drawTextBackground`，這些函式與物件實例無關，作為純粹的型別工具函式使用。

(四) 跨物件共享的狀態或設定

在 `mods/GameOfLife.hpp` 中，`currentRules` 以 static 變數形式存在於 namespace `gameOfLife` 中，表示該模組的全域規則狀態，可供模組內多個函式共享。

(五) 區域內部的延遲初始化與狀態保存

在 `BeatRunner.cpp` 中，透過區域 static 變數（`initialized`、`lastFragmentTime`、`accumulatedError`）：

- 實現僅初始化一次，並在每次呼叫時更新狀態。
- 避免將此類狀態提升為全域變數，降低命名衝突風險。
- 明確限定變數作用範圍，提升程式結構可讀性。

(六) 延遲建立且全程共享的容器

在 `Mods.hpp` 中，`getModMap` 回傳一個區域 static 變數：

- 保證模組註冊表僅建立一次。
- 避免 **Static Initialization Order Fiasco (SIOF)**，因為初始化延遲到首次使用時才執行。
- 提供全域存取點。

二十一、 2-C-2-4 static member 有無的差異

若上述設計未採用 static member，將可能產生以下問題：

- 必須建立不具實際意義的物件實例，增加使用成本。
- 容易誤導使用者或無法自然表達共享語意。
- 實作上更容易引入全域變數、重複初始化或 SIOF 問題。

相對地，透過 static member：

- 語意上清楚區分「型別層級」與「物件層級」的責任。
- 可能降低記憶體配置與初始化成本。
- 與 STL 的設計風格保持一致。
- 提升程式碼可讀性與可維護性。

二十二、 2-C-3-1 const member 宣告、2-C-3-2 const member 程式碼位置（僅列沈威宇撰寫部分）

(一) include/allocator.hpp

Line 409–432：

```
template <class T, class Alloc, class NonPair>
  requires is_pair_v<std::remove_cvref_t<T>> &&
           (!std::is_constructible_v<std::remove_cvref_t<T>,
                                     std::remove_cvref_t<NonPair>>)
constexpr auto uses_allocator_construction_args(const Alloc &alloc,
                                                NonPair &&np) noexcept {
  struct pair_constructor {
    const Alloc &alloc_;
    NonPair &u_;
    constexpr auto reconstruct(const std::remove_cv_t<T> &p) const {
      return std::make_obj_using_allocator<std::remove_cv_t<T>>(alloc_, p);
    }
    constexpr auto reconstruct(std::remove_cv_t<T> &&p) const {
      return std::make_obj_using_allocator<std::remove_cv_t<T>>(alloc_,
                                                                std::move(p));
    }
    constexpr operator std::remove_cv_t<T>() const {
      return reconstruct(std::remove_cv_t<T>(std::forward<NonPair>(u_).first,
                                             std::forward<NonPair>(u_).second));
    }
  };
  pair_constructor construction{alloc, np};
  return make_tuple(construction);
}
```

(二) include/array.hpp

Line 41–45：

```
  constexpr const T &at(std::size_t index) const {
    if (index >= N)
      throw std::out_of_range("array");
    return elems[index];
  }
```

Line 48–50：

```
  constexpr const T &operator[](std::size_t index) const {
    return elems[index];
  }
```

Line 53：

```
  constexpr const T &front() const { return elems[0]; }
```

Line 56：

```
  constexpr const T &back() const { return elems[N - 1]; }
```

Line 59：

```
  constexpr const T *data() const noexcept { return elems; }
```

Line 62–63：

```
constexpr const T *begin() const noexcept { return elems; }
constexpr const T *cbegin() const noexcept { return elems; }
```

Line 66–67：

```
constexpr const T *end() const noexcept { return elems + N; }
constexpr const T *cend() const noexcept { return elems + N; }
```

Line 72–77：

```
constexpr const_reverse_iterator rbegin() const noexcept {
  return const_reverse_iterator(end());
}
constexpr const_reverse_iterator crbegin() const noexcept {
  return const_reverse_iterator(end());
}
```

Line 82–91：

```
constexpr const_reverse_iterator rend() const noexcept {
  return const_reverse_iterator(begin());
}
constexpr const_reverse_iterator crend() const noexcept {
  return const_reverse_iterator(begin());
}

constexpr bool empty() const noexcept { return N == 0; }
constexpr std::size_t size() const noexcept { return N; }
constexpr std::size_t max_size() const noexcept { return N; }
```

(三) include/vector.hpp

Line 488：

```
constexpr allocator_type get_allocator() const noexcept { return alloc; }
```

Line 496–500：

```
constexpr const T &at(std::size_t index) const {
  if (index >= sz)
    throw std::out_of_range("vector");
  return elems[index];
}
```

Line 503–505：

```
constexpr const T &operator[](std::size_t index) const {
  return elems[index];
}
```

Line 507：

```
constexpr const T &front() const { return *elems; }
```

Line 509：

```
constexpr const T &back() const { return elems[sz - 1]; }
```

Line 512：

```
constexpr const T *data() const noexcept { return elems; }
```

Line 515–516：

```
constexpr const_iterator begin() const noexcept { return elems; }
constexpr const_iterator cbegin() const noexcept { return elems; }
```

Line 519–520：

```
constexpr const_iterator end() const noexcept { return elems + sz; }
constexpr const_iterator cend() const noexcept { return elems + sz; }
```

Line 525–530：

```
constexpr const_reverse_iterator rbegin() const noexcept {
  return const_reverse_iterator(cend());
}
constexpr const_reverse_iterator crbegin() const noexcept {
  return const_reverse_iterator(cend());
}
```

Line 535–548：

```
constexpr const_reverse_iterator rend() const noexcept {
  return const_reverse_iterator(cbegin());
}
constexpr const_reverse_iterator crend() const noexcept {
  return const_reverse_iterator(cbegin());
}

constexpr bool empty() const noexcept { return sz == 0; }
constexpr std::size_t size() const noexcept { return sz; }
constexpr std::size_t max_size() const noexcept {
  return std::min(
      mystd::allocator_traits<Allocator>::max_size(alloc),
      static_cast<std::size_t>(std::numeric_limits<std::ptrdiff_t>::max()));
}
```

(四) include/circulate.hpp

Line 60–68：

```
_circulate_iterator operator+(difference_type n) const {
  return _circulate_iterator(circ, idx + n);
}
_circulate_iterator operator-(difference_type n) const {
  return _circulate_iterator(circ, idx - n);
}
difference_type operator-(const _circulate_iterator &other) const {
  return idx - other.idx;
}
```

Line 101：

```
const circulate *circ;
```

Line 110–111：

```
reference operator*() const { return (*circ)[idx]; }
pointer operator->() const { return &(**this); }
```

Line 140–150：

```
_circulate_const_iterator operator+(difference_type n) const {
  return _circulate_const_iterator(circ, idx + n);
}
_circulate_const_iterator operator-(difference_type n) const {
  return _circulate_const_iterator(circ, idx - n);
}
difference_type operator-(const _circulate_const_iterator &other) const {
  return idx - other.idx;
}

reference operator[](difference_type n) const { return *(*this + n); }
```

Line 207–211：

```
constexpr size_type size() const noexcept { return c.size(); }
constexpr difference_type ssize() const noexcept {
  return static_cast<difference_type>(c.size());
}
constexpr bool empty() const noexcept { return c.size() == 0; }
```

Line 216–224：

```
constexpr size_type circular_index(difference_type i) const noexcept {
  if (empty())
    return 0;
  difference_type idx = static_cast<difference_type>(start) + i;
  idx %= ssize();
  if (idx < 0)
    idx += ssize();
  return static_cast<size_type>(idx);
}
```

Line 296–298：

```
constexpr const_reference operator[](difference_type index) const {
  return c[circular_index(index)];
}
```

Line 301：

```
constexpr const_reference front() const { return c[start]; }
```

Line 303–305：

```
constexpr const_reference back() const {
  return c[circular_index(ssize() - 1)];
}
```

Line 308–309：

```
const_iterator begin() const noexcept { return const_iterator(this, 0); }
const_iterator cbegin() const noexcept { return const_iterator(this, 0); }
```

Line 311–312：

```
const_iterator end() const noexcept { return const_iterator(this, ssize()); }
const_iterator cend() const noexcept { return const_iterator(this, ssize()); }
```

Line 318–323：

```
const_reverse_iterator rbegin() const noexcept {
  return const_reverse_iterator(end());
}
const_reverse_iterator crbegin() const noexcept {
  return const_reverse_iterator(cend());
}
```

Line 326–331：

```
const_reverse_iterator rend() const noexcept {
  return const_reverse_iterator(begin());
}
const_reverse_iterator crend() const noexcept {
  return const_reverse_iterator(cbegin());
}
```

(五) include/ignore.hpp

Line 6–13：

```
template <class T>
constexpr const ignore_t &operator=(const T &) const noexcept {
  return *this;
}

template <class T> constexpr const ignore_t &operator=(T &&) const noexcept {
  return *this;
}
```

(六) include/priority_queue.hpp

Line 177–181：

```
const_reference top() const { return c.front(); }

bool empty() const { return c.empty(); }

size_type size() const { return c.size(); }
```

(七) include/tuple.hpp

Line 222–231：

```
template <std::size_t J>
constexpr decltype(auto) get() const &
  requires(J >= I && J <= I + sizeof...(Tail))
{
  if constexpr (I == J) {
    return (head);
  } else {
    return static_cast<const base &>(*this).template get<J>();
  }
}
```

Line 244–253：

```
template <std::size_t J>
constexpr decltype(auto) get() const &&
  requires(J >= I && J <= I + sizeof...(Tail))
```

```
  {
    if constexpr (I == J) {
      return std::move(head);
    } else {
      return static_cast<const base &&>(*this).template get<J>();
    }
  }
```

Line 474–480：

```
  template <std::size_t J> constexpr decltype(auto) get() const & noexcept {
    return _impl.template get<J>();
  }

  template <std::size_t J> constexpr decltype(auto) get() const && noexcept {
    return std::move(_impl).template get<J>();
  }
```

(八) mods/GameOfLife.hpp

Line 257–282：

```
// Self-register with settings function
namespace {
const bool registered = [] {
  registerMod("Game of Life (hold notes counted as alive cell, before "
              "new fragments load)",
              &gameOfLife::gameOfLifeHoldAlive, nullptr,
              gameOfLife::gameOfLifeSettings);

  registerMod("Game of Life (hold notes counted as alive cell, after new "
              "fragments load)",
              nullptr, &gameOfLife::gameOfLifeHoldAlive,
              gameOfLife::gameOfLifeSettings);

  registerMod("Game of Life (hold notes counted as dead cell, before new "
              "fragments load)",
              &gameOfLife::gameOfLifeHoldDead, nullptr,
              gameOfLife::gameOfLifeSettings);

  registerMod("Game of Life (hold notes counted as dead cell, after new "
              "fragments load)",
              nullptr, &gameOfLife::gameOfLifeHoldDead,
              gameOfLife::gameOfLifeSettings);

  return true;
}();
} // namespace
```

(九) Game.hpp

Line 22–32：

```
const uint32_t NO_LANE_EFFECT = 0u;
const uint32_t PERFECT = 1u;
const uint32_t GREAT = 1u << 1;
const uint32_t GOOD = 1u << 2;
const uint32_t BAD = 1u << 3;
const uint32_t MISS = 1u << 4;
const uint32_t HOLD_RELEASED = 1u << 5;
const uint32_t CLEAR = ~(PERFECT | GREAT | GOOD | BAD | MISS | HOLD_RELEASED);

```

```
const uint32_t COMBO = 1u;
const uint32_t SCORE = 2u;
```

二十三、 2-C-3-3 const member 用途

(一) 支援 const 版本的物件

在 `include/array.hpp`、`include/vector.hpp` 及 `include/circulate.hpp` 中，對容器的訪問函式（如 `at`、`operator[]`、`front`、`back`、`data`、`begin/ end`、`rbegin/ rend`）皆宣告有 `const` 版本，`include/ignore.hpp` 中的 `operator=` 則宣告為 `const`，以支援 const 版本的物件。

(二) 保證不可變的值封裝

在 `include/allocator.hpp` 的 `pair_constructor` 中，`alloc_` 為 const reference：

- 確保 allocator 不會被意外修改。
- 配合 `reconstruct` 方法實現安全的型別轉換與建構。

(三) 常數定義

使用 const 定義無須再被更改的常數。

二十四、 2-C-3-4 const member 有無的差異

若未使用 const member，可能造成以下問題：

- 容器訪問函式無法保證不修改內部資料，降低編譯期安全性。
- 無法支援 const 版本物件。
- 呼叫端不易分辨哪些操作是只讀、哪些可能修改物件狀態。
- 增加潛在錯誤修改風險。

相對地，使用 const member 帶來的效益：

- 清楚區分「查詢/只讀操作」與「修改操作」的責任範圍。
- 提升程式碼可讀性、可維護性及 STL 相容性。
- 支援編譯期錯誤檢查，降低 runtime 錯誤風險。
- 常數封裝值的不可變性使程式設計更安全、語意更明確。

二十五、 2-C-4-1 polymorphism 宣告

本專題未使用 runtime polymorphism，而是廣泛採用 compile-time polymorphism，以降低執行時的額外開銷，其所有實作均在我撰寫的部分完成。

在實作上，我大量使用了 template，並提供 **Class Template Argument Deduction (CTAD)** 以簡化使用，此外也於 `include/tuple.hpp` 中運用 **Template Compile-Time Recursion** 與 **Empty Base Optimization (EBO)**，實現泛型且高效的操作。

由於 `include/` 資料夾下多數類別與函式均採用 compile-time polymorphism，以下內容主要列出各類別與函式的 compile-time polymorphism 成員宣告（signature），對 `if constexpr`、CTAD 或 SFINAE 等編譯期分派機制多予省略，相關設計理念與核心概念將於 **2-C-4-3 polymorphism 相關的 class 或 function 間的對應關係** 中說明。

(一) include/algorithm-copy.hpp

Line 7–9：

```
1 template <std::input_iterator InputIt, std::weakly_incrementable OutputIt>
2   requires std::indirectly_writable<OutputIt, std::iter_value_t<InputIt>>
3 constexpr OutputIt copy(InputIt first, InputIt last, OutputIt d_first)
```

Line 34–38：

```
1 template <std::input_iterator InputIt, std::weakly_incrementable OutputIt,
2          typename UnaryPred>
3   requires std::indirectly_writable<OutputIt, std::iter_value_t<InputIt>>
4 constexpr OutputIt copy_if(InputIt first, InputIt last, OutputIt d_first,
5                            UnaryPred pred);
```

Line 49–52：

```
1 template <std::bidirectional_iterator BidirIt1,
2          std::bidirectional_iterator BidirIt2>
3 constexpr BidirIt2 copy_backward(BidirIt1 first, BidirIt1 last,
4                                  BidirIt2 d_last);
```

(二) include/algorithm-find.hpp

Line 7–8：

```
1 template <class InputIt, class UnaryPred>
2 constexpr InputIt find_if(InputIt first, InputIt last, UnaryPred p);
```

Line 17–18：

```
1 template <class InputIt, class T>
2 constexpr InputIt find(InputIt first, InputIt last, const T &value);
```

Line 22–23：

```
1 template <class InputIt, class UnaryPred>
2 constexpr InputIt find_if_not(InputIt first, InputIt last, UnaryPred q);
```

(三) include/algorithm-heap.hpp

Line 11–14：

```
1 template <std::random_access_iterator It>
2   requires std::is_swappable_v<It> && std::is_move_constructible_v<It> &&
3            std::is_move_assignable_v<It>
4 constexpr void push_heap(It first, It last);
```

Line 28–32：

```
template <std::random_access_iterator It, class Comp>
  requires std::predicate<Comp,
                          const typename std::iterator_traits<It>::reference,
                          const typename std::iterator_traits<It>::reference>
constexpr void push_heap(It first, It last, Comp comp);
```

Line 46–49：

```
template <std::random_access_iterator It>
  requires std::is_swappable_v<It> && std::is_move_constructible_v<It> &&
           std::is_move_assignable_v<It>
constexpr void pop_heap(It first, It last);
```

Line 76–80：

```
template <std::random_access_iterator It, class Comp>
  requires std::predicate<Comp,
                          const typename std::iterator_traits<It>::reference,
                          const typename std::iterator_traits<It>::reference>
constexpr void pop_heap(It first, It last, Comp comp);
```

Line 107–110：

```
template <std::random_access_iterator It>
  requires std::is_swappable_v<It> && std::is_move_constructible_v<It> &&
           std::is_move_assignable_v<It>
constexpr void make_heap(It first, It last);
```

Line 135–139：

```
template <std::random_access_iterator It, class Comp>
  requires std::predicate<Comp,
                          const typename std::iterator_traits<It>::reference,
                          const typename std::iterator_traits<It>::reference>
constexpr void make_heap(It first, It last, Comp comp);
```

Line 164–165：

```
template <std::random_access_iterator It>
constexpr void sort_heap(It first, It last);
```

Line 172–173：

```
template <std::random_access_iterator It, class Comp>
constexpr void sort_heap(It first, It last, Comp comp);
```

Line 180–181：

```
template <std::random_access_iterator It>
constexpr bool is_heap(It first, It last);
```

Line 197–198：

```
template <std::random_access_iterator It, class Comp>
constexpr bool is_heap(It first, It last, Comp comp);
```

Line 214–215：

```
template <std::random_access_iterator It>
constexpr It is_heap_until(It first, It last);
```

Line 231–232：

```
template <std::random_access_iterator It, class Comp>
constexpr It is_heap_until(It first, It last, Comp comp);
```

(四) include/algorithm-remove.hpp

Line 9–10：

```
template <class ForwardIt, class UnaryPred>
constexpr ForwardIt remove_if(ForwardIt first, ForwardIt last, UnaryPred p);
```

Line 24–25：

```
template <class ForwardIt, class T>
constexpr ForwardIt remove(ForwardIt first, ForwardIt last, const T &value);
```

(五) include/allocator.hpp

Line 12：

```
template <class T> struct allocator;
```

Line 21：

```
  template <class U> constexpr allocator(const allocator<U> &other) noexcept {};
```

Line 36–38：

```
template <class T1, class T2>
constexpr bool operator==(const allocator<T1> &lhs,
                          const allocator<T2> &rhs) noexcept;
```

Line 42–316：

```
template <typename Alloc, typename T, typename = void>
struct __rebind_helper {};

template <typename Alloc, typename T>
struct __rebind_helper<Alloc, T,
                       std::void_t<typename Alloc::template rebind<T>::other>> {
  using type = typename Alloc::template rebind<T>::other;
};

template <template <class, class...> class Alloc, class U, class... Args,
          class T>
struct __rebind_helper<Alloc<U, Args...>, T> {
  using type = Alloc<T, Args...>;
};

template <typename Alloc> struct allocator_traits {
  using allocator_type = Alloc;
  using value_type = typename Alloc::value_type;

private:
  template <typename A, typename = void> struct pointer_helper {
    using type = value_type *;
  };
  template <typename A>
  struct pointer_helper<A, std::void_t<typename A::pointer>> {
    using type = typename A::pointer;
```

```
  };

public:
  using pointer = typename pointer_helper<Alloc>::type;

private:
  template <typename A, typename = void> struct const_pointer_helper {
    using type = typename std::pointer_traits<pointer>::template rebind<
        const value_type>;
  };
  template <typename A>
  struct const_pointer_helper<A, std::void_t<typename A::const_pointer>> {
    using type = typename A::const_pointer;
  };

public:
  using const_pointer = typename const_pointer_helper<Alloc>::type;

private:
  template <typename A, typename = void> struct void_pointer_helper {
    using type = typename std::pointer_traits<pointer>::template rebind<void>;
  };
  template <typename A>
  struct void_pointer_helper<A, std::void_t<typename A::void_pointer>> {
    using type = typename A::void_pointer;
  };

public:
  using void_pointer = typename void_pointer_helper<Alloc>::type;

private:
  template <typename A, typename = void> struct const_void_pointer_helper {
    using type =
        typename std::pointer_traits<pointer>::template rebind<const void>;
  };
  template <typename A>
  struct const_void_pointer_helper<
      A, std::void_t<typename A::const_void_pointer>> {
    using type = typename A::const_void_pointer;
  };

public:
  using const_void_pointer = typename const_void_pointer_helper<Alloc>::type;

private:
  template <typename A, typename = void> struct difference_type_helper {
    using type = std::pointer_traits<pointer>::difference_type;
  };
  template <typename A>
  struct difference_type_helper<A, std::void_t<typename A::difference_type>> {
    using type = typename A::difference_type;
  };

public:
  using difference_type = typename difference_type_helper<Alloc>::type;

private:
  template <typename A, typename = void> struct size_type_helper {
    using type = std::make_unsigned<difference_type>::type;
  };
  template <typename A>
  struct size_type_helper<A, std::void_t<typename A::size_type>> {
    using type = typename A::size_type;
  };

```

```
public:
  using size_type = typename size_type_helper<Alloc>::type;

private:
  template <typename A, typename = void>
  struct propagate_on_container_copy_assignment_helper {
    using type = std::false_type;
  };
  template <typename A>
  struct propagate_on_container_copy_assignment_helper<
      A, std::void_t<typename A::propagate_on_container_copy_assignment>> {
    using type = typename A::propagate_on_container_copy_assignment;
  };

public:
  using propagate_on_container_copy_assignment =
      typename propagate_on_container_copy_assignment_helper<Alloc>::type;

private:
  template <typename A, typename = void>
  struct propagate_on_container_move_assignment_helper {
    using type = std::false_type;
  };
  template <typename A>
  struct propagate_on_container_move_assignment_helper<
      A, std::void_t<typename A::propagate_on_container_move_assignment>> {
    using type = typename A::propagate_on_container_move_assignment;
  };

public:
  using propagate_on_container_move_assignment =
      typename propagate_on_container_move_assignment_helper<Alloc>::type;

private:
  template <typename A, typename = void>
  struct propagate_on_container_swap_helper {
    using type = std::false_type;
  };
  template <typename A>
  struct propagate_on_container_swap_helper<
      A, std::void_t<typename A::propagate_on_container_swap>> {
    using type = typename A::propagate_on_container_swap;
  };

public:
  using propagate_on_container_swap =
      typename propagate_on_container_swap_helper<Alloc>::type;

private:
  template <typename A, typename = void> struct is_always_equal_helper {
    using type = std::is_empty<Alloc>::type;
  };
  template <typename A>
  struct is_always_equal_helper<A, std::void_t<typename A::is_always_equal>> {
    using type = typename A::is_always_equal;
  };

public:
  using is_always_equal = typename is_always_equal_helper<Alloc>::type;

  template <typename U>
  using rebind_alloc = typename __rebind_helper<Alloc, U>::type;

  template <typename U> using rebind_traits = allocator_traits<rebind_alloc<U>>;

```

```
  static constexpr pointer allocate(Alloc &a, size_type n) {
    return a.allocate(n);
  }

private:
  template <typename A>
  static constexpr auto test(int)
      -> decltype(std::declval<A &>().allocate(
                      std::declval<size_type>(),
                      std::declval<const_void_pointer>()),
                  std::true_type{});

  template <typename> static constexpr std::false_type test(...);

  static constexpr bool has_allocate_with_hint =
      decltype(test<Alloc>(0))::value;

public:
  static constexpr pointer allocate(Alloc &a, size_type n,
                                    const_void_pointer hint) {
    if constexpr (has_allocate_with_hint) {
      return a.allocate(n, hint);
    } else {
      (void)hint;
      return a.allocate(n);
    }
  }

  static constexpr void deallocate(Alloc &a, pointer p, size_type n) {
    a.deallocate(p, n);
  }

  static constexpr size_type max_size(const Alloc &a) noexcept {
    if constexpr (requires { a.max_size(); }) {
      return a.max_size();
    } else {
      return std::numeric_limits<size_type>::max() / sizeof(value_type);
    }
  }

private:
  template <typename A, typename T, typename... Args>
  static constexpr auto has_construct(int)
      -> decltype(std::declval<A>().construct(std::declval<T *>(),
                                              std::declval<Args>()...),
                  std::true_type{});
  template <typename, typename, typename...>
  static constexpr std::false_type has_construct(...);

  template <typename A, typename T>
  static constexpr auto has_destroy(int)
      -> decltype(std::declval<A>().destroy(std::declval<T *>()),
                  std::true_type{});
  template <typename, typename>
  static constexpr std::false_type has_destroy(...);

public:
  template <typename T, typename... Args>
  static constexpr void construct(Alloc &a, T *p, Args &&...args) {
    if constexpr (decltype(has_construct<Alloc, T, Args...>(0))::value) {
      a.construct(p, std::forward<Args>(args)...);
    } else {
      std::construct_at(p, std::forward<Args>(args)...);
    }
  }
```

```

  template <typename T> static constexpr void destroy(Alloc &a, T *p) {
    if constexpr (decltype(has_destroy<Alloc, T>(0))::value) {
      a.destroy(p);
    } else {
      std::destroy_at(p);
    }
  }

  static constexpr Alloc select_on_container_copy_construction(const Alloc &a) {
    if constexpr (requires { a.select_on_container_copy_construction(); }) {
      return a.select_on_container_copy_construction();
    } else {
      return a;
    }
  }
};

template <class Alloc>
static typename allocator_traits<Alloc>::pointer
allocate(Alloc &a, typename allocator_traits<Alloc>::size_type n) {
  return allocator_traits<Alloc>::allocate(a, n);
}

template <class Alloc>
static typename allocator_traits<Alloc>::pointer
allocate(Alloc &a, typename allocator_traits<Alloc>::size_type n,
         typename allocator_traits<Alloc>::const_void_pointer hint) {
  return allocator_traits<Alloc>::allocate(a, n, hint);
}

template <class Alloc>
static constexpr void
deallocate(Alloc &a, typename allocator_traits<Alloc>::pointer p,
           typename allocator_traits<Alloc>::size_type n) {
  allocator_traits<Alloc>::deallocate(a, p, n);
}

struct allocator_arg_t {
  explicit allocator_arg_t() = default;
};

inline constexpr allocator_arg_t allocator_arg{};

template <class T, class Alloc, class = void>
struct uses_allocator : std::false_type {};

template <class T, class Alloc>
struct uses_allocator<T, Alloc, std::void_t<typename T::allocator_type>>
    : std::bool_constant<
          std::is_convertible_v<Alloc, typename T::allocator_type>> {};

template <class T, class Alloc>
constexpr bool uses_allocator_v = uses_allocator<T, Alloc>::value;
```

Line 324–448：

```
template <typename> struct is_pair : std::false_type {};

template <typename T1, typename T2>
struct is_pair<std::pair<T1, T2>> : std::true_type {};

template <typename T> constexpr bool is_pair_v = is_pair<T>::value;

template <class T, class Alloc, class... Args>
```

```
  requires(!is_pair_v<std::remove_cvref_t<T>>) &&
          ((!uses_allocator_v<T, Alloc> &&
            std::is_constructible_v<T, Args...>) ||
           std::is_constructible_v<T, allocator_arg_t, const Alloc &,
                                   Args...> ||
           std::is_constructible_v<T, Args..., const Alloc &>)
constexpr auto uses_allocator_construction_args(const Alloc &alloc,
                                                Args &&...args) noexcept {
  if constexpr (!uses_allocator_v<T, Alloc> &&
                std::is_constructible_v<T, Args...>) {
    return forward_as_tuple(std::forward<Args>(args)...);
  } else if constexpr (std::is_constructible_v<T, allocator_arg_t,
                                               const Alloc &, Args...>) {
    return tuple<allocator_arg_t, const Alloc &, Args &&...>(
        std::allocator_arg, alloc, std::forward<Args>(args)...);
  } else {
    return forward_as_tuple(std::forward<Args>(args)..., alloc);
  }
}

template <class T, class Alloc, class Tuple1, class Tuple2>
  requires is_pair_v<std::remove_cvref_t<T>>
constexpr auto
uses_allocator_construction_args(const Alloc &alloc, std::piecewise_construct_t,
                                 Tuple1 &&x, Tuple2 &&y) noexcept {
  return make_tuple(
      std::piecewise_construct,
      std::apply(
          [&alloc](auto &&...args1) {
            return uses_allocator_construction_args<typename T::first_type>(
                alloc, std::forward<decltype(args1)>(args1)...);
          },
          std::forward<Tuple1>(x)),
      std::apply(
          [&alloc](auto &&...args2) {
            return uses_allocator_construction_args<typename T::second_type>(
                alloc, std::forward<decltype(args2)>(args2)...);
          },
          std::forward<Tuple2>(y)));
}

template <class T, class Alloc>
  requires is_pair_v<std::remove_cvref_t<T>>
constexpr auto uses_allocator_construction_args(const Alloc &alloc) noexcept {
  return uses_allocator_construction_args<T>(alloc, std::piecewise_construct,
                                             tuple<>{}, std::tuple<>{});
}

template <class T, class Alloc, class U, class V>
  requires is_pair_v<std::remove_cvref_t<T>>
constexpr auto uses_allocator_construction_args(const Alloc &alloc, U &&u,
                                                V &&v) noexcept {
  return uses_allocator_construction_args<T>(
      alloc, std::piecewise_construct, forward_as_tuple(std::forward<U>(u)),
      forward_as_tuple(std::forward<V>(v)));
}

template <class T, class Alloc, class U, class V>
  requires is_pair_v<std::remove_cvref_t<T>>
constexpr auto
uses_allocator_construction_args(const Alloc &alloc,
                                 const std::pair<U, V> &pr) noexcept {
  return uses_allocator_construction_args<T>(alloc, std::piecewise_construct,
                                             forward_as_tuple(pr.first),
                                             forward_as_tuple(pr.second));
```

```
}

template <class T, class Alloc, class U, class V>
  requires is_pair_v<std::remove_cvref_t<T>>
constexpr auto uses_allocator_construction_args(const Alloc &alloc,
                                                std::pair<U, V> &&pr) noexcept {
  return uses_allocator_construction_args<T>(
      alloc, std::piecewise_construct,
      forward_as_tuple(std::forward<U>(pr.first)),
      forward_as_tuple(std::forward<V>(pr.second)));
}

template <class T, class Alloc, class NonPair>
  requires is_pair_v<std::remove_cvref_t<T>> &&
           (!std::is_constructible_v<std::remove_cvref_t<T>,
                                     std::remove_cvref_t<NonPair>>)
constexpr auto uses_allocator_construction_args(const Alloc &alloc,
                                                NonPair &&np) noexcept {
  struct pair_constructor {
    const Alloc &alloc_;
    NonPair &u_;
    constexpr auto reconstruct(const std::remove_cv_t<T> &p) const {
      return std::make_obj_using_allocator<std::remove_cv_t<T>>(alloc_, p);
    }
    constexpr auto reconstruct(std::remove_cv_t<T> &&p) const {
      return std::make_obj_using_allocator<std::remove_cv_t<T>>(alloc_,
                                                                std::move(p));
    }
    constexpr operator std::remove_cv_t<T>() const {
      return reconstruct(std::remove_cv_t<T>(std::forward<NonPair>(u_).first,
                                             std::forward<NonPair>(u_).second));
    }
  };
  pair_constructor construction{alloc, np};
  return make_tuple(construction);
}

template <class T, class Alloc, class... Args>
constexpr T make_obj_using_allocator(const Alloc &alloc, Args &&...args) {
  return make_from_tuple<T>(
      uses_allocator_construction_args<T>(alloc, std::forward<Args>(args)...));
}

template <class T, class Alloc, class... Args>
constexpr T *uninitialized_construct_using_allocator(T *p, const Alloc &alloc,
                                                     Args &&...args) {
  return std::apply(
      [&]<class... Xs>(Xs &&...xs) {
        return std::construct_at(p, std::forward<Xs>(xs)...);
      },
      uses_allocator_construction_args<T>(alloc, std::forward<Args>(args)...));
}
```

(六) include/array.hpp

Line 20：

```
1 template <class T, std::size_t N> struct array;
```

Line 113：

```
1 template <class T, std::size_t N>
2 constexpr bool operator==(const mystd::array<T, N> &lhs,
```

```
                                const mystd::array<T, N> &rhs);
```

Line 119–121：

```
template <class T, std::size_t N>
constexpr std::compare_three_way_result_t<T>
operator<=>(const mystd::array<T, N> &lhs, const mystd::array<T, N> &rhs);
```

Line 126–127：

```
template <std::size_t I, class T, std::size_t N>
constexpr T &get(mystd::array<T, N> &arr) noexcept;
```

Line 131–132：

```
template <std::size_t I, class T, std::size_t N>
constexpr T &&get(mystd::array<T, N> &&arr) noexcept;
```

Line 136–137：

```
template <std::size_t I, class T, std::size_t N>
constexpr const T &get(const mystd::array<T, N> &arr) noexcept;
```

Line 141–142：

```
template <std::size_t I, class T, std::size_t N>
constexpr const T &&get(const mystd::array<T, N> &&arr) noexcept;
```

Line 146–147：

```
template <class T, size_t N>
constexpr mystd::array<std::remove_cv_t<T>, N> to_array(T (&arr)[N]);
```

Line 153–154：

```
template <class T, std::size_t N>
constexpr mystd::array<std::remove_cv_t<T>, N> to_array(T (&&arr)[N]);
```

Line 162–168：

```
template <class T, std::size_t N>
struct tuple_size<mystd::array<T, N>> : integral_constant<std::size_t, N> {};

template <std::size_t I, class T, std::size_t N>
struct tuple_element<I, mystd::array<T, N>> {
  using type = T;
};
```

(七) include/circulate.hpp

Line 15：

```
template <class circulate> class _circulate_iterator;
```

Line 92：

```
template <class circulate> class _circulate_const_iterator;
```

Line 191：

```
template <class T, class Container = mystd::vector<T>> class circulate;
```

Line 342–345：

```
template <class T, class Container>
void swap(
    mystd::circulate<T, Container> &lhs,
    mystd::circulate<T, Container> &rhs) noexcept(noexcept(lhs.swap(rhs)));
```

(八) include/ignore.hpp

Line 6–7：

```
  template <class T>
  constexpr const ignore_t &operator=(const T &) const noexcept;
```

Line 11：

```
  template <class T> constexpr const ignore_t &operator=(T &&) const noexcept;
```

(九) include/priority_queue.hpp

Line 18–22：

```
template <class T, class Container = mystd::vector<T>,
          class Compare = std::less<typename Container::value_type>>
  requires std::random_access_iterator<typename Container::iterator> &&
           std::predicate<Compare, const T &, const T &>
class priority_queue
```

Line 56–61：

```
  template <std::input_iterator InputIt>
    requires requires(InputIt first, InputIt last) {
      { Container(first, last) };
    }
  priority_queue(InputIt first, InputIt last,
                 const Compare &compare = Compare());
```

Line 66–71：

```
  template <std::input_iterator InputIt>
    requires requires(InputIt first, InputIt last) {
      { std::declval<Container &>().insert(first, last) };
    }
  priority_queue(InputIt first, InputIt last, const Compare &compare,
                 const Container &cont);
```

Line 77–82：

```
  template <std::input_iterator InputIt>
    requires requires(InputIt first, InputIt last) {
      { std::declval<Container &>().insert(first, last) };
    }
  priority_queue(InputIt first, InputIt last, const Compare &compare,
                 Container &&cont);
```

Line 88–89：

```
template <class Alloc>
explicit priority_queue(const Alloc &alloc);
```

Line 91–92：

```
template <class Alloc>
priority_queue(const Compare &compare, const Alloc &alloc);
```

Line 95–97：

```
template <class Alloc>
priority_queue(const Compare &compare, const Container &cont,
               const Alloc &alloc);
```

Line 102–103：

```
template <class Alloc>
priority_queue(const Compare &compare, Container &&cont, const Alloc &alloc);
```

Line 108–109：

```
template <class Alloc>
priority_queue(const priority_queue &other, const Alloc &alloc);
```

Line 112–113：

```
template <class Alloc>
priority_queue(priority_queue &&other, const Alloc &alloc);
```

Line 116–120：

```
template <std::input_iterator InputIt, class Alloc>
  requires requires(InputIt first, InputIt last) {
    { std::declval<Container &>().insert(first, last) };
  }
priority_queue(InputIt first, InputIt last, const Alloc &alloc);
```

Line 126–131：

```
template <std::input_iterator InputIt, class Alloc>
  requires requires(InputIt first, InputIt last) {
    { std::declval<Container &>().insert(first, last) };
  }
priority_queue(InputIt first, InputIt last, const Compare &compare,
               const Alloc &alloc);
```

Line 137–142：

```
template <std::input_iterator InputIt, class Alloc>
  requires requires(InputIt first, InputIt last) {
    { std::declval<Container &>().insert(first, last) };
  }
priority_queue(InputIt first, InputIt last, const Compare &compare,
               const Container &cont, const Alloc &alloc);
```

Line 148–153：

```
template <std::input_iterator InputIt, class Alloc>
  requires requires(InputIt first, InputIt last) {
    { std::declval<Container &>().insert(first, last) };
```

```
    }
  priority_queue(InputIt first, InputIt last, const Compare &compare,
                 Container &&cont, const Alloc &alloc);
```

Line 193：

```
  template <class... Args> reference emplace(Args &&...args);
```

Line 244–247：

```
template <class T, class Container, class Compare, class Alloc>
  requires std::predicate<Compare, const T &, const T &>
struct uses_allocator<mystd::priority_queue<T, Compare, Container>, Alloc>
    : std::uses_allocator<Container, Alloc> {};
```

Line 249–253：

```
template <class T, class Container, class Compare>
  requires std::predicate<Compare, const T &, const T &>
constexpr void swap(mystd::priority_queue<T, Container, Compare> &lhs,
                    mystd::priority_queue<T, Container, Compare>
                        &rhs) noexcept(noexcept(lhs.swap(rhs)));
```

(十) include/qsort.hpp

Line 9–11：

```
template<std::random_access_iterator RandomIt>
requires std::sortable<RandomIt>
constexpr void qsort(RandomIt first, RandomIt last);
```

Line 25–27：

```
template<std::random_access_iterator RandomIt, class Compare>
requires std::sortable<RandomIt, Compare>
constexpr void qsort(RandomIt first, RandomIt last, Compare comp);
```

(十一) include/range-access.hpp

Line 10–142：

```
template <class C> constexpr auto begin(const C &c) -> decltype(c.begin()) {
  return c.begin();
}

template <class C> constexpr auto begin(C &c) -> decltype(c.begin()) {
  return c.begin();
}

template <class T, std::size_t N> constexpr T *begin(T (&array)[N]) noexcept {
  return array;
}

template <class C>
constexpr auto cbegin(const C &c) noexcept(noexcept(mystd::begin(c)))
    -> decltype(mystd::begin(c)) {
  return c.begin();
}

template <class C> constexpr auto end(const C &c) -> decltype(c.end()) {
```

```
  return c.end();
}

template <class C> constexpr auto end(C &c) -> decltype(c.end()) {
  return c.end();
}

template <class T, std::size_t N> constexpr T *end(T (&array)[N]) noexcept {
  return array + N;
}

template <class C>
constexpr auto cend(const C &c) noexcept(noexcept(mystd::end(c)))
    -> decltype(mystd::end(c)) {
  return c.end();
}

template <class C> constexpr auto rbegin(C &c) -> decltype(c.rbegin()) {
  return c.rbegin();
}

template <class C> constexpr auto rbegin(const C &c) -> decltype(c.rbegin()) {
  return c.rbegin();
}

template <class T, std::size_t N>
constexpr std::reverse_iterator<T *> rbegin(T (&array)[N]) {
  return std::reverse_iterator<T *>(array + N);
}

template <class T>
constexpr std::reverse_iterator<const T *> rbegin(std::initializer_list<T> il) {
  return std::reverse_iterator<const T *>(il.end());
}

template <class C>
constexpr auto crbegin(const C &c) -> decltype(mystd::rbegin(c)) {
  return c.rbegin();
}

template <class C> constexpr auto rend(C &c) -> decltype(c.rend()) {
  return c.rend();
}

template <class C> constexpr auto rend(const C &c) -> decltype(c.rend()) {
  return c.rend();
}

template <class T, std::size_t N>
constexpr std::reverse_iterator<T *> rend(T (&array)[N]) {
  return std::reverse_iterator<T *>(array);
}

template <class T>
constexpr std::reverse_iterator<const T *> rend(std::initializer_list<T> il) {
  return std::reverse_iterator<const T *>(il.begin());
}

template <class C>
constexpr auto crend(const C &c) -> decltype(mystd::rend(c)) {
  return c.rend();
}

template <class C> constexpr auto size(const C &c) -> decltype(c.size()) {
  return c.size();
```

```
85 }
86
87 template <class C>
88 constexpr auto ssize(const C &c)
89     -> std::common_type_t<std::ptrdiff_t,
90                           std::make_signed_t<decltype(c.size())>> {
91   return static_cast<std::common_type_t<
92       std::ptrdiff_t, std::make_signed_t<decltype(c.size())>>>(c.size());
93 }
94
95 template <class T, std::size_t N>
96 constexpr std::size_t size(const T (&array)[N]) noexcept {
97   return N;
98 }
99
100 template <class T, std::ptrdiff_t N>
101 constexpr std::ptrdiff_t ssize(const T (&array)[N]) noexcept {
102   return N;
103 }
104
105 template <class C> constexpr auto empty(const C &c) -> decltype(c.empty()) {
106   return c.empty();
107 }
108
109 template <class T, std::size_t N>
110 constexpr bool empty(const T (&array)[N]) noexcept {
111   return false;
112 }
113
114 template <class E> constexpr bool empty(std::initializer_list<E> il) noexcept {
115   return il.size() == 0;
116 }
117
118 template <class C> constexpr auto data(C &c) -> decltype(c.data()) {
119   return c.data();
120 }
121
122 template <class C> constexpr auto data(const C &c) -> decltype(c.data()) {
123   return c.data();
124 }
125
126 template <class T, std::size_t N> constexpr T *data(T (&array)[N]) noexcept {
127   return array;
128 }
129
130 template <class E>
131 constexpr const E *data(std::initializer_list<E> il) noexcept {
132   return il.begin();
133 }
```

(十二) include/swap.hpp

Line 9–13：

```
1 template <class T>
2   requires(std::is_move_constructible_v<T> && std::is_move_assignable_v<T>)
3 constexpr void swap(T &a,
4                     T &b) noexcept(std::is_nothrow_move_constructible_v<T> &&
5                                    std::is_nothrow_move_assignable_v<T>);
```

Line 19–22：

```
1 template <class T, std::size_t N>
```

```
  requires std::is_swappable_v<T>
constexpr void swap(T (&a)[N],
                    T (&b)[N]) noexcept(noexcept(mystd::swap(a[0], b[0])));
```

(十三) include/tuple.hpp

Line 12–751：

```
namespace mystd {

template <class First, class... Rest> struct _first_type {
  using type = First;
};

template <class... Ts> using _first_type_t = typename _first_type<Ts...>::type;

template <typename T> struct _is_tuple : std::false_type {};

template <typename T> inline constexpr bool _is_tuple_v = _is_tuple<T>::value;

template <std::size_t I, class... Types> struct _tuple_impl;

template <std::size_t I> struct _tuple_impl<I> {
  template <std::size_t J> using element_type = void;

  constexpr _tuple_impl() {}
  constexpr _tuple_impl(const _tuple_impl &) noexcept {}
  constexpr _tuple_impl(_tuple_impl &&) noexcept {}

  template <class Alloc>
  constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &) noexcept {}

  template <class Alloc>
  constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &,
                        const _tuple_impl &) noexcept {}

  template <class Alloc>
  constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &,
                        _tuple_impl &&) noexcept {}

  constexpr _tuple_impl &operator=(const _tuple_impl &) noexcept {
    return *this;
  }

  constexpr _tuple_impl &operator=(_tuple_impl &&) noexcept { return *this; }

  void swap(_tuple_impl &other) {}
};

template <std::size_t I, class Head, class... Tail>
struct _tuple_impl<I, Head, Tail...> : public _tuple_impl<I + 1, Tail...> {
  using base = _tuple_impl<I + 1, Tail...>;
  Head head;

  template <std::size_t J>
  using element_type =
      std::conditional_t<J == I, Head, typename base::template element_type<J>>;

  constexpr _tuple_impl()
    requires(std::is_default_constructible_v<Head> &&
             (std::is_default_constructible_v<Tail> && ...))
      : base(), head() {}

  explicit((!std::is_convertible_v<const Head &, Head> ||
```

```
          (!std::is_convertible_v<const Tail &, Tail> ||
           ...))) constexpr _tuple_impl(const Head &arg, const Tail &...args)
  requires std::is_constructible_v<Head, const Head &> &&
               (std::is_constructible_v<Tail, const Tail &> && ...)
    : base(args...), head(arg) {}

template <class UHead, class... UTail>
explicit((!std::is_convertible_v<UHead, Head> ||
          (!std::is_convertible_v<UTail, Tail> ||
           ...))) constexpr _tuple_impl(UHead &&arg, UTail &&...args)
  requires(sizeof...(Tail) == sizeof...(UTail)) &&
              std::is_constructible_v<Head, UHead> &&
              (std::is_constructible_v<Tail, UTail> && ...)
    : base(std::forward<UTail>(args)...), head(std::forward<UHead>(arg)) {}

template <typename Alloc>
constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc)
  requires std::is_default_constructible_v<Head> &&
               (std::is_default_constructible_v<Tail> && ...)
    : base(mystd::allocator_arg, alloc), head() {}

template <typename Alloc>
constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc,
                      const Head &arg, const Tail &...args)
  requires std::is_constructible_v<Head, const Head &> &&
               (std::is_constructible_v<Tail, const Tail &> && ...)
    : base(mystd::allocator_arg, alloc, args...), head(arg) {}

template <typename Alloc, class UHead, class... UTail>
constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc, UHead &&arg,
                      UTail &&...args)
  requires(sizeof...(Tail) == sizeof...(UTail)) &&
              std::is_constructible_v<Head, UHead> &&
              (std::is_constructible_v<Tail, UTail> && ...)
    : base(mystd::allocator_arg, alloc, std::forward<UTail>(args)...),
      head(std::forward<UHead>(arg)) {}

constexpr _tuple_impl(const _tuple_impl &other)
  requires(std::is_copy_constructible_v<Head> &&
           std::is_copy_constructible_v<base>)
    : base(static_cast<const base &>(other)), head(other.head) {}

constexpr _tuple_impl(_tuple_impl &&other)
  requires(std::is_move_constructible_v<Head> &&
           std::is_move_constructible_v<base>)
    : base(std::move(static_cast<base &>(other))),
      head(std::move(other.head)) {}

template <class Alloc>
constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc,
                      const _tuple_impl &other)
  requires(std::is_copy_constructible_v<Head> &&
           std::is_copy_constructible_v<base>)
    : base(mystd::allocator_arg, alloc, static_cast<const base &>(other)),
      head(other.head) {}

template <class Alloc>
constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc,
                      _tuple_impl &&other)
  requires(std::is_move_constructible_v<Head> &&
           std::is_move_constructible_v<base>)
    : base(mystd::allocator_arg, alloc,
           std::move(static_cast<base &>(other))),
      head(std::move(other.head)) {}

```

```
  template <class Alloc, class... UTypes>
  constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc,
                        const _tuple_impl<I, UTypes...> &other)
    requires(sizeof...(Tail) + 1 == sizeof...(UTypes)) &&
                std::is_constructible_v<Head,
                                        const typename std::remove_cvref_t<
                                            _first_type_t<UTypes...>> &> &&
                std::is_constructible_v<
                    base, const typename _tuple_impl<I, UTypes...>::base &>
      : base(mystd::allocator_arg, alloc,
             static_cast<const typename _tuple_impl<I, UTypes...>::base &>(
                 other)),
        head(other.head) {}

  template <class Alloc, class... UTypes>
  constexpr _tuple_impl(mystd::allocator_arg_t, const Alloc &alloc,
                        _tuple_impl<I, UTypes...> &&other)
    requires(sizeof...(Tail) + 1 == sizeof...(UTypes)) &&
                std::is_constructible_v<
                    Head, std::remove_cvref_t<_first_type_t<UTypes...>>> &&
                std::is_constructible_v<
                    base, typename _tuple_impl<I, UTypes...>::base>
      : base(mystd::allocator_arg, alloc,
             std::move(static_cast<typename _tuple_impl<I, UTypes...>::base &>(
                 other))),
        head(std::move(other.head)) {}

  constexpr _tuple_impl &operator=(const _tuple_impl &other)
    requires std::is_copy_assignable_v<Head> && std::is_copy_assignable_v<base>
  {
    head = other.head;
    base::operator=(other);
    return *this;
  }

  constexpr _tuple_impl &operator=(_tuple_impl &&other) noexcept(
      std::is_nothrow_move_assignable_v<Head> &&
      std::is_nothrow_move_assignable_v<base>)
    requires(std::is_move_assignable_v<Head> && std::is_move_assignable_v<base>)
  {
    head = std::move(other.head);
    base::operator=(std::move(other));
    return *this;
  }

  template <class UHead, class... UTail>
  constexpr _tuple_impl &operator=(const _tuple_impl<I, UHead, UTail...> &other)
    requires(sizeof...(Tail) == sizeof...(UTail)) &&
            std::is_assignable_v<Head &, const UHead &> &&
            std::is_assignable_v<base &, const _tuple_impl<I + 1, UTail...> &>
  {
    head = other.head;
    base::operator=(static_cast<const _tuple_impl<I + 1, UTail...> &>(other));
    return *this;
  }

  template <class UHead, class... UTail>
  constexpr _tuple_impl &
  operator=(_tuple_impl<I, UHead, UTail...> &&other) noexcept(
      std::is_nothrow_assignable_v<Head &, UHead &&> &&
      std::is_nothrow_assignable_v<base &, _tuple_impl<I + 1, UTail...> &&>)
    requires(sizeof...(Tail) == sizeof...(UTail)) &&
            std::is_assignable_v<Head &, UHead &&> &&
            std::is_assignable_v<base &, _tuple_impl<I + 1, UTail...> &&>
  {
```

```
    head = std::move(other.head);
    base::operator=(
        static_cast<_tuple_impl<I + 1, UTail...> &&>(std::move(other)));
    return *this;
  }

  constexpr void swap(_tuple_impl &other) noexcept(
      std::is_nothrow_swappable_v<Head> &&
      noexcept(std::declval<base &>().swap(std::declval<base &>()))) {
    mystd::swap(head, other.head);
    static_cast<base &>(*this).swap(static_cast<base &>(other));
  }

  template <std::size_t J>
  constexpr decltype(auto) get() &
    requires(J >= I && J <= I + sizeof...(Tail))
  {
    if constexpr (I == J) {
      return (head);
    } else {
      return static_cast<base &>(*this).template get<J>();
    }
  }

  template <std::size_t J>
  constexpr decltype(auto) get() const &
    requires(J >= I && J <= I + sizeof...(Tail))
  {
    if constexpr (I == J) {
      return (head);
    } else {
      return static_cast<const base &>(*this).template get<J>();
    }
  }

  template <std::size_t J>
  constexpr decltype(auto) get() &&
    requires(J >= I && J <= I + sizeof...(Tail))
  {
    if constexpr (I == J) {
      return std::move(head);
    } else {
      return static_cast<base &&>(*this).template get<J>();
    }
  }

  template <std::size_t J>
  constexpr decltype(auto) get() const &&
    requires(J >= I && J <= I + sizeof...(Tail))
  {
    if constexpr (I == J) {
      return std::move(head);
    } else {
      return static_cast<const base &&>(*this).template get<J>();
    }
  }
};

template <class... Types> class tuple {
public:
  _tuple_impl<0, Types...> _impl;

  constexpr tuple()
    requires(std::is_default_constructible_v<Types> && ...)
      : _impl() {}
```

```
  explicit((!std::is_convertible_v<const Types &, Types> ||
            ...)) constexpr tuple(const Types &...args)
    requires(sizeof...(Types) >= 1) &&
            (std::is_copy_constructible_v<Types> && ...)
      : _impl(args...) {}

  template <class... UTypes>
  explicit((!std::is_convertible_v<UTypes, Types> ||
            ...)) constexpr tuple(UTypes &&...args)
    requires(sizeof...(Types) >= 1) &&
            (sizeof...(Types) == sizeof...(UTypes)) &&
            (std::is_constructible_v<Types, UTypes> && ...) &&
            (sizeof...(Types) > 1 ||
             !mystd::_is_tuple_v<
                 std::remove_cvref_t<_first_type_t<UTypes...>>>) &&
            (sizeof...(Types) > 3 ||
             !std::is_same_v<std::remove_cvref_t<_first_type_t<UTypes...>>,
                             mystd::allocator_arg_t> ||
             std::is_same_v<_first_type_t<Types...>, mystd::allocator_arg_t>)
      : _impl(std::forward<UTypes>(args)...) {}

  template <class... UTypes>
  explicit((!std::is_convertible_v<Types, const UTypes &> ||
            ...)) constexpr tuple(const tuple<UTypes...> &other)
    requires(sizeof...(Types) == sizeof...(UTypes)) &&
            (std::is_constructible_v<Types, const UTypes &> && ...) &&
            (!(sizeof...(Types) == 1 &&
               ((std::is_convertible_v<UTypes, Types> || ...) ||
                (std::is_constructible_v<Types, UTypes> || ...) ||
                (std::is_same_v<Types, UTypes> || ...))))
      : _impl(other._impl) {}

  template <class... UTypes>
  explicit((!std::is_convertible_v<Types, UTypes &&> ||
            ...)) constexpr tuple(tuple<UTypes...> &&other)
    requires(sizeof...(Types) == sizeof...(UTypes)) &&
            (std::is_constructible_v<Types, UTypes &&> && ...) &&
            (!(sizeof...(Types) == 1 &&
               ((std::is_convertible_v<UTypes, Types> || ...) ||
                (std::is_constructible_v<Types, UTypes> || ...) ||
                (std::is_same_v<Types, UTypes> || ...))))
      : _impl(std::move(other._impl)) {}

  template <class U1, class U2>
  explicit(
      (!std::is_convertible_v<
          U1, typename _tuple_impl<0, Types...>::template element_type<0>> ||
       !std::is_convertible_v<
          U2, typename _tuple_impl<0, Types...>::template element_type<
                  1>>)) constexpr tuple(const std::pair<U1, U2> &p)
    requires(sizeof...(Types) == 2) &&
            std::is_constructible_v<
                typename _tuple_impl<0, Types...>::template element_type<0>,
                U1> &&
            std::is_constructible_v<
                typename _tuple_impl<0, Types...>::template element_type<1>, U2>
      : _impl(p.first, p.second) {}

  template <class U1, class U2>
  explicit((
      !std::is_convertible_v<
          U1 &&, typename _tuple_impl<0, Types...>::template element_type<0>> ||
      !std::is_convertible_v<
          U2 &&, typename _tuple_impl<0, Types...>::template element_type<
```

```
                    1>>)) constexpr tuple(std::pair<U1, U2> &&p)
  requires(sizeof...(Types) == 2) &&
          std::is_constructible_v<
              typename _tuple_impl<0, Types...>::template element_type<0>,
              U1 &&> &&
          std::is_constructible_v<
              typename _tuple_impl<0, Types...>::template element_type<1>,
              U2 &&>
    : _impl(std::move(p.first), std::move(p.second)) {}

tuple(const tuple &other) : _impl(other._impl) {}

tuple(tuple &&other) : _impl(std::move(other._impl)) {}

template <class Alloc>
constexpr tuple(mystd::allocator_arg_t, const Alloc &a)
  requires(std::is_default_constructible_v<Types> && ...)
    : _impl(mystd::allocator_arg, a) {}

template <class Alloc>
explicit((!std::is_convertible_v<const Types &, Types> ||
          ...)) constexpr tuple(mystd::allocator_arg_t, const Alloc &a,
                                const Types &...args)
  requires(sizeof...(Types) >= 1) &&
          (std::is_copy_constructible_v<Types> && ...)
    : _impl(mystd::allocator_arg, a, args...) {}

template <class Alloc, class... UTypes>
explicit((!std::is_convertible_v<UTypes, Types> ||
          ...)) constexpr tuple(mystd::allocator_arg_t, const Alloc &a,
                                UTypes &&...args)
  requires(sizeof...(Types) >= 1) &&
          (sizeof...(Types) == sizeof...(UTypes)) &&
          (std::is_constructible_v<Types, UTypes> && ...) &&
          (sizeof...(Types) > 1 ||
           !mystd::_is_tuple_v<std::remove_cvref_t<_first_type_t<UTypes...>>>)
    : _impl(mystd::allocator_arg, a, std::forward<UTypes>(args)...) {}

template <class Alloc, class... UTypes>
explicit((!std::is_convertible_v<Types, const UTypes &> ||
          ...)) constexpr tuple(mystd::allocator_arg_t, const Alloc &a,
                                const tuple<UTypes...> &other)
  requires(sizeof...(Types) == sizeof...(UTypes)) &&
          (std::is_constructible_v<Types, const UTypes &> && ...) &&
          (!(sizeof...(Types) == 1 &&
             ((std::is_convertible_v<UTypes, Types> || ...) ||
              (std::is_constructible_v<Types, UTypes> || ...) ||
              (std::is_same_v<Types, UTypes> || ...))))
    : _impl(mystd::allocator_arg, a, other._impl) {}

template <class Alloc, class... UTypes>
explicit((!std::is_convertible_v<Types, UTypes &&> ||
          ...)) constexpr tuple(mystd::allocator_arg_t, const Alloc &a,
                                tuple<UTypes...> &&other)
  requires(sizeof...(Types) == sizeof...(UTypes)) &&
          (std::is_constructible_v<Types, UTypes &&> && ...) &&
          (!(sizeof...(Types) == 1 &&
             ((std::is_convertible_v<UTypes, Types> || ...) ||
              (std::is_constructible_v<Types, UTypes> || ...) ||
              (std::is_same_v<Types, UTypes> || ...))))
    : _impl(mystd::allocator_arg, a, std::move(other._impl)) {}

template <class Alloc, class U1, class U2>
explicit(
    (!std::is_convertible_v<
```

```
382             U1, typename _tuple_impl<0, Types...>::template element_type<0>> ||
383         !std::is_convertible_v<
384             U2, typename _tuple_impl<0, Types...>::template element_type<
385                     1>>)) constexpr tuple(mystd::allocator_arg_t, const Alloc &a,
386                                           const std::pair<U1, U2> &p)
387     requires(sizeof...(Types) == 2) &&
388             std::is_constructible_v<
389                 typename _tuple_impl<0, Types...>::template element_type<0>,
390                 U1> &&
391             std::is_constructible_v<
392                 typename _tuple_impl<0, Types...>::template element_type<1>, U2>
393         : _impl(mystd::allocator_arg, a, p.first, p.second) {}
394
395     template <class Alloc, class U1, class U2>
396     explicit((
397         !std::is_convertible_v<
398             U1 &&, typename _tuple_impl<0, Types...>::template element_type<0>> ||
399         !std::is_convertible_v<
400             U2 &&, typename _tuple_impl<0, Types...>::template element_type<
401                     1>>)) constexpr tuple(mystd::allocator_arg_t,
402                                           const Alloc &a,
403                                           std::pair<U1, U2> &&p)
404     requires(sizeof...(Types) == 2) &&
405             std::is_constructible_v<
406                 typename _tuple_impl<0, Types...>::template element_type<0>,
407                 U1 &&> &&
408             std::is_constructible_v<
409                 typename _tuple_impl<0, Types...>::template element_type<1>,
410                 U2 &&>
411         : _impl(mystd::allocator_arg, a, std::move(p.first),
412                 std::move(p.second)) {}
413
414     template <class Alloc>
415     tuple(mystd::allocator_arg_t, const Alloc &a, const tuple &other)
416         : _impl(mystd::allocator_arg, a, other._impl) {}
417
418     template <class Alloc>
419     tuple(mystd::allocator_arg_t, const Alloc &a, tuple &&other)
420         : _impl(mystd::allocator_arg, a, std::move(other._impl)) {}
421
422     constexpr tuple &operator=(const tuple &other)
423       requires(std::is_copy_assignable_v<Types> && ...)
424     {
425       _impl = other._impl;
426       return *this;
427     }
428
429     constexpr tuple &operator=(tuple &&other) noexcept(
430         (std::is_nothrow_move_assignable_v<Types> && ...))
431       requires(std::is_move_assignable_v<Types> && ...)
432     {
433       _impl = std::move(other._impl);
434       return *this;
435     }
436
437     template <class... UTypes>
438     constexpr tuple &operator=(const tuple<UTypes...> &other)
439       requires(sizeof...(Types) == sizeof...(UTypes)) &&
440               (std::is_assignable_v<Types &, const UTypes &> && ...)
441     {
442       _impl = other._impl;
443       return *this;
444     }
445
446     template <class... UTypes>
```

```
447   constexpr tuple &operator=(tuple<UTypes...> &&other)
448     requires(sizeof...(Types) == sizeof...(UTypes)) &&
449             (std::is_assignable_v<Types &, UTypes &&> && ...)
450   {
451     _impl = std::move(other._impl);
452     return *this;
453   }
454
455   template <std::size_t J> constexpr decltype(auto) get() & noexcept {
456     return _impl.template get<J>();
457   }
458
459   template <std::size_t J> constexpr decltype(auto) get() && noexcept {
460     return std::move(_impl).template get<J>();
461   }
462
463   template <std::size_t J> constexpr decltype(auto) get() const & noexcept {
464     return _impl.template get<J>();
465   }
466
467   template <std::size_t J> constexpr decltype(auto) get() const && noexcept {
468     return std::move(_impl).template get<J>();
469   }
470
471   template <class E1, class E2>
472   constexpr tuple &operator=(const std::pair<E1, E2> &p)
473     requires(sizeof...(Types) == 2) &&
474             std::is_assignable_v<decltype(this -> get<0>()), const E1 &> &&
475             std::is_assignable_v<decltype(this->get<1>()), const E2 &>
476   {
477     this->get<0>() = p.first;
478     this->get<1>() = p.second;
479     return *this;
480   }
481
482   template <class E1, class E2>
483   constexpr tuple &operator=(std::pair<E1, E2> &&p) noexcept(
484       std::is_nothrow_assignable_v<decltype(this->get<0>()), E1 &&> &&
485       std::is_nothrow_assignable_v<decltype(this->get<1>()), E2 &&>)
486     requires(sizeof...(Types) == 2) &&
487             std::is_assignable_v<decltype(this -> get<0>()), E1 &&> &&
488             std::is_assignable_v<decltype(this->get<1>()), E2 &&>
489   {
490     this->get<0>() = std::move(p.first);
491     this->get<1>() = std::move(p.second);
492     return *this;
493   }
494
495   constexpr void swap(tuple &other) noexcept(
496       noexcept((std::is_nothrow_swappable_v<Types> && ...))) {
497     _impl.swap(other._impl);
498   }
499
500 private:
501   template <std::size_t... I>
502   static constexpr bool tuple_equal_impl(const tuple &a, const tuple &b,
503                                          std::index_sequence<I...>) {
504     return ((a.get<I>() == b.get<I>()) && ...);
505   }
506
507 public:
508   friend constexpr bool operator==(const tuple &a, const tuple &b) {
509     return tuple_equal_impl(a, b, std::make_index_sequence<sizeof...(Types)>{});
510   }
511
```

```
private:
  template <std::size_t... I>
  static constexpr auto tuple_three_way_impl(const tuple &a, const tuple &b,
                                             std::index_sequence<I...>)
      -> std::common_comparison_category_t<decltype(a.get<I>() <=>
                                                    b.get<I>())...> {
    using result_t = std::common_comparison_category_t<decltype(a.get<I>() <=>
                                                                b.get<I>())...>;
    result_t result = result_t::equal;
    ((result = (result != 0 ? result : (a.get<I>() <=> b.get<I>()))), ...);
    return result;
  }

public:
  friend constexpr auto operator<=>(const tuple &a, const tuple &b) {
    return tuple_three_way_impl(a, b,
                                std::make_index_sequence<sizeof...(Types)>{});
  }
};

template <> class tuple<> {
public:
  constexpr tuple() noexcept = default;

  constexpr tuple(const tuple &) noexcept = default;
  constexpr tuple(tuple &&) noexcept = default;

  constexpr tuple &operator=(const tuple &) noexcept = default;
  constexpr tuple &operator=(tuple &&) noexcept = default;

  constexpr void swap(tuple &) noexcept {}

  friend constexpr bool operator==(const tuple &, const tuple &) noexcept {
    return true;
  }

  friend constexpr auto operator<=>(const tuple &, const tuple &) noexcept {
    return std::strong_ordering::equal;
  }
};

template <class... UTypes> tuple(UTypes...) -> tuple<UTypes...>;

template <class T1, class T2> tuple(std::pair<T1, T2>) -> tuple<T1, T2>;

template <class Alloc, class... UTypes>
tuple(std::allocator_arg_t, Alloc, UTypes...) -> tuple<UTypes...>;

template <class Alloc, class T1, class T2>
tuple(std::allocator_arg_t, Alloc, std::pair<T1, T2>) -> tuple<T1, T2>;

template <class Alloc, class... UTypes>
tuple(std::allocator_arg_t, Alloc, tuple<UTypes...>) -> tuple<UTypes...>;

template <typename... Ts>
struct _is_tuple<mystd::tuple<Ts...>> : std::true_type {};

template <std::size_t I, class... Types>
constexpr decltype(auto) get(tuple<Types...> &t) noexcept {
  return t.template get<I>();
}

template <std::size_t I, class... Types>
constexpr decltype(auto) get(tuple<Types...> &&t) noexcept {
  return std::move(t).template get<I>();
```

```
}

template <std::size_t I, class... Types>
constexpr decltype(auto) get(const tuple<Types...> &t) noexcept {
  return t.template get<I>();
}

template <std::size_t I, class... Types>
constexpr decltype(auto) get(const tuple<Types...> &&t) noexcept {
  return std::move(t).template get<I>();
}

template <class T, class... Types> struct _count_of_type;

template <class T>
struct _count_of_type<T> : std::integral_constant<std::size_t, 0> {};

template <class T, class Head, class... Tail>
struct _count_of_type<T, Head, Tail...>
    : std::integral_constant<std::size_t,
                             (std::is_same_v<T, Head> ? 1 : 0) +
                                 _count_of_type<T, Tail...>::value> {};

template <class T, class... Types>
inline constexpr std::size_t _count_of_type_v =
    _count_of_type<T, Types...>::value;

template <class T, class... Types> struct _index_of_type;

template <class T, class... Tail>
struct _index_of_type<T, T, Tail...> : std::integral_constant<std::size_t, 0> {
};

template <class T, class Head, class... Tail>
struct _index_of_type<T, Head, Tail...>
    : std::integral_constant<std::size_t,
                             1 + _index_of_type<T, Tail...>::value> {};

template <class T> struct _index_of_type<T> {};

template <class T, class... Types>
inline constexpr std::size_t _index_of_type_v =
    _index_of_type<T, Types...>::value;

template <class T, class... Types>
constexpr decltype(auto) get(mystd::tuple<Types...> &t) noexcept
  requires(_count_of_type_v<T, Types...> == 1)
{
  return get<_index_of_type_v<T, Types...>>(t);
}

template <class T, class... Types>
constexpr decltype(auto) get(mystd::tuple<Types...> &&t) noexcept
  requires(_count_of_type_v<T, Types...> == 1)
{
  return get<_index_of_type_v<T, Types...>>(std::move(t));
}

template <class T, class... Types>
constexpr decltype(auto) get(const mystd::tuple<Types...> &t) noexcept
  requires(_count_of_type_v<T, Types...> == 1)
{
  return get<_index_of_type_v<T, Types...>>(t);
}

```

```
template <class T, class... Types>
constexpr decltype(auto) get(const mystd::tuple<Types...> &&t) noexcept
  requires(_count_of_type_v<T, Types...> == 1)
{
  return get<_index_of_type_v<T, Types...>>(std::move(t));
}

template <class... Types> constexpr auto make_tuple(Types &&...args) {
  return mystd::tuple<typename std::decay<Types>::type...>(
      std::forward<Types>(args)...);
}

template <class... Types>
constexpr mystd::tuple<Types &...> tie(Types &...args) noexcept {
  return mystd::tuple<Types &...>(args...);
}

template <class... Types>
constexpr tuple<Types &&...> forward_as_tuple(Types &&...args) noexcept {
  return tuple<Types &&...>(std::forward<Types>(args)...);
}

template <class... Tuples> struct _tuple_cat_type;

template <> struct _tuple_cat_type<> {
  using type = mystd::tuple<>;
};

template <class... Ts> struct _tuple_cat_type<mystd::tuple<Ts...>> {
  using type = mystd::tuple<Ts...>;
};

template <class... Ts1, class... Ts2, class... Rest>
struct _tuple_cat_type<mystd::tuple<Ts1...>, mystd::tuple<Ts2...>, Rest...> {
  using type =
      typename _tuple_cat_type<mystd::tuple<Ts1..., Ts2...>, Rest...>::type;
};

template <class... Tuples>
using _tuple_cat_type_t = typename _tuple_cat_type<Tuples...>::type;

template <class T1, class T2, std::size_t... I1, std::size_t... I2>
constexpr auto _tuple_cat_two_impl(T1 &&t1, T2 &&t2, std::index_sequence<I1...>,
                                   std::index_sequence<I2...>) {
  return mystd::tuple{mystd::get<I1>(std::forward<T1>(t1))...,
                      mystd::get<I2>(std::forward<T2>(t2))...};
}

template <class T1, class T2> constexpr auto _tuple_cat_two(T1 &&t1, T2 &&t2) {
  return _tuple_cat_two_impl(
      std::forward<T1>(t1), std::forward<T2>(t2),
      std::make_index_sequence<
          std::tuple_size<std::remove_cvref_t<T1>>::value>{},
      std::make_index_sequence<
          std::tuple_size<std::remove_cvref_t<T2>>::value>{});
}

template <class T> constexpr auto _tuple_cat_impl(T &&t) {
  return std::forward<T>(t);
}

template <class T1, class T2, class... Rest>
constexpr auto _tuple_cat_impl(T1 &&t1, T2 &&t2, Rest &&...rest) {
  auto first = _tuple_cat_two(std::forward<T1>(t1), std::forward<T2>(t2));
  return _tuple_cat_impl(std::move(first), std::forward<Rest>(rest)...);
```

```
}

template <class... Tuples> constexpr auto tuple_cat(Tuples &&...tpls) {
  return _tuple_cat_impl(std::forward<Tuples>(tpls)...);
}

template <class... Types>
constexpr void
swap(mystd::tuple<Types...> &lhs,
     mystd::tuple<Types...> &rhs) noexcept(noexcept(lhs.swap(rhs))) {
  lhs.swap(rhs);
}

template <class... Types, class Alloc>
struct uses_allocator<std::tuple<Types...>, Alloc> : std::true_type {};

} // namespace mystd

namespace std {

template <class... Types>
struct tuple_size<mystd::tuple<Types...>>
    : std::integral_constant<std::size_t, sizeof...(Types)> {};

template <class Head, class... Tail>
struct tuple_element<0, mystd::tuple<Head, Tail...>> {
  using type = Head;
};

template <std::size_t I, class Head, class... Tail>
struct tuple_element<I, mystd::tuple<Head, Tail...>>
    : tuple_element<I - 1, mystd::tuple<Tail...>> {};

} // namespace std
```

(十四) include/vector.hpp

已於 **2-B-1-2 指標成員處理** 的 **include/vector.hpp** 列出。

二十六、 2-C-4-2 polymorphism 成員

承上，本專題未使用 runtime polymorphism，而是廣泛使用 compile-time polymorphism。在 `mystd` 部分，compile-time polymorphism 的函式成員已於前一章節列出；於本節僅列出 compile-time polymorphism 的成員變數及 mystd 以外的 compile-time polymorphism 成員。

(一) include/allocator.hpp

已於前一章節的 **include/allocator.hpp** 列出。

(二) include/circulate.hpp

Line 24–25：

```
  circulate *circ;
  difference_type idx;
```

Line 101–102：

```
const circulate *circ;
difference_type idx;
```

Line 205：

```
Container c;
```

(三) include/priority_queue.hpp

Line 32–33：

```
Container c = Container();
Compare comp = Compare();
```

(四) include/tuple.hpp

Line 56：

```
Head head;
```

Line 258：

```
_tuple_impl<0, Types...> _impl;
```

(五) include/vector.hpp

Line 46–47：

```
[[no_unique_address]] Allocator alloc;
T *elems;
```

(六) Game.hpp

Line 44：

```
mystd::vector<KeyNoteData> &notes; // sorted by startFragment
```

Line 50：

```
mystd::vector<mystd::circulate<int8_t, mystd::vector<int8_t>>> highway;
```

Line 53：

```
mystd::vector<bool> lanePressed;
```

Line 56：

```
mystd::vector<uint32_t> holdPressedTime;
```

Line 64–65：

```
mystd::vector<Effect> laneEffects;
game_priority_queue<Effect> centerEffects = game_priority_queue<Effect>();
```

(七) Renderer.hpp

Line 55–59：

```
  std::unordered_map<mystd::tuple<int8_t, bool, uint32_t>, SDL_Texture *,
                     NotesCacheHash>
      notesTextureCache;
  std::unordered_map<std::string, SDL_Texture *> textTextureCache;
  std::unordered_map<std::string, SDL_Texture *> imageTextureCache;
```

(八) mods/GameOfLife.hpp

Line 14：

```
static mystd::array<uint16_t, 2> currentRules = {0b111111111, 0};
```

(九) Mods.hpp

Line 15–20：

```
inline std::map<std::string, mystd::tuple<ModFunc, ModFunc, SettingsFunc>> &
getModMap() {
  static std::map<std::string, mystd::tuple<ModFunc, ModFunc, SettingsFunc>>
      modMap;
  return modMap;
}
```

二十七、 2-C-4-3 polymorphism 相關的 class 或 function 間的對應關係

承上，本專題未使用 runtime polymorphism，而是廣泛使用 compile-time polymorphism。

具體個案的對應關係可參考前兩章所列之內容中 template arguments 等處得知，下文將針對其各類型說明設計理念與核心概念。

(一) Substitution Failure Is Not An Error (SFINAE)

SFINAE is a compile-time technique in template programming. If substituting template parameters results in an invalid type or expression, the compiler silently ignores that overload instead of producing an error. This mechanism enables selective template instantiation and overload resolution.

(二) Concepts

C++20 introduces **concepts**, which allow explicit constraints on template parameters in a readable and expressive way. Concepts replace some uses of SFINAE with clearer syntax. Key benefits include:

- Clear compile-time diagnostics.
- Explicit template constraints.
- Improved code readability and maintainability.

In my STL implementation, concepts are heavily used to determine the appropriate overload or `if constexpr` branch. This technique enables optimizations for trivially copyable types, contiguous iterators, allocator traits-based optimization, etc.

Many examples are demonstrated in **2-B-1-2 指標成員處理** 中的 **include/vector.hpp** and described in its **描述**.

Some other examples in `include/algorithm-copy.hpp` is described as follows. The file provides three basic data transfer algorithms: `copy`, `copy_if`, and `copy_backward`, mimicking the interface design of C++20 `<algorithm>`. The copy function family demonstrates the polymorphism programming techniques, particularly conceptual parameter constraints and conditional optimization strategies.

I utilize concepts for constraints, such as `std::input_iterator`, `std::weakly_incrementable`, `std::bidirectional_iterator`, and `std::indirectly_writable` in the definition, and `std::contiguous_iterator` and `std::is_trivially_copyable_v` for specialized acceleration.

In `copy` algorithms, I use `std::memcpy` when possible and element-wise assignment only when necessary, as demonstrated below (line 7–32)

```
1  template <std::input_iterator InputIt, std::weakly_incrementable OutputIt>
2    requires std::indirectly_writable<OutputIt, std::iter_value_t<InputIt>>
3  constexpr OutputIt copy(InputIt first, InputIt last, OutputIt d_first) {
4    if constexpr (std::contiguous_iterator<InputIt> &&
5                  std::contiguous_iterator<OutputIt> &&
6                  std::is_trivially_copyable_v<
7                      typename std::iterator_traits<InputIt>::value_type> &&
8                  std::is_same_v<
9                      typename std::iterator_traits<InputIt>::value_type,
10                     typename std::iterator_traits<OutputIt>::value_type>) {
11     if (first == last)
12       return d_first;
13     const auto *src = std::to_address(first);
14     const auto *src_end = std::to_address(last);
15     auto *dest = std::to_address(d_first);
16     const std::size_t count = (src_end - src) * sizeof(*src);
17     std::memcpy(dest, src, count);
18     return d_first + (last - first);
19   }
20   while (first != last) {
21     *d_first = *first;
22     ++first;
23     ++d_first;
24   }
25   return d_first;
26 }
```

Similarly, in `copy_backward` algorithm, `std::memmove` is used when possible, as demonstrated below (line 49–73)

```
1  template <std::bidirectional_iterator BidirIt1,
2            std::bidirectional_iterator BidirIt2>
3  constexpr BidirIt2 copy_backward(BidirIt1 first, BidirIt1 last,
4                                   BidirIt2 d_last) {
5    using ValueType1 = typename std::iterator_traits<BidirIt1>::value_type;
6    using ValueType2 = typename std::iterator_traits<BidirIt2>::value_type;
7    if constexpr (std::contiguous_iterator<BidirIt1> &&
```

```
                std::contiguous_iterator<BidirIt2> &&
                std::is_trivially_copyable_v<ValueType1> &&
                std::is_same_v<ValueType1, ValueType2>) {
    auto n = last - first;
    if (n > 0) {
      std::memmove(std::addressof(*d_last) - n, std::addressof(*first),
                   n * sizeof(ValueType1));
    }
    return d_last - n;
  } else {
    while (first != last) {
      --last;
      --d_last;
      *d_last = *last;
    }
    return d_last;
  }
}
```

(三) Class Template Argument Deduction (CTAD)

C++17 introduced **Class Template Argument Deduction (CTAD)**, allowing the compiler to deduce template parameters from constructor arguments. STL containers provide **deduction guides** to assist this process, which are also implemented in my library to improve usability.

Take `mystd::priority_queue` for example, the CTADs are as follows:
(`include/priority_queue.hpp` line 212–242)

```
1  template <class Comp, class Container>
2  priority_queue(Comp, Container)
3      -> priority_queue<typename Container::value_type, Container, Comp>;
4
5  template <
6      std::input_iterator InputIt,
7      class Comp = std::less<typename std::iterator_traits<InputIt>::value_type>,
8      class Container =
9          mystd::vector<typename std::iterator_traits<InputIt>::value_type>>
10 priority_queue(InputIt, InputIt, Comp = Comp(), Container = Container())
11     -> priority_queue<typename std::iterator_traits<InputIt>::value_type,
12                       Container, Comp>;
13
14 template <class Comp, class Container, class Alloc>
15 priority_queue(Comp, Container, Alloc)
16     -> priority_queue<typename Container::value_type, Container, Comp>;
17
18 template <std::input_iterator InputIt, class Alloc>
19 priority_queue(InputIt, InputIt, Alloc) -> priority_queue<
20     typename std::iterator_traits<InputIt>::value_type,
21     mystd::vector<typename std::iterator_traits<InputIt>::value_type>,
22     std::less<typename std::iterator_traits<InputIt>::value_type>>;
23
24 template <std::input_iterator InputIt, class Comp, class Alloc>
25 priority_queue(InputIt, InputIt, Comp, Alloc) -> priority_queue<
26     typename std::iterator_traits<InputIt>::value_type,
27     mystd::vector<typename std::iterator_traits<InputIt>::value_type>, Comp>;
28
29 template <std::input_iterator InputIt, class Comp, class Container, class Alloc>
30 priority_queue(InputIt, InputIt, Comp, Container, Alloc)
31     -> priority_queue<typename Container::value_type, Container, Comp>;
```

(四) Empty Base Optimization (EBO)

In C++, an empty class normally has size 1 to ensure unique addresses. EBO is a compiler optimization that allows empty base classes to occupy no additional storage when used as a base.

In my implementation, Empty Base Optimization (EBO) is applied within `_tuple_impl` in `include/tuple.hpp`, which underlies the internal recursive inheritance mechanism of `tuple`. The design begins with an empty base class for the recursion terminator: (line 26)

```
1 template <std::size_t I> struct _tuple_impl<I>
```

Then, the recursive inheritance is implemented as follows: (line 53–55)

```
1 template <std::size_t I, class Head, class... Tail>
2 struct _tuple_impl<I, Head, Tail...> : public _tuple_impl<I + 1, Tail...> {
3   using base = _tuple_impl<I + 1, Tail...>;
```

(五) Template Compile-Time Recursion

Templates can perform computation at compile time using recursion:

- Each template instantiation computes one step of the recursion.
- Recursion terminates at a base case specialization.
- Computation occurs entirely at compile time, producing constant expressions.

This technique is used extensively in `include/tuple.hpp` to implement functions that access elements, indices, element types, etc. An example is demonstrated by the **get functions in include/tuple.hpp** section. Some more examples are listed in **2-C-4-1 polymorphism 宣告 的 include/tuple.hpp**.

(六) std::declval template

`std::declval<T>()` defined in `<utility>` is a template intended exclusively for use in unevaluated contexts (such as within `decltype`, `sizeof`, or SFINAE expressions). It provides a fictional value of type `T` or `T&` without requiring that the type be constructible. This enables compile-time type deduction even for non-instantiable or non-default-constructible types.

`std::declval` is indispensable in implementing traits and SFINAE-based detection idioms. One example is as follows: (`include/tuple.hpp` line 204–209)

```
1   constexpr void swap(_tuple_impl &other) noexcept(
2       std::is_nothrow_swappable_v<Head> &&
3       noexcept(std::declval<base &>().swap(std::declval<base &>()))) {
4     mystd::swap(head, other.head);
5     static_cast<base &>(*this).swap(static_cast<base &>(other));
6   }
```

(七) Ref-Qualified Member Functions

C++11 introduced **ref-qualified** member functions. These allow a class to distinguish overloads based on whether the object is an lvalue (`&`) or an rvalue (`&&`).

This mechanism is widely used in the member functions overloads in STL to control move semantics and perfect forwarding. An example is demonstrated by the **get functions in include/tuple.hpp** section. Some more examples are listed in **2-C-4-1 polymorphism 宣告 的 include/tuple.hpp**.

(八) decltype

`decltype` is a compile-time operator that deduces the declared type of an expression without performing the expression's evaluation. Unlike `auto`, which uses assignment-like deduction rules, `decltype` preserves the exact type, including references and cv-qualifiers.

Given an expression `e`, the type deduced by `decltype(e)` depends on the expression category.

- If `e` is a unparenthesized id-expression naming a variable, then `decltype(e)` yields the declared type of the variable. Note that a parenthesized one is a lvalue expression.
- If `e` is an **lvalue** expression, `decltype(e)` yields `T&`.
- If `e` is an **xvalue**, `decltype(e)` yields `T&&`.
- If `e` is a **prvalue**, `decltype(e)` yields `T`.

`decltype(auto)` preserves the exact type of the initializer.

`decltype` is crucial in producing type traits or meta-functions in template metaprogramming in many templates in my implementation, and `decltype(auto)` is also used together with perfect forwarding and template overloads in my implementation. An example is demonstrated by the **get functions in include/tuple.hpp** section. Some more examples are listed in **2-C-4-1 polymorphism 宣告 的 include/tuple.hpp**.

(九) Curiously Recurring Template Pattern (CRTP)

The Curiously Recurring Template Pattern (CRTP) is a C++ idiom in which a class Derived inherits from a class template Base that takes Derived itself as a template parameter.

CRTP allows classes to achieve polymorphic behavior at compile time, combining interface abstraction, code reuse, and high performance. It is particularly suitable when runtime overhead of virtual functions is undesirable or when the set of derived classes is fixed at compile time.

(十) get functions in include/tuple.hpp

- Member functions of

```
template <std::size_t I, class Head, class... Tail>
struct _tuple_impl<I, Head, Tail...>
```

  : `get<J>` member functions provide compile-time element access and are overloaded for all value categories: lvalue, const lvalue, rvalue, and const rvalue. The implementation uses `if constexpr` to check whether the requested index J matches the current index I. This design allows efficient compile-time recursion to navigate tuple elements while preserving proper value category semantics without runtime overhead.

  Line 211–253:

```
constexpr decltype(auto) get() &
  requires(J >= I && J <= I + sizeof...(Tail))
{
  if constexpr (I == J) {
    return (head);
  } else {
    return static_cast<base &>(*this).template get<J>();
  }
}

template <std::size_t J>
constexpr decltype(auto) get() const &
  requires(J >= I && J <= I + sizeof...(Tail))
{
  if constexpr (I == J) {
    return (head);
  } else {
    return static_cast<const base &>(*this).template get<J>();
  }
}

template <std::size_t J>
constexpr decltype(auto) get() &&
  requires(J >= I && J <= I + sizeof...(Tail))
{
  if constexpr (I == J) {
    return std::move(head);
  } else {
    return static_cast<base &&>(*this).template get<J>();
  }
}

template <std::size_t J>
constexpr decltype(auto) get() const &&
  requires(J >= I && J <= I + sizeof...(Tail))
{
  if constexpr (I == J) {
    return std::move(head);
  } else {
    return static_cast<const base &&>(*this).template get<J>();
  }
}
```

- Member functions of `template <class... Types> class tuple`: The `tuple` class delegates all element access to its internal `_tuple_impl` member `_impl`. All public `get<J>` member functions forward the access while preserving constness and value categories.

  Line 466–480:

```
template <std::size_t J> constexpr decltype(auto) get() & noexcept {
  return _impl.template get<J>();
}

template <std::size_t J> constexpr decltype(auto) get() && noexcept {
  return std::move(_impl).template get<J>();
}

template <std::size_t J> constexpr decltype(auto) get() const & noexcept {
  return _impl.template get<J>();
}

template <std::size_t J> constexpr decltype(auto) get() const && noexcept {
  return std::move(_impl).template get<J>();
```

```
}
```

- Non-member functions and helpers: Non-member `get<I>` functions allow element access using ADL and provide interface for tuples complying with C++20 standard. Compile-time type traits, `_count_of_type` and `_index_of_type`, enable type-based element access for types that occur exactly once in the tuple. This guarantees type-safe retrieval while all computations occur at compile-time.

  Line 580–658:

```
1  template <std::size_t I, class... Types>
2  constexpr decltype(auto) get(tuple<Types...> &t) noexcept {
3    return t.template get<I>();
4  }
5
6  template <std::size_t I, class... Types>
7  constexpr decltype(auto) get(tuple<Types...> &&t) noexcept {
8    return std::move(t).template get<I>();
9  }
10
11 template <std::size_t I, class... Types>
12 constexpr decltype(auto) get(const tuple<Types...> &t) noexcept {
13   return t.template get<I>();
14 }
15
16 template <std::size_t I, class... Types>
17 constexpr decltype(auto) get(const tuple<Types...> &&t) noexcept {
18   return std::move(t).template get<I>();
19 }
20
21 template <class T, class... Types> struct _count_of_type;
22
23 template <class T>
24 struct _count_of_type<T> : std::integral_constant<std::size_t, 0> {};
25
26 template <class T, class Head, class... Tail>
27 struct _count_of_type<T, Head, Tail...>
28     : std::integral_constant<std::size_t,
29                              (std::is_same_v<T, Head> ? 1 : 0) +
30                                  _count_of_type<T, Tail...>::value> {};
31
32 template <class T, class... Types>
33 inline constexpr std::size_t _count_of_type_v =
34     _count_of_type<T, Types...>::value;
35
36 template <class T, class... Types> struct _index_of_type;
37
38 template <class T, class... Tail>
39 struct _index_of_type<T, T, Tail...> : std::integral_constant<std::size_t, 0> {
40 };
41
42 template <class T, class Head, class... Tail>
43 struct _index_of_type<T, Head, Tail...>
44     : std::integral_constant<std::size_t,
45                              1 + _index_of_type<T, Tail...>::value> {};
46
47 template <class T> struct _index_of_type<T> {};
48
49 template <class T, class... Types>
50 inline constexpr std::size_t _index_of_type_v =
51     _index_of_type<T, Types...>::value;
52
53 template <class T, class... Types>
```

```
constexpr decltype(auto) get(mystd::tuple<Types...> &t) noexcept
  requires(_count_of_type_v<T, Types...> == 1)
{
  return get<_index_of_type_v<T, Types...>>(t);
}

template <class T, class... Types>
constexpr decltype(auto) get(mystd::tuple<Types...> &&t) noexcept
  requires(_count_of_type_v<T, Types...> == 1)
{
  return get<_index_of_type_v<T, Types...>>(std::move(t));
}

template <class T, class... Types>
constexpr decltype(auto) get(const mystd::tuple<Types...> &t) noexcept
  requires(_count_of_type_v<T, Types...> == 1)
{
  return get<_index_of_type_v<T, Types...>>(t);
}

template <class T, class... Types>
constexpr decltype(auto) get(const mystd::tuple<Types...> &&t) noexcept
  requires(_count_of_type_v<T, Types...> == 1)
{
  return get<_index_of_type_v<T, Types...>>(std::move(t));
}
```

二十八、 2-D-1-1 演算法設計

程式執行效率相關請參見 **2-D-1-2 程式執行效率優化**；指標相關請參見 **2-B-1-2 指標成員處理** 中之各 **描述**；依照 STL C++20 標準實作的部分及其優化與巧思請參見本報告各章節與程式碼檔案；以下僅就 Heap 與 priority_queue 資料結構與演算法及 Quick sort 演算法進行描述。

(一) Heap 與 priority_queue 資料結構與演算法

Heap 是一種以完全二元樹為邏輯模型、並以連續儲存空間實作的資料結構，常用於需要快速取得最大或最小元素的情境，如優先佇列（`priority_queue`）。本實作以隨機存取疊代器（`random_access_iterator`）為基礎，提供一組符合 C++20 標準的 heap 演算法於 `include/algorithm-heap.hpp` 中，又以 `mystd::vector` 為預設底層容器，提供符合 C++20 標準的 `mystd::priority_queue` container adapter 於 `include/priority_queue.hpp` 中。

在結構表示上，heap 以陣列形式儲存元素。對於索引為 i 的節點，其父節點索引為

$$\left\lfloor \frac{i-1}{2} \right\rfloor,$$

左右子節點索引分別為 $2i+1$ 與 $2i+2$。本實作預設以 `std::less<>` 建立最大堆（max-heap），並提供接受比較子（`Compare`）的多載版本，使 heap 的排序準則可完全由使用者自訂。

1. push_heap

`push_heap` 用於在既有 heap 的尾端插入一個新元素後，恢復 heap 性質。實作上，新元素位於最後一個索引，演算法透過「向上調整」（sift-up / bubble-up），反覆與其父節點比較；若新元素依比較子具有更高優先序，則與父節點交換，直到抵達根節點或不再違反 heap 性質為止。其時間複雜度為 $O(\log n)$。

2. pop_heap

`pop_heap` 用於移除 heap 的堆頂元素。實作上，先將根節點與尾端元素交換，使原本的最大（或最小）元素被移動至尾端；接著對新的根節點執行向下調整（heapify-down），在左右子節點中選擇優先序較高者進行交換，直到 heap 性質恢復。其時間複雜度為 $O(\log n)$。

3. make_heap

`make_heap` 用於將任意排列的元素區間轉換為合法 heap。本實作採用自底向上的 heapify 策略，從最後一個非葉節點（索引 $\lfloor n/2 \rfloor - 1$）開始，依序對每個節點進行向下調整。相較於逐一插入並呼叫 `push_heap` 的 $O(n \log n)$ 總時間複雜度，此方法的時間複雜度為 $O(n)$，為建構 heap 的最佳解。此特性在 `priority_queue` 的建構子中尤為重要，能有效降低由大量初始資料建立優先佇列的成本。

4. sort_heap

`sort_heap` 假設輸入區間已滿足 heap 性質，透過反覆呼叫 `pop_heap`，將目前堆頂元素移至區間尾端，並逐步縮小 heap 範圍，最終得到一個依比較子排序完成的序列。其時間複雜度為 $O(n \log n)$，屬於不需額外記憶體的原地排序（in-place sort），但不具穩定性。

5. is_heap 與 is_heap_until

`is_heap` 用於檢查一個區間是否滿足 heap 性質。實作上，對每一個父節點檢查其與左右子節點之間是否符合比較子所定義的順序關係，最壞情況需檢查所有內部節點，時間複雜度為 $O(n)$。

`is_heap_until` 則在檢查過程中，一旦發現第一個違反 heap 性質的子節點，立即回傳其疊代器位置，否則回傳 `last`。

6. priority_queue

`mystd::priority_queue` 內部以一個隨機存取容器（預設為 `mystd::vector`）作為底層儲存結構，並完全透過上述 heap 演算法維護其不變量。在所有建構子中，當底層容器含有初始元素時，皆呼叫 `make_heap` 以確保正確初始化。

`push` 與 `emplace` 透過 `push_heap` 在插入後恢復 heap 性質；`pop` 則以 `pop_heap` 搭配容器尾端刪除完成移除操作；`top` 則直接回傳根節點，時間複雜度為 $O(1)$。

7. 設計總結

本實作將 heap 的所有核心操作以泛型演算法形式獨立實現，並配合 Concepts 與比較子設計，再由 `priority_queue` 組合使用，清楚區分資料結構介面與演算法細節，並確保符合 C++20 標準。

(二) Quick sort 演算法

Quick sort（快速排序）是一種基於「分割（partition）」與「分治（divide and conquer）」策略的比較式排序演算法。我在 `include/qsort.hpp` 的實作中以隨機存取疊代器（`random_access_iterator`）為前提，並透過 Concepts（`std::sortable`）在編譯期保證元素可比較與可交換，設計目標在於語意清楚、實作精簡，並與 C++ 標準庫風格一致。

1. 核心概念

Quick sort 的核心流程可分為三個步驟：

1. 選擇一個樞紐元素（pivot）。
2. 對區間進行分割，使所有小於 pivot 的元素位於其左側，大於等於 pivot 的元素位於其右側。
3. 對左右兩個子區間遞迴套用相同的排序流程。

在本實作中，pivot 固定選擇為區間尾端元素（`*(last - 1)`），以降低實作複雜度並避免額外儲存空間。

2. Partition（分割）實作方式

分割階段使用兩個疊代器：

- `i`：指向目前「小於 pivot 區間」的尾端。
- `j`：由 `first` 走訪至 `last - 1`，逐一檢查元素。

對於每一個 `*j`：

- 若 `*j < pivot`（或使用比較子 `comp(*j, pivot)`），則將 `*j` 與 `*i` 交換，並將 `i` 前進一格。

當走訪完成後，`i` 所指的位置即為 pivot 的最終正確位置，透過將 `*i` 與 pivot 交換，使得：

$$[\text{first}, i) < \text{pivot}, \quad *i = \text{pivot}, \quad (i, \text{last}) \geq \text{pivot}.$$

3. 遞迴結構

完成分割後，演算法遞迴處理：

- 左子區間：`[first, i)`
- 右子區間：`[i + 1, last)`

遞迴終止條件為區間長度小於等於 1，此時區間已自然有序。

4. 比較子版本

第二個多載版本接受使用者自訂的比較子 `Compare comp`，所有原本以 `<` 判斷的地方，皆改為 `comp(a, b)`。此設計與 `std::sort` 的介面設計保持一致。

5. 時間複雜度

- 平均時間複雜度：$O(n \log n)$：遞迴深度 $O(\log n)$，每一層分割成本 $O(n)$。
- 最壞時間複雜度：$O(n^2)$：當 pivot 選擇不佳（例如固定選尾端），遞迴深度可能退化為 n。

6. 演算法特性

- 原地排序（in-place sort）：僅使用遞迴呼叫堆疊，不需額外輔助陣列。
- 非穩定排序：相等元素在排序後的相對順序不保證維持。
- Pivot 選擇策略簡單：本實作未引入 median-of-three 或隨機化 pivot，故在理論最壞情況下可能退化。

7. 程式碼

Line 9–39：

```
1  template<std::random_access_iterator RandomIt>
2  requires std::sortable<RandomIt>
3  constexpr void qsort(RandomIt first, RandomIt last) {
4      if (first >= last || std::distance(first, last) <= 1) return;
5      RandomIt i = first;
6      for (RandomIt j=first; j<last-1; j++) {
7          if (*j < *(last - 1)) {
8            mystd::swap(*i, *j);
9            ++i;
10         }
11     }
12     mystd::swap(*i, *(last - 1));
13     qsort(first, i);
14     qsort(i+1, last);
15 }
16
17 template<std::random_access_iterator RandomIt, class Compare>
18 requires std::sortable<RandomIt, Compare>
19 constexpr void qsort(RandomIt first, RandomIt last, Compare comp) {
20     if (first >= last || std::distance(first, last) <= 1) return;
21     RandomIt i = first;
22     for (RandomIt j=first; j<last-1; j++) {
23         if (comp(*j, *(last - 1))) {
24           mystd::swap(*i, *j);
25           ++i;
26         }
27     }
28     mystd::swap(*i, *(last - 1));
29     qsort(first, i, comp);
30     qsort(i+1, last, comp);
31 }
```

二十九、 2-D-1-2 程式執行效率優化

下針對我使用的各程式執行效率優化的技巧先闡述概念與有無使用的差異再舉出，再指出在本專案之用途。

(一) Compile-time branching with if constexpr in pointer and iterator operations

The `if constexpr` statement allows compile-time branching based on type traits or constant expressions. Unlike a regular `if`, branches that evaluate to `false` are discarded during compilation, so no code is generated for them. This feature increases efficiency in pointer and iterator operations in several ways:

- **Elimination of runtime checks:** Operations such as `allocate`, `construct`, or `destroy` can adapt to whether an allocator provides a specific interface without incurring runtime conditional checks.
- **Faster paths for specific scenarios:** Optimized paths can be implemented for particular cases, such as trivially copyable elements, forward iterators, or contiguous iterators.
- **Optimized code paths:** Only the necessary branch is compiled, allowing the compiler to produce specialized, streamlined machine code for each type or iterator category.
- **Enabling inlining and further compile-time optimizations:** With `if constexpr`, the compiler sees a fully resolved control flow, enabling aggressive inlining and constant propagation.
- **Reduced template bloat and branching overhead:** Generic containers and iterators can handle multiple scenarios (e.g., const vs. non-const access, random access vs. circular access) without runtime penalties.

Many examples are demonstrated in **2-B-1-2 指標成員處理** 中的 **include/vector.hpp** and described in its **描述**.

(二) constexpr function

The constexpr specifier is introduced in C++11, and declares that it is possible to evaluate the value of the entities at compile time. Such entities can then be used where only compile time constant expressions are allowed (provided that appropriate function arguments are given).

Since C++20, many functions in STL is constexpr, notably many of the constructors and member functions of most containers. The list below is the specification of constexpr function in C++20, the standard my implementation complies with.

- A constexpr specifier used in the first declaration of a function or static data member implies inline. If any declaration of a function or function template has a constexpr specifier, then every declaration must contain that specifier.
- A function or function template can be declared constexpr. A function is constexpr-suitable if all following conditions are satisfied:
  - If it is a constructor or destructor, its class does not have any virtual base class.
  - Its return type (if exists) is a literal type.
  - Each of its parameter types is a literal type.
  - It is not a coroutine.
  - Its function body is = default, = delete, or does not enclose the following:
    - goto statements
    - statements with labels other than case and default
    - definitions of variables of non-literal types

    * definitions of variables of static or thread storage duration

- Except for instantiated constexpr functions, non-templated constexpr functions must be constexpr-suitable.
- For a non-constructor constexpr function that is neither defaulted nor templated, if no argument values exist such that an invocation of the function could be an evaluated subexpression of a core constant expression, the program is ill-formed, no diagnostic required.
- For a templated constexpr function, if no specialization of the function/class template would make the templated function constexpr-suitable when considered as a non-templated function, the program is ill-formed, no diagnostic required.
- An invocation of a constexpr function in a given context produces the same result as an invocation of an equivalent non-constexpr function in the same context in all respects, with the following exceptions:
  - An invocation of a constexpr function can appear in a constant expression.
  - Copy elision is not performed in a constant expression.
- constexpr constructor: On top of the requirements of constexpr functions, a constructor also needs to satisfy all following conditions to be constexpr-suitable:
  - Its function body is = delete or satisfies the following additional requirements:
    * Every non-variant non-static data member and base class subobject is initialized.
    * If the constructor is a delegating constructor, the target constructor is a constexpr constructor.
    * If the constructor is a non-delegating constructor, every constructor selected to initialize non-static data members and base class subobjects is a constexpr constructor.
  - The class does not have any virtual base class.

  For a constexpr constructor that is neither defaulted nor templated, if no argument values exist such that an invocation of the function could be an evaluated subexpression of the initialization full-expression of some object subject to constant expression, the program is ill-formed, no diagnostic required.
- constexpr destructor: On top of the requirements of constexpr functions, a destructor also needs to satisfy all following conditions to be constexpr-suitable:
  - For every subobject of class type or (possibly multi-dimensional) array thereof, that class type has a constexpr destructor.
  - The class does not have any virtual base class.

When a function is marked as `constexpr`, the compiler can evaluate its result during compilation if all arguments are known, producing a constant value in the generated code. This has several efficiency benefits:

- **Reduced runtime overhead:** Computations are moved from execution time to compilation, eliminating repeated calculations during program execution.
- **Improved inlining opportunities:** Compile-time results can be directly substituted into expressions, reducing function call overhead and enabling further compiler optimizations.
- **Enabling compile-time checks:** Conditions such as array bounds, template arguments, or static assertions can be verified at compile time, avoiding runtime errors.
- **Optimized memory layout:** Constant expressions can be embedded directly in read-only memory or used to size arrays and other structures efficiently.

Many examples are shown in **2-B-1-2 指標成員處理** 的 **include/vector.hpp**.

(三) Compile-time polymorphism-related optimization

請參見 **2-C-4-3 polymorphism 相關的 class 或 function 間的對應關係**。

(四) Named Return Value Optimization (NRVO)

Named Return Value Optimization (NRVO) is a compiler optimization that eliminates unnecessary copy or move constructions when returning a local object from a function.

If a function returns a named local variable matching the function's return type directly without parenthesis, the compiler is allowed to construct the object directly in the caller's storage, bypassing temporary objects.

(五) no_unique_address attribute

Introduced in C++20, `[[no_unique_address]]` attribute allows empty member objects to be optimized similarly to empty bases, reducing storage overhead. It is used in `include/vector.hpp` line 46:

```
[[no_unique_address]] Allocator alloc;
```

三十、 2-D-1-3 程式碼管理版本控制

在本專案中，我們採用 **Git** 作為程式碼版本控制工具，並使用 **GitHub** 作為遠端託管平台，以支持多人協作與專案管理。Git 以其分散式版本控制的特性，允許每位開發者在本地完整保存整個程式碼庫，這不僅使得開發者能夠離線進行提交與檢查，也確保即便遠端倉庫出現問題，本地版本仍可完整恢復。GitHub 則提供了遠端集中管理、協作審核及歷史追蹤功能，讓專案能夠在多個開發者之間順利運作。

利用 Git 的分支（branch）機制，我們可以針對不同功能、修正或實驗性功能建立獨立分支。每位組員可在自己的分支上進行開發，彼此的修改互不干擾，並避免直接影響主分支（main）的穩定性。完成開發後，透過 merge 或 rebase 將功能分支合併至主分支，Git 會自動檢測修改衝突，並提供差異比較工具以協助解決。

Git 提供多種便利工具與指令以強化程式碼管理，例如：

- `git clone`：從遠端倉庫複製完整專案到本地，建立初始工作環境。

- `git status`：顯示工作目錄與暫存區的狀態，檢查檔案是否被修改、暫存或尚未追蹤。
- `git add`：將檔案變更加入暫存區，準備提交。
- `git mv`：重新命名或移動檔案，Git 會正確追蹤檔案變更歷史。
- `git rm`：刪除檔案並同步移出版本控制。
- `git commit`：將暫存區內容提交至本地倉庫，形成可追蹤的版本紀錄。
- `git diff`：比較不同提交、分支或檔案狀態的差異，協助檢查修改內容。
- `git log`：檢視提交歷史、作者與提交訊息，便於回溯版本與審查變更。
- `git grep`：在倉庫版本控制下的檔案中搜尋關鍵字，方便快速定位程式碼。
- `git branch`：查看、建立或刪除分支，管理不同開發線路。
- `git checkout`：切換分支或還原特定檔案至指定提交狀態。
- `git merge`：將其他分支的修改合併到目前分支，可能需要處理衝突。
- `git rebase`：將一個分支的提交重新套用到另一分支之上，使提交歷史保持線性。
- `git stash`：暫存尚未提交的修改，清理工作目錄以便切換分支或進行其他操作。
- `git reset`：重設 HEAD 或暫存區狀態，常用於取消暫存或回退提交（需謹慎使用）。
- `git revert`：建立一個新的提交以反向還原指定提交，安全地修正已發布的錯誤。
- `git cherry-pick`：選取特定提交套用到另一分支，常用於單一錯誤修正或功能移植。
- `git tag`：為特定提交建立標籤，常用於版本發布與里程碑標記。
- `git fetch`：從遠端倉庫取得最新提交資訊，但不自動合併。
- `git pull`：從遠端取得最新提交並自動合併到目前分支。
- `git push`：將本地提交推送至遠端倉庫，與團隊成員同步進度。

相較於傳統版本控制方式（如手動備份、單一中央儲存庫或共享網路磁碟），Git 的優勢十分明顯：

- **高度分支靈活性**：允許多分支並行開發，減少程式碼覆寫與衝突機率。
- **完整歷史追蹤**：每次提交都包含作者、時間、修改內容，便於審查、回滾與責任追溯。
- **離線操作能力**：本地倉庫完整，開發者可離線編輯、提交與檢查程式碼，再於網路恢復時同步遠端。
- **協作效率高**：結合 GitHub，支援 pull request、code review 與 issue 管理，使多人協作更有條理。
- **衝突解決工具完善**：自動檢測 merge conflict，並提供比對與手動解決工具，降低人為錯誤。

在本專案中，我們也使用 GitHub CLI（`gh`）進行 GitHub 操作，例如：

- 建立、檢視及管理 pull request 和 issue。
- 透過命令列快速審核、合併分支以及發布 release。

- 自動化部分操作流程，提升團隊效率。

在團隊專案中，我負責教導組員如何使用 Git 與 GitHub CLI，包括：

- 建立分支、切換分支、合併分支。
- 提交與推送程式碼，保持本地與遠端同步。
- 使用 pull request、issue、review 流程管理功能合併。
- 使用 `git diff`、`git log` 與 `git status` 等進行程式碼檢查與追蹤修改。

此外，我也負責主要處理專案開發中最具挑戰性的部分——**merge conflict**。當多位組員在同一程式碼區塊進行修改時，Git 會自動標示衝突。我需要仔細分析各分支修改的意圖，判斷正確的合併方式，並手動解決衝突，以確保最終程式碼可以正常編譯與運行。在此過程中，我也與組員保持溝通，統一程式碼風格與命名規則，減少未來衝突發生的可能性。

另外，我負責撰寫 Linux 與 Windows 的 release scripts，根據老師要求將編譯完成的程式包裝為 zip 檔，並自動建立 GitHub release，以提供團隊與使用者下載最新穩定版本。

總結來說，透過 Git 與 GitHub 的版本控制與分支管理，我們的專案能夠在多人開發、頻繁修改與測試的情況下，保持程式碼的穩定性、完整性與可追溯性。這大幅提升了團隊的協作效率，降低了程式碼錯誤與重工風險，使整個開發流程更加順暢與可控。

三十一、 2-D-2-1 美工細節

我所設計的美工部分細節如下：

- **Tap notes**：使用 網路圖片。
- **Hold notes**：使用 網路圖片。
- **Hold notes 長按變色**：此效果由我設計，程式碼節錄自 `Renderer.hpp` 第 405 406 行：

```
float pulse = 0.7f + 0.3f * sin((float)holdPressedTime / 100.0f);
color = {0, static_cast<Uint8>(150 * pulse), 0, 255};
```

- **遊戲中的計分與資訊文字**：由我設計。
- **各 lane 特效**：基於 Anubis 吉祥物。
- **中央特效**：基於 GNU 與 GCC 吉祥物。
- **設定頁面**：由我設計，程式碼節錄自 `BeatRunner.cpp` 第 634 775 行：

```
void showSettings(SDL_Renderer *renderer, SDL_Window *window) {
  bool running = true;
  SDL_Event e;

  SDL_Color white = {255, 255, 255, 255};
  SDL_Color grey = {100, 100, 100, 255};
  SDL_Color blue = {0, 128, 255, 255};

  SDL_Rect lanesMinus = {310, 80, 40, 40};
  SDL_Rect lanesPlus = {410, 80, 40, 40};
  SDL_Rect fragmentsMinus = {310, 140, 40, 40};
  SDL_Rect fragmentsPlus = {410, 140, 40, 40};
  SDL_Rect modDropdown = {310, 200, 850, 40};
```

```
14  SDL_Rect okButton = {SCREEN_WIDTH / 2 - 50, 280, 100, 50};
15
16  std::vector<std::string> modKeys;
17  for (auto &p : getModMap())
18    modKeys.push_back(p.first);
19  int selectedModIndex = 0;
20
21  bool dropdownOpen = false;
22
23  while (running) {
24    while (SDL_PollEvent(&e)) {
25      if (e.type == SDL_QUIT ||
26          (e.type == SDL_KEYDOWN && !e.key.repeat &&
27           (e.key.keysym.mod & KMOD_CTRL) && e.key.keysym.sym == SDLK_w)) {
28        destroyMainMenuTextures();
29        delete chartParser;
30        delete musicManager;
31        SDL_DestroyRenderer(renderer);
32        SDL_DestroyWindow(window);
33        TTF_CloseFont(large_font);
34        TTF_CloseFont(medium_font);
35        TTF_CloseFont(small_font);
36        TTF_Quit();
37        IMG_Quit();
38        SDL_Quit();
39        exit(0);
40      }
41
42      if (e.type == SDL_MOUSEBUTTONDOWN && e.button.button == SDL_BUTTON_LEFT) {
43        int mx = e.button.x, my = e.button.y;
44        if (pointInRect(mx, my, lanesMinus)) {
45          if (LANES > 1)
46            --LANES;
47        } else if (pointInRect(mx, my, lanesPlus)) {
48          if (LANES < 9)
49            ++LANES;
50        } else if (pointInRect(mx, my, fragmentsMinus)) {
51          if (FRAGMENTS > 2)
52            --FRAGMENTS;
53        } else if (pointInRect(mx, my, fragmentsPlus)) {
54          if (FRAGMENTS < 100)
55            ++FRAGMENTS;
56        } else if (pointInRect(mx, my, modDropdown)) {
57          dropdownOpen = !dropdownOpen;
58        } else if (pointInRect(mx, my, okButton)) {
59          running = false;
60        }
61
62        if (dropdownOpen) {
63          for (int i = 0; i < (int)modKeys.size(); ++i) {
64            SDL_Rect itemRect = {modDropdown.x, modDropdown.y + 40 * (i + 1),
65                                 modDropdown.w, 40};
66            if (pointInRect(mx, my, itemRect)) {
67              selectedModIndex = i;
68              MOD = modKeys[i];
69              dropdownOpen = false;
70            }
71          }
72        }
73      }
74    }
75
76    SDL_SetRenderDrawColor(renderer, 0, 0, 0, 255);
77    SDL_RenderClear(renderer);
78
```

```
    renderRoundedRect(renderer, lanesMinus, 10, blue);
    renderRoundedRect(renderer, lanesPlus, 10, blue);
    renderRoundedRect(renderer, fragmentsMinus, 10, blue);
    renderRoundedRect(renderer, fragmentsPlus, 10, blue);
    renderRoundedRect(renderer, modDropdown, 10, grey);
    renderRoundedRect(renderer, okButton, 15, blue);

    renderText(renderer, medium_font, "LANES:", 50, 100, white, ALIGN_LEFT);
    renderText(renderer, medium_font, "FRAGMENTS:", 50, 160, white, ALIGN_LEFT);
    renderText(renderer, medium_font, "MOD:", 50, 220, white, ALIGN_LEFT);

    renderText(renderer, medium_font, std::to_string(LANES), 380, 105, white);
    renderText(renderer, medium_font, std::to_string(FRAGMENTS), 380, 165,
               white);
    renderText(renderer, small_font, MOD, 330, 220, white, ALIGN_LEFT);

    renderText(renderer, medium_font, "-", lanesMinus.x + 20, lanesMinus.y + 20,
               white);
    renderText(renderer, medium_font, "+", lanesPlus.x + 20, lanesPlus.y + 20,
               white);
    renderText(renderer, medium_font, "-", fragmentsMinus.x + 20,
               fragmentsMinus.y + 20, white);
    renderText(renderer, medium_font, "+", fragmentsPlus.x + 20,
               fragmentsPlus.y + 20, white);
    renderText(renderer, medium_font, "OK", okButton.x + 49, okButton.y + 27,
               white);

    if (dropdownOpen) {
      for (int i = 0; i < (int)modKeys.size(); ++i) {
        SDL_Rect itemRect = {modDropdown.x, modDropdown.y + 40 * (i + 1),
                             modDropdown.w, 40};
        renderText(renderer, small_font, modKeys[i], itemRect.x + 10,
                   itemRect.y, white, ALIGN_LEFT);
      }
    }

    SDL_RenderPresent(renderer);
  }

  if (chartParser->load(selectedChartPath)) {
    // std::cout << "[OK] Chart loaded successfully: " << selectedChartPath <<
    // std::endl;
    mouseNotesPtr = &chartParser->getMouseNotes();
    musicManager->loadMusic(chartParser->getMusicFile());
  } else {
    std::cerr << "[ERROR] Failed to load chart" << std::endl;
  }

  exactFragmentMs = 60000.0f / (double)chartParser->getBPM() /
                    (double)chartParser->getFragmentsPerBeat();
  MS_PER_FRAGMENT = (uint32_t)exactFragmentMs;
  if (game)
    delete game;
  game = new Game(LANES, FRAGMENTS, MS_PER_FRAGMENT, keyNotes);
  if (gameRenderer)
    delete gameRenderer;
  gameRenderer = new Renderer(*game, SCREEN_WIDTH, SCREEN_HEIGHT, renderer,
                              large_font, medium_font, small_font);

  SettingsFunc modSettingsFuncLocal = mystd::get<2>(getModMap()[MOD]);
  if (modSettingsFuncLocal) {
    modSettingsFuncLocal(renderer, small_font, SCREEN_WIDTH, SCREEN_HEIGHT);
  }
}
```

- **康威生命遊戲模組設定頁面**：由我設計，程式碼節錄自 BeatRunner.cpp 第 103 253 行：

```
void gameOfLifeSettings(SDL_Renderer *renderer, TTF_Font *font, int screenWidth,
                        int screenHeight) {
  mystd::array<uint16_t, 2> tempRules = currentRules;

  const int buttonSize = 40;
  const int buttonSpacing = 5;
  const int rowSpacing = 60;

  int totalWidth = 9 * buttonSize + 8 * buttonSpacing;
  int startX = (screenWidth - totalWidth) / 2;
  int row1Y = screenHeight / 2 - rowSpacing;
  int row2Y = screenHeight / 2 + rowSpacing;

  const int okButtonWidth = 100;
  const int okButtonHeight = 50;
  int okButtonX = (screenWidth - okButtonWidth) / 2;
  int okButtonY = row2Y + rowSpacing * 2;

  bool settingsRunning = true;
  bool redrawNeeded = true;

  while (settingsRunning) {
    SDL_Event event;
    while (SDL_PollEvent(&event)) {
      if (event.type == SDL_QUIT) {
        settingsRunning = false;
        break;
      } else if (event.type == SDL_KEYDOWN) {
        if (event.key.keysym.sym == SDLK_RETURN ||
            event.key.keysym.sym == SDLK_ESCAPE) {
          currentRules = tempRules;
          settingsRunning = false;
        }
      } else if (event.type == SDL_MOUSEBUTTONDOWN) {
        if (event.button.button == SDL_BUTTON_LEFT) {
          int mouseX = event.button.x;
          int mouseY = event.button.y;

          for (int i = 0; i < 9; ++i) {
            int buttonX = startX + i * (buttonSize + buttonSpacing);
            SDL_Rect buttonRect = {buttonX, row1Y, buttonSize, buttonSize};

            if (mouseX >= buttonRect.x &&
                mouseX <= buttonRect.x + buttonRect.w &&
                mouseY >= buttonRect.y &&
                mouseY <= buttonRect.y + buttonRect.h) {
              tempRules[0] ^= (1 << i);
              redrawNeeded = true;
            }
          }

          for (int i = 0; i < 9; ++i) {
            int buttonX = startX + i * (buttonSize + buttonSpacing);
            SDL_Rect buttonRect = {buttonX, row2Y, buttonSize, buttonSize};

            if (mouseX >= buttonRect.x &&
                mouseX <= buttonRect.x + buttonRect.w &&
                mouseY >= buttonRect.y &&
                mouseY <= buttonRect.y + buttonRect.h) {
              tempRules[1] ^= (1 << i);
              redrawNeeded = true;
            }
          }

```

```
        SDL_Rect okRect = {okButtonX, okButtonY, okButtonWidth,
                           okButtonHeight};
        if (mouseX >= okRect.x && mouseX <= okRect.x + okRect.w &&
            mouseY >= okRect.y && mouseY <= okRect.y + okRect.h) {
          currentRules = tempRules;
          settingsRunning = false;
        }
      }
    }
  }

  if (redrawNeeded) {
    SDL_SetRenderDrawColor(renderer, 30, 30, 40, 255);
    SDL_RenderClear(renderer);

    for (int i = 0; i < 9; ++i) {
      int buttonX = startX + i * (buttonSize + buttonSpacing);
      SDL_Rect buttonRect = {buttonX, row1Y, buttonSize, buttonSize};

      bool enabled = (tempRules[0] >> i) & 0b1;

      if (enabled) {
        SDL_SetRenderDrawColor(renderer, 100, 200, 100, 255);
      } else {
        SDL_SetRenderDrawColor(renderer, 200, 100, 100, 255);
      }
      SDL_RenderFillRect(renderer, &buttonRect);

      SDL_SetRenderDrawColor(renderer, 255, 255, 255, 255);
      SDL_RenderDrawRect(renderer, &buttonRect);

      SDL_Color numColor = {255, 255, 255, 255};
      renderText(renderer, std::to_string(i), buttonX + buttonSize / 2,
                 row1Y + buttonSize / 2, font, numColor, ALIGN_CENTER);
    }

    for (int i = 0; i < 9; ++i) {
      int buttonX = startX + i * (buttonSize + buttonSpacing);
      SDL_Rect buttonRect = {buttonX, row2Y, buttonSize, buttonSize};

      bool enabled = (tempRules[1] >> i) & 0b1;

      if (enabled) {
        SDL_SetRenderDrawColor(renderer, 100, 200, 100, 255);
      } else {
        SDL_SetRenderDrawColor(renderer, 200, 100, 100, 255);
      }
      SDL_RenderFillRect(renderer, &buttonRect);

      SDL_SetRenderDrawColor(renderer, 255, 255, 255, 255);
      SDL_RenderDrawRect(renderer, &buttonRect);
      SDL_Color numColor = {255, 255, 255, 255};
      renderText(renderer, std::to_string(i), buttonX + buttonSize / 2,
                 row2Y + buttonSize / 2, font, numColor, ALIGN_CENTER);
    }

    SDL_Rect okRect = {okButtonX, okButtonY, okButtonWidth, okButtonHeight};
    SDL_SetRenderDrawColor(renderer, 100, 150, 255, 255);
    SDL_RenderFillRect(renderer, &okRect);
    SDL_SetRenderDrawColor(renderer, 255, 255, 255, 255);
    SDL_RenderDrawRect(renderer, &okRect);

    SDL_Color textColor = {255, 255, 255, 255};
    std::string surviveText = "Survive with neighbors:";
    std::string reviveText = "Revive with neighbors:";
```

```

      int surviveTextX = screenWidth / 2;
      int surviveTextY = row1Y - 40;
      renderText(renderer, surviveText, surviveTextX, surviveTextY, font,
                 textColor, ALIGN_CENTER);

      int reviveTextX = screenWidth / 2;
      int reviveTextY = row2Y - 40;
      renderText(renderer, reviveText, reviveTextX, reviveTextY, font,
                 textColor, ALIGN_CENTER);

      std::string okText = "OK";
      int okTextX = okButtonX + okButtonWidth / 2;
      int okTextY = okButtonY + okButtonHeight / 2;
      renderText(renderer, okText, okTextX, okTextY, font, textColor,
                 ALIGN_CENTER);

      SDL_RenderPresent(renderer);
      redrawNeeded = false;
    }
  }
}
```

- **倒數動畫**：GO 頁面的圖片使用 The Go Programming Language 吉祥物，其餘部分由我設計。

三十二、 2-D-2-2 音樂細節

本專題之音樂非自行創作，又本人不負責音樂處理；本人實作之部分與音樂或有關連者僅誤差修正系統，請參見 **誤差修正系統**。